IMPERIAL

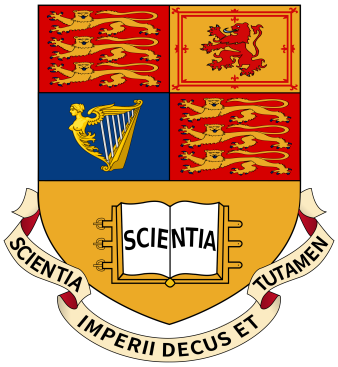
Imperial College London
Department of Computing

Discrete Neural Flow Samplers with Constraints for Materials Discovery

Author:
Mitchell Crabb

Co-supervisors:
Dr Yingzhen Li
Dr Alexander Ganose
Zijing Ou

Second Marker:
Dr Joey Bose



Submitted in partial fulfilment of the requirements for the MSc degree in *Artificial Intelligence* of Imperial College London

September 2026

In der Beschränkung zeigt sich erst der Meister,
Und das Gesetz nur kann uns Freiheit geben.

In limitation the master first shows himself,
and law alone can give us freedom.

– Johann Wolfgang von Goethe, *Natur und Kunst* (1800); author's translation

Abstract

Efficient study of the thermodynamic properties of materials often necessitates the use of ensemble sampling, where samples are taken from a surrogate model to replace costly alternatives. Practitioners are burdened by classical Markov chain Monte Carlo (MCMC)-based methods, which struggle with critical slowing-down and mode trapping as problem sizes grow and the landscape becomes more ordered. These methods also fail to amortise, which becomes a problem when the object of study requires a sweep over compositions, for example a free-energy curve, which is necessary for studying phase stability. Here you'd require one MCMC chain *per composition*, followed by thermodynamic integration. Learned neural samplers offer an alternative, offering an interesting route to overcoming these shortcomings. Samplers in this family, for the most part, remain focussed on the unconstrained problem. In this thesis however, our main goal is to tackle the exact-constrained problem, where our sampler must provide draws that hit a specific condition with no room for error. We show that through a combination of existing neural methods and a half-century-old classical move set, we're able to enforce the constraint as a law, holding at every step along the trajectory. Not only that, we find that constructing such a sampler is competitive against both the classical and neural methods on Ising models and a Cu–Au cluster expansion. One sampler, compositions hit exactly, amortised, with canonical free energies on top. Quite a landscape to traverse.

Acknowledgements

I'd first like to thank my supervisors: Dr Yingzhen Li, Dr Alexander Ganose, and Zijing Ou, for their support and guidance throughout this thesis. Their ability to consistently ask the right questions and lead me toward interesting avenues for problem solving has been invaluable.

I'd also like to thank my second marker, Dr Joey Bose, for his feedback during the early stages of this project. His advice to read more widely and consider applications outside of my initial scope has helped me become a more mature researcher.

What a blessing the Imperial DoC GPU cluster has been throughout the thesis – without it I'd either have very few experimental results to share, or have much lighter pockets. If this project has taught me one thing, it's that access to frontier-level compute is a must-have for deep learning based projects, and for that I'm truly grateful.

Thanks also go out to the maintainers of the fantastic open-source software that this project is built on, including the research repositories that accompany their respective papers. We really are standing on the shoulders of giants, and each incremental contribution makes us all a little taller. The table below gives an idea of the software used throughout, and while my citations give a good indication of the papers I enjoyed reading over the last few months, it is by no means extensive.

scientific Python	PyTorch, NumPy, SciPy, numba, pandas, matplotlib, Jupyter
materials stack	ASE, icet, mchammer, CLEASE
research code	DNFS[†], LEAPS[†], MDNS, MetaDNS
tooling & infrastructure	pixi, pytest, ruff, git / GitHub, wandb, Modal, DoC Slurm cluster, LaTeX / Overleaf

[†] reference repositories consulted for structure only; implementations follow the papers' equations.

Declarations

Generative AI Statement

Generative AI tools were used in a supporting capacity for prose-level editing of report drafts, TikZ figures, tests, and general programming assistance. All methodology, experimental design, analysis, claims, conclusions, and mistakes are my own.

Ethical Considerations

The work included in this report is purely methodological, and involves no human subjects or personal, sensitive, or private data to obtain results. The main ethical considerations here are environmental impact and reproducibility, which are covered in the sections that follow.

The methodology itself (constrained neural samplers for discrete probability distributions) is somewhat application agnostic. Plausible uses include materials discovery and probabilistic inference; however, future applications would require a case-by-case ethical review.

To ensure research integrity, any codebases accompanying published papers cited here were consulted only for structural and engineering inspiration, with my implementations derived directly from the paper's equations and algorithmic descriptions.

Computational Resources and Sustainability

Given the experimental nature of this research project, care had to be taken to ensure computational footprint was optimised. All experiments used a single GPU per run, with no distributed parallelism. The `NVIDIA L4` and `NVIDIA A30` were reserved for smaller experiments, while the `A100-40&80GB` instances were used in cases where the higher memory and compute were required. A handful of runs took place on `NVIDIA H100`, `NVIDIA H200`, and `NVIDIA B200`, with multi-process service memory allocation enabled where necessary to ensure optimal balancing.

Experiments used either DoC or Modal training infrastructure, allowing for shared on-demand allocation, so no dedicated hardware sat idle between runs. The latter was used as an ad-hoc backup when the DoC cluster was busy. The total compute used across all reported experiments was around `2000` GPU-hours. Almost all experiments were run multi-seed, which is an accepted trade-off for a multiplicative factor on the base compute.

We can make some assumptions to convert this to a naive (excluding external factors) emissions figure. If we take the rated board power of the `A100` since it was the most frequently used card (0.40 kW per GPU-hour, an overestimate for the smaller cards and vice versa for the larger ones) and a data centre power-usage effectiveness of 1.2, the compute corresponds to roughly 960 kWh;[1] this is approximately 138 kg CO_2e, comparable to 860 km of direct emissions from an average petrol car [1]. This scenario is an indicative UK-based one, and makes no claim that the hardware used actually drew from the UK grid.

Availability of Data and Materials

All data and code shown here will be made publicly available at `https://github.com/m-crabb/msc-dissertation-dnfs` at the time of submission of the final dissertation report.

[1] using the 2026 UK electricity factor of 0.14395 kg CO_2e per kWh (generation plus transmission and distribution)

Contents

1 Introduction **1**
- 1.1 Motivation . . . 1
- 1.2 Formal Problem Statement . . . 2
- 1.3 Thesis and Contributions . . . 2

2 Background **3**
- 2.1 The Sampling Problem . . . 3
  - 2.1.1 Boltzmann distributions and notation . . . 3
  - 2.1.2 Classical MCMC for discrete distributions . . . 3
  - 2.1.3 The Ising model as a running example . . . 4
  - 2.1.4 Evaluating samplers . . . 5
- 2.2 Neural Samplers for Discrete Distributions . . . 7
  - 2.2.1 Generative modelling as learned transport . . . 7
  - 2.2.2 Continuous-time Markov chains . . . 8
  - 2.2.3 A survey of existing discrete neural samplers . . . 9
- 2.3 Constrained Sampling for Materials Configurations . . . 10
  - 2.3.1 Constraint types and handling strategies . . . 10
  - 2.3.2 Constrained sampling with Kawasaki dynamics . . . 11
  - 2.3.3 Cluster expansions and alloy thermodynamics . . . 11
  - 2.3.4 Sampling cluster expansions in practice . . . 12
  - 2.3.5 Constrained neural samplers . . . 12
- 2.4 Summary and Research Gap . . . 13

3 The Unconstrained Baseline **14**
- 3.1 Discrete Neural Flow Samplers . . . 14
  - 3.1.1 Training objective . . . 14
  - 3.1.2 Variance reduction techniques . . . 15
  - 3.1.3 Hollow and locally equivariant architecture . . . 16
- 3.2 Experimental Setup . . . 18
- 3.3 Experimental Results . . . 18
- 3.4 Rejection Sampling . . . 20
- 3.5 Chapter Summary . . . 21

4 Constrain with a Penalty **22**
- 4.1 The Penalised Target . . . 22
- 4.2 Experimental Setup . . . 23
- 4.3 Tackling Penalty Variance . . . 23
  - 4.3.1 Closed-form residual of the penalty . . . 24
  - 4.3.2 A matched base for off-centre targets . . . 25
  - 4.3.3 Penalty strength . . . 26
- 4.4 Headline Results . . . 26
  - 4.4.1 Enumerable check . . . 26
  - 4.4.2 The 64 site lattice . . . 26
  - 4.4.3 Against the variance-constrained semi-grand-canonical (VCSGC) ensemble 27
- 4.5 Amortisation and Transfer . . . 27
- 4.6 The Cost of Exact Composition . . . 28
  - 4.6.1 The Distributional Ceiling . . . 28

4.6.2 Rejection Sampling . . . 28
4.7 Chapter Summary . . . 30

5 Constrain by Construction **31**
5.1 The Canonical Target . . . 31
5.1.1 From penalty to indicator . . . 31
5.1.2 Kawasaki dynamics and critical slowing-down . . . 32
5.2 A Neural Approach to Swaps . . . 33
5.2.1 The on-manifold base and path . . . 33
5.2.2 A pair potential antisymmetric under the swap . . . 34
5.2.3 The swap log-ratio in closed form . . . 35
5.2.4 Resolving swap conflicts . . . 36
5.3 A Multi-Headed Approach . . . 37
5.3.1 Blindness by order: the causal-stream heads . . . 37
5.3.2 A convolutional approach: the two-hole patch head . . . 41
5.3.3 Engineering optimisations . . . 42
5.4 Experimental Setup . . . 43
5.5 Experimental Results . . . 44
5.5.1 Enumerable check . . . 44
5.5.2 The 64 site lattice . . . 45
5.5.3 Scaling the sampler, from 16×16 to 24×24 . . . 45
5.5.4 Visualising the trained generator . . . 47
5.6 Transfer and Amortisation . . . 48
5.6.1 Pathwise coupling transfer . . . 48
5.6.2 Amortising across compositions . . . 49
5.7 Free-Energy Curve . . . 51
5.8 From Binary Models to Binary Alloys . . . 52
5.9 Chapter Summary . . . 53

6 Discussion and Conclusion **54**
6.1 A Trident of Neural Samplers . . . 54
6.2 Scope and Limitations . . . 54
6.3 Future Work . . . 55

Bibliography **56**

Glossary of Acronyms **65**
A Initial Project Description **67**
B Kolmogorov Forward Equation Derivation **68**
C Discrete Stein Control Variate **69**
D Training Algorithms **72**
E Proof of Soft Constraint Variance **74**
F Supplementary Soft Results **76**
G Budget-Masked Reference Conditionals **78**
H Training Curves **79**
I Validation at the Enumerable Size **80**
J Neural Swap Graveyard **81**
K Supplementary Hard Results **82**
L Hyperparameter Soup **86**

1 Introduction

1.1 Motivation

What an interesting time for scientific discovery. It feels like only yesterday that the Erdős unit-distance conjecture got disproved [2]; weeks later the Jacobian conjecture suffered the same fate in dimension three and above [3]. Not long after that, OpenAI reported ten further significant advances in mathematics [4], and as I conclude the writing of this thesis, rumours are circulating that progress has been made towards solving the Navier-Stokes Millennium Prize Problem [5]. It feels like breakthroughs of this magnitude are happening all the more frequently in recent times – perhaps the lofty projections of AI labs are not too far from reality [6]. It's not only mathematics problems that are getting attention, there are plenty such examples if you know where to look. Protein structure prediction went from *painfully* open to solved thanks to Jumper et al. [7], and deep learning has even helped researchers identify and experimentally test new classes of antibiotics [8]. The limelight has shone on structural biology; it feels like time for such an event in materials. We're not asking for much, not even a Nobel Prize. Simply an effective constrained neural sampler for Boltzmann distributions.

Materials scientists are often interested in predicting thermodynamic properties of disordered solids, for example substitutional alloys such as Cu–Au, which require sampling configurations from their finite-temperature statistical ensemble [9, 10], an unnormalised Boltzmann distribution. Free energy of mixing helps determine stability of phases, while atomic ordering can affect mechanical properties and catalytic behaviour in alloys [11, 12]. Research is hotting up in the area, both on the on the academic and the commercial side, to try to combine the progress in artificial intelligence (AI) and machine learning (ML) to these types of problems. There are dedicated programmes at many of the leading labs – Google DeepMind [13], Microsoft [14] and Meta [15], alongside a push on the startup side founded by alumni – CuspAI ($450M, 2026) [16], Periodic Labs ($300M seed) [17] and Orbital Materials [18] are a handful of examples.

It's not that the tools for sampling these distributions don't exist – they do, but not without issues that would keep us all up at night. Combinatorial optimisation is ruled out since we need distributional properties, where they can only provide a point estimate. Instead we turn to the Markov chain Monte Carlo (MCMC) family, from Metropolis and Gibbs updates to more recent algorithms that leverage gradients [19, 20], though it isn't without fault, and has three well-documented failure modes. Single-site chains can suffer from critical slowing-down and mode trapping; these can be alleviated with collective moves [21, 22] and replica exchange or flat-histogram methods [23, 24], which provide better mixing at the cost of running more chains or energy evaluations per sample. To conserve composition we can move from single flips to Kawasaki exchange [25], and to hold the composition on average, we can turn to a semi-grand-canonical ensemble, which involves a tuned chemical-potential bias [26]. These point to our third issue, lack of amortisation. Each target, temperature, and constraint can require a fresh chain with its own burn-in. We'll refer to these collective methods as *classical* throughout.

The rapidly accelerating field of AI and ML is working to overcome the deficiencies of the classical methods, and lots of exciting progress has been made in the adjacent fields of discrete diffusion [27, 28] and energy-based sampling [29, 30, 31, 32], though many of the energy driven methods target the unconstrained problem, and can't natively satisfy the hard constrained problems required for fixed-composition alloy sampling. Methods are starting to emerge; the recent JANUS preprint [33] tackles this problem using budget-masked diffusion to reach fixed composition, and is able to read free energy from its path weights. Its constraint is a property of the final configuration, so each step along the denoising path need not conform to it. A

more well-established route for this problem class is generative flow networks (GFlowNets) [34], which learn policies on a directed acyclic graph (DAG) of partial configurations, can handle the constraints by construction. However, training these models is notoriously challenging; they train well on short trajectories, but start to struggle with credit assignment as these grow [35]. We discuss both throughout, and use the latter as a neural comparator in Chapter 5.

1.2 Formal Problem Statement

Given a lattice of d sites taking one of S values, and an energy function $E(x)$ derived from physical interactions, the goal is to draw configurations from our target distribution π satisfying:

$$\pi(x) = \frac{1}{Z}\exp[-\beta E(x)], \qquad x \in \mathcal{X} \cap \mathcal{C}, \qquad Z = \sum_{x \in \mathcal{X} \cap \mathcal{C}} \exp[-\beta E(x)], \tag{1.1}$$

where $\mathcal{X} = \{0, \ldots, S-1\}^d$ is the unconstrained discrete state space, $\mathcal{C} \subseteq \mathcal{X}$ is the set of states satisfying some constraint (fixed composition for example), and β the *inverse temperature*, which controls the contrast between system configurations. Large β concentrates mass on low-energy states; small β flattens towards uniform. The constraints, high-dimensional and multi-modal target, and combinatorial state space together make this a rich and challenging research problem.

1.3 Thesis and Contributions

Our thesis is fairly simple to state – there exists a classical solution to fixed-composition sampling, the Kawasaki swap, but it shares the same deficiencies as its other MCMC neighbours. Neural samplers offer an interesting alternative, and we posit that an extension of an existing sampler to mirror the Kawasaki swap will enable us to learn the swap dynamics as a route to targeting the constraints. Roughly speaking, start your journey towards the hard target on a distribution which respects your constraints, simulate swaps at the rate matrix level of your continuous-time Markov chain (CTMC), and they'll remain respected throughout the trajectory.

We present our contributions using a *trident* of discrete neural samplers, inspired by the tools used by materials researchers. Our journey begins in Chapter 3 with a replication of discrete neural flow sampler (DNFS), evaluated against exact and Monte Carlo references, on the unconstrained target – analogous to the semi-grand-canonical (SGC) ensemble at zero chemical-potential difference. In Chapter 4, we apply this replicated sampler to the soft constrained case, targeting the variance-constrained semi-grand-canonical (VCSGC) ensemble, where we study the training adjustments needed for this target, amortisation across compositions, and just how far off we remain from the hard target. Our main contribution lives on the hard constrained route in Chapter 5, where we target the canonical ensemble. We keep the learned sampler's dynamics, replace its single-site moves with composition-preserving swaps, and let the constraint be a conservation law of the move set such that it's satisfied at every step. This is tackled first by developing the necessary theory, and then constructing myriad architectures to enable this to be computed efficiently, and testing its sampling fidelity, cost, and amortisation on fixed-composition Ising models and a Cu–Au cluster expansion. Oh, and we extract the free energy!

The chapters throughout are designed to be relatively self-contained, with Chapter 2 laying the foundations for this three-pronged approach to the study of discrete neural samplers. Our focus then narrows specifically to DNFS for the chapters that remain, though Chapter 3, and Chapter 4 are both sampler agnostic in as much as once you have a replicated unconstrained sampler, the soft case follows naturally. We conjecture that the hard constrained work of Chapter 5 transfers to other samplers in the family, though given the involved nature of the derivations and engineering, this remains to be seen and is left to future work (Section 6.3).

2 Background

2.1 The Sampling Problem

2.1.1 Boltzmann distributions and notation

The problem we'll be tackling throughout this thesis is arguably one of the most famous in probabilistic machine learning – draw samples from a Boltzmann distribution where we cannot analytically solve for the normalising constant, since it would require taking an intractable sum over all states. This appears in many problem settings, including Bayesian inference, statistical physics, computational biology, and materials sampling.

In the general unconstrained discrete case we seek to sample from the target distribution $\pi(x)$:

$$\pi(x) = \frac{1}{Z}\exp[-\beta E(x)], \qquad x \in \mathcal{X} = \{0, \dots, S-1\}^d, \qquad Z = \sum_{x\in\mathcal{X}} \exp[-\beta E(x)]. \tag{2.1}$$

Were E is the energy, and β the inverse temperature as defined in Chapter 1. Our working assumption is that $E(x)$ can be evaluated at any state but Z cannot, so π is known only up to its normalising constant. We'll start our studies with this target, and move toward the constrained variant mentioned in Chapter 1 in Section 2.3.

The traditional tools used in sampling here are the MCMC family of algorithms, which sample without ever needing to evaluate Z (Section 2.1.2). The same unnormalised Boltzmann form also appears in machine learning as the energy-based model, where E_θ is learned from data [36, 37]. In this thesis however, we derive our energy E from physics, either in closed form or derived from a cluster expansion, fitted to density functional theory (DFT).

2.1.2 Classical MCMC for discrete distributions

The MCMC family of algorithms is favoured in settings where we have an intractable normalising constant, since it constructs a Markov chain whose *stationary* distribution is the target π, and transitions between states are compared through ratios $\pi(x')/\pi(x)$, banishing the normalising constant. Formally, stationarity means if we represent π as a vector of probabilities and P the transition matrix associated with our Markov Chain, we have that $\pi = P\pi$, i.e. π is a fixed point of P. Unlike later-mentioned annealing-based methods which start at an easy distribution and progress towards the target, MCMC methods start at an arbitrary state, and we converge *towards* π, provided the chain is irreducible and aperiodic [38].

There are many variants to choose from – a particularly popular option is Metropolis-Hastings, which proposes a sample x' from its proposal $q(x' \mid x)$, and accepts with probability $\min\big(1, \frac{\pi(x')\,q(x|x')}{\pi(x)\,q(x'|x)}\big)$, chosen so that *detailed balance* ($\pi(x)\,P(x' \mid x) = \pi(x')\,P(x \mid x')$) holds, which is a sufficient condition to satisfy stationarity. Another well-studied option is Gibbs sampling, which updates one coordinate at a time from its full conditional [39]. The issue with both of these (and the family more generally) is correlation – consecutive samples aren't independent, so how quickly the chain forgets where it started (its mixing time) governs how many useful samples we actually get, a theme we'll return to when evaluating samplers in Section 2.1.4.

Mode collapse is another MCMC failure mode, and various methods have been introduced to try and mitigate its effect. Parallel tempering, or replica exchange [23], runs several chains at different temperatures and periodically proposes swapping their configurations, which lets a chain stuck in one mode borrow the easy mixing of its neighbours. This is similar in spirit to the

temperature curriculum idea we borrow from Zhu et al. [31] and their masked discrete neural sampler (MDNS) methodology, introduced in Chapter 3 and used throughout.

Modern discrete samplers attempt to leverage information to reduce the number of wasted proposals per accepted move. In Zanella [40], candidate flips are weighted by their estimated effect on the energy, so proposals focus on more highly-promising areas. In a similar vein, Gibbs with gradients (GWG) [19] uses gradients to estimate local energy decreases for all flips at once, and discrete Metropolis-adjusted Langevin algorithm (DMALA) [20] uses a Metropolis-adjusted discrete Langevin proposal, taking a physics inspired walk to find promising sampling regions.

Importance sampling is another approach, where we draw from some tractable proposal distribution q, and reweight its samples by $w = \pi(x)/q(x)$, where we require *consistency*, i.e. $q(x) > 0$ wherever $\pi(x) > 0$. These weighted averages remain consistent, though only really useful when q covers π, else the weights start to degenerate, an idea that is covered more in Section 2.1.4. Annealed importance sampling [41] combines ideas from MCMC and importance weighting, where instead of attempting to sample from the hard target π directly, it walks along a path of intermediate distributions from an easy base (say a uniform or high-temperature distribution) towards π, taking a few MCMC steps at each round and accumulating an importance weight along the way. Those weights correct for the chain never quite equilibrating at any single round, and as a by-product give an unbiased estimate of Z/Z_0, and since we know Z_0 from its tractable base, we get Z. It is worth keeping this idea in mind – the learned CTMC samplers of Section 2.2 use this same idea with a learned rate matrix instead of MCMC steps; locally equivariant discrete annealed proactive sampler (LEAPS) even describes itself as a strict generalisation of annealed importance sampling and sequential Monte Carlo (SMC) [30].

The final step here for now is annealed SMC [42], which runs this annealed importance sampling idea and tracks when the weights start to degenerate. Instead of letting poor samples dominate the trajectory, we can instead resample more promising particles and concentrate our efforts on their descendants, at the potential cost of lower diversity due to shared ancestry. It's particularly relevant here since LEAPS [30] leverages SMC resampling during training, and is something we experimented with during our hard constrained experiments (Table J.1 in Appendix J).

2.1.3 The Ising model as a running example

Our running example is a two-dimensional ferromagnetic Ising model [43] on a $D \times D$ lattice of sites, each either spin up or spin down, with neighbouring sites interacting through bonds which favour alignment (since this lowers the energy). They can be arbitrarily large in size, though even at $D = 10$ we have 2^{100} configurations, which puts the sampling problem well out of reach for exact enumeration. The model has a canonical closed-form energy function $E(x) = -J_{phys} \sum_{\langle ij \rangle} x_i x_j$, where the sum is taken over unordered neighbour pairs; we instead sum over ordered pairs, double counting the edges. Every bond thus appears twice in our exponent, so the target $\exp(x^T J x)$ with $J = \sigma A_D$ matches the usual convention $\pi \propto \exp[-\beta E]$ at $J_{\text{phys}} = 1$ with $\beta = 2\sigma$.

This yields our (unnormalised) target Boltzmann distribution:

$$\pi(x) \propto \exp\big(x^T J x + b \textstyle\sum_i x_i\big) \tag{2.2}$$

where $x \in \{-1, 1\}^d$, $J = \sigma A_D$ for $\sigma > 0$ and A_D the adjacency matrix of the $D \times D$ grid with periodic boundary conditions (i.e. on a torus). The bias $b \in \mathbb{R}$ controls the asymmetry between the two spin states; throughout this work we take $b = 0$, following literature conventions.

Temperature enters the target through the coupling term β, the inverse temperature. Our convention gives $\beta = 2\sigma$, where larger σ means a lower temperature and a stronger tendency towards alignment. Figure 2.1 shows the three regimes of interest – at high temperature (left),

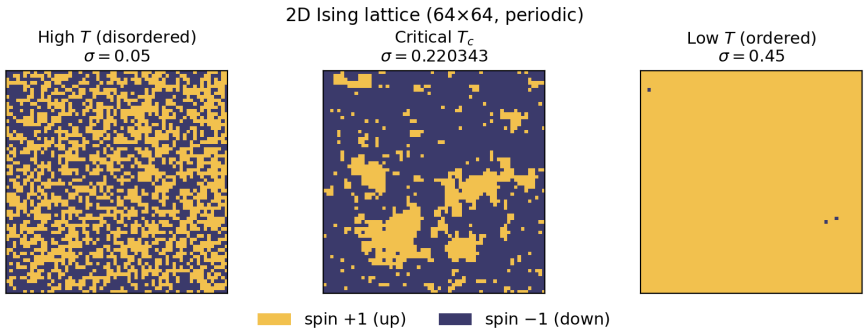


Figure 2.1. Equilibrium configurations of a 64×64 periodic 2D Ising lattice at three temperatures, drawn by Gibbs sampling for the outer panels (the low-T panel from an aligned start) and Wolff cluster updates for the critical panel. Gold cells are spin $+1$ (up), indigo cells are spin -1 (down). Temperature enters through the coupling σ in $J = \sigma A_D$ (larger σ = lower temperature): high T ($\sigma = 0.05$, left), the critical point ($\sigma = \sigma_c \approx 0.2203$, centre), and low T ($\sigma = 0.45$, right).

mass is broadly spread with easy mixing; near criticality (centre) we observe critical slowing down, where the states cluster, and the correlation length diverges in the thermodynamic limit and grows to the system size. This regime is often considered the hardest to sample and the one used by sampling literature to benchmark. Deep ordering (right) occurs at low temperatures, with a bimodal target. For our instantiation of the Ising model, this critical inverse temperature is well known – $\beta_c = \frac{1}{2}\ln(1+\sqrt{2}) \approx 0.4407$ [44], i.e. $\sigma_c = \frac{1}{4}\ln(1+\sqrt{2}) \approx 0.2203$, which is our headline temperature throughout.

There are classical methods designed to work in the more challenging sampling regimes described here, specifically focussed on the unconstrained problem. The cluster algorithms of Swendsen–Wang and Wolff [21, 22] flip whole correlated domains at once in an attempt to overcome the slowing-down. This however won't help us when we move to the constrained case, since since a Wolff or Swendsen–Wang flip changes the count a constrained sampler must conserve (Chapter 5). Cluster moves that conserve the count do exist (the geometric spin-exchange clusters of Heringa and Blöte [45], generalised to fluids by Liu and Luijten [46]), but they rely on a self-inverse point symmetry of the lattice and remain symmetry-specific specialists; the general-purpose chain at fixed composition, and the one most commonly used by materials practitioners, is the Kawasaki dynamics of Section 2.3.2.

2.1.4 Evaluating samplers

Once we have our trained sampler, how do we verify its quality? The challenge here is that outside of enumerable examples we don't always have a certifiable "ground truth" to compare with; indeed the target only being known up to its normaliser is the reason we're building these samplers in the first place. We'll mainly focus on those that evaluate the quality of the sample chain and don't require knowledge of the target (effective sample size (ESS), autocorrelation), as well as errors on physical observables (magnetisation profile, two-point correlation, energy distribution) against samples from a long reference chain, in-line with the literature.

Sample-quality diagnostics

A popular choice of sample-quality diagnostic is the importance-sampling ESS, which estimates how many independent, equally weighted draws a set of K importance-weighted samples is worth [47]. Given samples from our proposal $x^{(k)} \sim q$ with importance weights $w^{(k)} = \pi(x^{(k)})/q(x^{(k)})$

and normalised weights $\tilde{w}^{(k)} = w^{(k)} / \sum_{j=1}^{K} w^{(j)}$, the (normalised) ESS is:

$$\mathrm{ESS} = \frac{\left(\sum_k w^{(k)}\right)^2}{K \sum_k \left(w^{(k)}\right)^2} = \frac{1}{K \sum_{k=1}^{K} \left(\tilde{w}^{(k)}\right)^2} \in \left[\tfrac{1}{K}, 1\right], \tag{2.3}$$

which equals 1 when all weights are equal and falls toward $1/K$ when a single sample dominates (the unnormalised form differs only by a factor of K). We may also intuit here by noticing that ESS is a second-moment ratio of the weights [48], so measures their spread (Appendix C.1). For a tractable state proposal these weights are proportional to $\pi(x)/q(x)$; the learned transport samplers throughout instead weight whole trajectories, so their ESS measures path-weight concentration.

For the correlated draws of an MCMC chain, we instead use $\mathrm{ESS}_{\mathrm{MCMC}} = N/\tau$, where N is the length of the MCMC chain, and τ the integrated autocorrelation time. Integrated autocorrelation time τ is loosely the number of steps the chain must take before it yields an effectively independent sample: writing $\rho(k)$ for the autocorrelation of an observable at lag k, we have $\tau = 1 + 2\sum_{k\geq 1} \rho(k)$. A poorly mixing chain at low temperature can usually be attributed to the multimodal regime – large τ, with a correspondingly tiny effective sample size.

ESS is not a perfect metric however, and only reports quality of the weights on the support our sampler visits – indeed Du et al. [49] reports the MDNS sampler at a normalised ESS of approximately 0.98 on 16×16 Ising at low temperature while covering only one $\mathbb{Z}_2$ phase. The converse is also true – Guo et al. [32] also report low trajectory ESS after convergence, tentatively attributing it to variance in their estimated path weights.

To provide rough sampling compute accounting, alongside all of our results we include *compute per effective sample* – sampling floating-point operations (FLOPs) divided by K ESS for weighted draws, or N/τ_f for a chain, which naively discounts weight concentration or serial correlation. This cost includes generation, weight evaluation, and chain burn-in, though we exclude training costs. We provide the usual caveats here as there are no real standards for cost accounting across these methods – weight ESS and observable-specific τ_f do not imply equal precision [50], so cost must be read alongside physical errors, and also not all FLOPs are created equal. They simply compare arithmetic work, not a runtime comparator; neural work can often be parallelised on a graphics processing unit (GPU), whereas chain-based runs are serial in nature.

Physical quantities

Our introduction to ESS above points to the need for other forms of evaluation to take place on our sampler, and here we will use physical observables, where we'll compare our samples to those from the relevant reference. The first of these is the magnetisation-profile error. With $M^{\mathrm{row}}(k) = \sum_{i \in \mathrm{row}\, k} \mathbb{E}[x_i]$ the summed mean spin of row k, and M^{col} its column analogue,

$$\Delta\mathrm{Mag} = \frac{1}{2D} \sum_{k=1}^{D} \Big(\left| M^{\mathrm{row}}(k) - M_\pi^{\mathrm{row}}(k) \right| + \left| M^{\mathrm{col}}(k) - M_\pi^{\mathrm{col}}(k) \right| \Big), \tag{2.4}$$

compares sampler to reference; it scales with D, so it is comparable within a lattice size, and can also be used as a mode-coverage read, since if a sampler misses a $\mathbb{Z}_2$ mode this will spike.

Second, we look to spatial structure through the two-point correlation error $\Delta\mathrm{Corr}$. With $C(i,j) = \mathbb{E}[x_i x_j] - \mathbb{E}[x_i]\mathbb{E}[x_j]$ and $C^{\mathrm{row}}(k,l)$ its sum over the D same-column site pairs in rows k and l,

$$\Delta\mathrm{Corr} = \frac{1}{D^2} \sum_{k,l=1}^{D} \Big(\left| C^{\mathrm{row}}(k,l) - C_\pi^{\mathrm{row}}(k,l) \right| + \left| C^{\mathrm{col}}(k,l) - C_\pi^{\mathrm{col}}(k,l) \right| \Big), \tag{2.5}$$

which is blind to a global spin flip and so complements ΔMag. Both of these follow the evaluation structure in MDNS [31].

Finally, we look at the energy marginal, specifically we measure the one-dimensional Wasserstein-2 distance EW_2 between the distributions of $E(x)/d$ under sampler and reference, a per-site version of the energy-distance diagnostic used by Guo et al. [32], computed in closed form from the two quantile functions (inverse CDFs) of the per-site energy, $\mathrm{EW}_2^2 = \int_0^1 \left(F^{-1}_{\text{sampler}}(u) - F^{-1}_{\text{ref}}(u)\right)^2 du$ – it needs no bins and reflects how far the sampled distribution would need to shift to overlap with the reference. Where the target is enumerable we also report the total variation (TV) distance $\mathrm{TV}(p,q) = \frac{1}{2}\sum_x |p(x) - q(x)|$ between a sampled marginal and its exact counterpart (Table 5.7).

Our results tables also separate reference uncertainty from finite-draw noise. Parenthesised reference errors estimate uncertainty in the reference pool, whereas the sampling-floor row gives the average error from N independent target draws. This forms a finite-sample benchmark, not a lower bound or significance threshold; we provide calculation details in Appendix C.3.

In Chapter 3, since exact Ferdinand-Fisher thermodynamic references exist in the unconstrained problem [51], we keep DNFS's per-site free energy $F/d = -\log Z/(2\sigma d)$ and internal energy E/d [29], with E the energy of Section 2.1.3. These metrics also come with their caveats, since thermodynamic agreement alone certifies very little – Choudhary et al. [52] showed that a model with the right one-body marginals is able to match the magnetisation curve while carrying no correlations at all; a combination-based approach is the right one to take.

2.2 Neural Samplers for Discrete Distributions

We now move from hand-designed chains to learned samplers, with a trained transportation from a tractable base distribution to target π, starting with the continuous cases and graduating to the discrete versions we build upon.

2.2.1 Generative modelling as learned transport

Normalising flows transform simple base distributions into more complex target distributions through a chain of invertible transformations $f_\theta : \mathbb{R}^d \to \mathbb{R}^d$, where the Jacobian gives the exact change of variables [53]. Boltzmann generators apply normalising flows to molecular Boltzmann distributions, learning from energies as well as example configurations [54].

Continuous normalising flows generalise the discrete-time flow to an ordinary differential equation (ODE) $\dot{x}(t) = v_\theta(x,t)$, where the probability density evolves by the instantaneous change of variables formula. Flow matching [55] manages to avoid simulation-based training by regressing v_θ against a known velocity field along a chosen probability path, and stochastic interpolants [56] generalise this further to stochastic dynamics. The shared idea here is that we prescribe a path p_t of distributions from base p_0 to target p_1, and learn a transportation object to realise it.

Between these data-driven flows and the problems we tackle in this thesis sits a family of continuous *non-equilibrium* neural samplers, which learn the transport for an unnormalised target rather than from data. Examples include the path-integral sampler [57], denoising diffusion samplers [58], controlled Monte Carlo diffusions [59] and the non-equilibrium transport sampler [60]. These can be seen as learned continuous-time variants of annealed importance sampling and SMC, something developed also by the discrete CTMC energy samplers in Section 2.2.3.

Discrete spaces require their own machinery to tackle analogous problems, where rate matrices R_t replace continuous velocity fields (Figure 2.2). The methods we study in Section 2.2.3 differ in how they achieve this objective – DNFS and LEAPS prescribe a geometric annealing path $p_t(x) \propto \eta(x)^{1-t}\tilde{\pi}(x)^t$ from tractable base η toward unnormalised target $\tilde{\pi}$, while masked diffusion

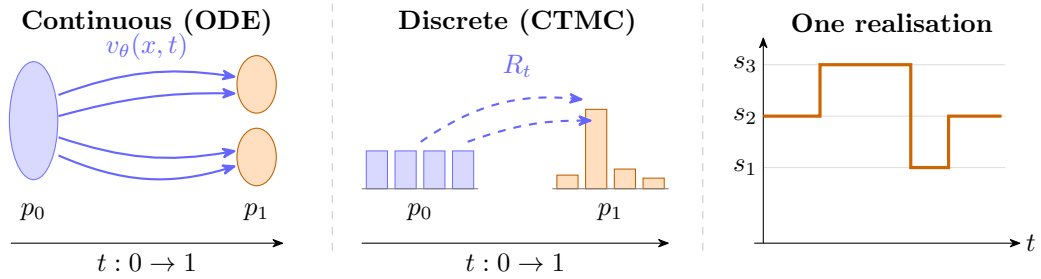


Figure 2.2. Learned transport, continuous and discrete. A prescribed path p_t carries probability mass from an easy base p_0 to the target p_1, and we learn the object that realises it: a velocity field $v_\theta(x,t)$ driving an ODE (left), or the rate matrix R_t of a continuous-time Markov chain (centre). Right, a single CTMC realisation: it waits in a state (for constant rates, an exponentially-distributed time; horizontal segments), then jumps instantaneously to another (vertical segments).

methods learn to fill in hidden sites through a denoising process, and adjoint methods adjust transition rates to favour the desired outputs. A commonality however, is the CTMC.

2.2.2 Continuous-time Markov chains

A CTMC is a random process that determines how we move between discrete states, where the next move depends only on the current state and not on the past, known as the *Markov property*. Formally, a CTMC takes values in a discrete space but transitions can happen at any time $t \in [0, T]$ rather than in fixed integer steps, with the dynamics set by a rate matrix $R_t : \mathcal{X} \times \mathcal{X} \to \mathbb{R}$ that gives the instantaneous rate of transitioning between two states at time t (Figure 2.2). These holding times between jumps are exponentially distributed; a low rate means a long wait before transitioning and vice versa for a high one.

Some properties of the rate matrix worth knowing:

1. **Off-diagonals are non-negative**, i.e. $R_t(y, x) \geq 0$ for $y \neq x$, since the entry is the instantaneous rate of going from x to y.
2. **Columns sum to zero**, i.e. $R_t(x, x) = -\sum_{y \neq x} R_t(y, x)$, forced by probability conservation, since the outflow from x must be accounted for.
3. **Connection to transition probabilities.** The rate matrix relates to the transition probabilities over an infinitesimal time step Δt via

$$p_{t+\Delta t|t}(y \mid x) = \mathbf{1}_{y=x} + R_t(y, x)\,\Delta t + o(\Delta t). \tag{2.6}$$

One of the issues with the rate matrix is that it is indexed by pairs of states, so for a $D \times D$ lattice it has on the order of $(2^{D^2})^2 = 2^{2D^2}$ entries, which can't be enumerated except in trivial cases. To keep this in hand, a design choice common amongst the discrete neural sampler family (Section 2.2) restricts the support to single-site changes, leaving sparse columns (only $d(S-1)$ non-zero off-diagonals per column). DNFS and LEAPS restrict to single site flips, masked diffusion samplers unmask one site at a time, and similarly in adjoint matching, though Guo et al. [32] notes that this isn't strictly necessary. A couple of notable outliers are discrete-time schemes such as scalable discrete diffusion sampler (SDDS) [61], as well as the aforementioned JANUS [33]; the latter of which is able to reveal several sites per finite step.

Our rate matrix gives us a description of the transition process, but what we need is the marginal $p_t(x)$, the probability of being in state x at time t. This is governed by the Kolmogorov forward equation (Kolmogorov forward equation (KFE)), which leverages (2.6) and is derived in Appendix B:

$$\partial_t p_t(x) = \sum_y R_t(x, y)\, p_t(y) \tag{2.7}$$

Intuitively, the rate of change of probability mass at x is the sum of the net flow of probability into x from all other states, which makes sense. This can be thought of as a conservation of probability, similar to Fokker–Planck in continuous spaces.

Dividing through by $p_t(x) > 0$, applying the log-derivative trick ($\partial_t \log p_t = \partial_t p_t / p_t$) and separating the $y = x$ term (Appendix B) gives:

$$\partial_t \log p_t(x) = \sum_{x \neq y} R_t(x, y) \frac{p_t(y)}{p_t(x)} - \sum_{x \neq y} R_t(y, x) \tag{2.8}$$

For an annealed Boltzmann path the left-hand side is $-\partial_t[\beta_t E(x)] - \partial_t \log Z_t$, a tractable energy term and one intractable scalar, the latter of which is discussed further in Section 3.1.2. It also separates the contributions involving probability ratios (where the normalising constant cancels for Boltzmann paths) from the contributions involving rate-matrix entries alone. DNFS and LEAPS enforce the KFE through their residual training objective (Section 3.1.1), whereas the masked-diffusion and adjoint samplers instead align the path measures of two CTMCs and don't train on the KFE directly.

It is worth noting that the Kolmogorov forward equation is a consistency equation but doesn't tell us how to find the rate matrix. There are infinitely many rate matrices that satisfy this equation for a given path [29, 30], though representing and finding a useful one in high dimensions can be computationally challenging. We tackle this by parametrising it with a neural network, and train it to minimise the Kolmogorov residual of (2.8) (Section 3.1.1).

2.2.3 A survey of existing discrete neural samplers

Before we take a deep-dive into a specific sampler, we first need to analyse the plethora of choices we have and how they might fit with our thesis states in Section 1.3. We can categorise each of the samplers based on four design choices, inspired by MDNS's own survey of the field [31], though the addition of the move set is our own and indeed the most important for our application.

(A) Path or reference process – which intermediate distributions p_t, or which noising or reference dynamics, define the transport problem.
(B) Move set – which transitions the generator is allowed to make, i.e. where the rate matrix (or its analogue) may place non-zero rate.
(C) Objective – the loss that makes the learned process realise the path.
(D) Parametrisation – the network that emits the transport object, and the tricks that keep evaluating it tractable.

DNFS [29], the sampler this dissertation builds on, combines an annealed Boltzmann path, single-site flips, a squared Kolmogorov-residual objective and a locally equivariant transformer. Its ancestor LEAPS [30] shares the path and move set; its physics-informed neural network (PINN) loss is the same squared residual, with $\partial_t \log Z_t$ learned by a small network instead of estimated by DNFSs Stein control variate.

Discrete diffusion [27, 62, 28, 63] presents its own challenges, especially if we want to train with energies. Knowing the energy of a complete configuration does not directly give us the probabilities we need for partially masked ones. MDNS and its uniform-noise variant UDNS tackle this by using generated trajectories and importance weighting to construct their training objectives [31]; proximal diffusion neural sampler (PDNS) instead improves the sampler through successive updates that penalise large changes in its distribution [64]. A recent addition to the family is JANUS, which uses energy-derived probabilities as labels for its own generated configurations [33]; its fixed-composition variant is discussed further in Section 2.3.5. TCSIS [65] and energy-based generator matching (EGM) [66] take an alternate route, and estimate the

method	path or reference process (A)	move set (B)	objective (C)	conserved count
DNFS [29]	annealed Boltzmann	single-site flip	squared KFE residual (Stein control variate)	none
LEAPS [30]	annealed Boltzmann	single-site flip	squared KFE residual (PINN, learned $\partial_t F$)	none
MDNS / UDNS [31]	masked or uniform diffusion	single-site unmask / replace	path-measure matching (incl. weighted denoising)	none
DAM / DASBS [67, 32]	reference bridge	single-site (masked or uniform)	adjoint matching / SB	none
PDNS [64]	masked diffusion	single-site unmask	Kullback–Leibler (KL)-proximal optimal control	none
EGM [66]	masked (discrete demonstration)	free-form; single-site used	generator matching	none
GFlowNets [69]	constructive DAG	build actions	trajectory balance	via masking
this work (Ch. 5)	annealed on-manifold	two-site swap	squared KFE residual	composition, exact

Table 2.1. Discrete neural samplers on the design axes; the parametrisation axis (D) is omitted for width. None of the listed CTMC samplers uses count-conserving moves; precedents outside this table are in Section 2.3.5.

reverse dynamics directly from energy-weighted samples, so they never need the masked-state probabilities at all.

The adjoint-matching family is different again; as the name suggests, Discrete adjoint Schrödinger bridge sampler (DASBS) trains a Boltzmann sampler by regressing the flip rates of the CTMC onto adjoint targets derived from the target's discrete score [32], whereas discrete adjoint matching (DAM) fine-tunes a pretrained base towards rewards [67]. So these are specifying noising or reference dynamics rather than the intermediate marginals of DNFS.

Last but by no means least, members of the family that construct configurations sequentially. Variational autoregressive networks minimise variational free energy with tractable conditional probabilities [68], and GFlowNets learn policies on a DAG through trajectory or subtrajectory balance [34, 69, 35], where feasible actions enforce constraints. Not quite materials but an adjacent and similar application of GFlowNets to crystal structures is Crystal-GFN [70].

2.3 Constrained Sampling for Materials Configurations

2.3.1 Constraint types and handling strategies

Recall the statement of the constrained sampling problem from Chapter 1; we have target distribution $\pi(x) \propto \exp[-\beta E(x)]$, restricted to $x \in \mathcal{C} \subseteq \mathcal{X}$, where the constrained space $\mathcal{C}$ is typically much smaller than the unconstrained state space $\mathcal{X}$. Even though we are restricting to a smaller subset $\mathcal{C}$, the state space can still be exponentially large and exact enumeration intractable. The geometry of the support also changes, since local moves that preserve $\mathcal{C}$ are often different from those that don't. It is exactly one of the main research questions of this thesis to extend neural architectures to the pair-swap moves of Section 2.3.2, and to do so in an efficient manner. The CTMC backbone presents no issues theoretically, since as previously mentioned, rate-matrix sparsity is a design choice and not a law.

Constraints themselves fall into roughly three families. The first of the three are equality constraints which fix a property exactly, with materials-based examples including composition, total charge, sublattice occupancy, and stoichiometry. We're most interested in the first of these for the alloy problems of this thesis – on our spin lattices fixed composition is $c(x) = \frac{1}{d}\sum_{i=1}^{d} \frac{1+x_i}{2} = c_{\text{target}}$, with c the up-spin fraction (in alloy terms, the fraction of sites occupied by one of the two species). Inequality constraints instead bound a quantity, for example a minimum spacing between atoms or a cap on the number of lattice defects, and combinatorial constraints cover structural properties like valid bonding topologies.

We study two routes to enforcing constraints, soft and hard, with the details deferred to their respective chapters (Chapters 4 and 5). The soft route leverages the base unconstrained sampler, and tilts the target with a penalty and is inspired by the established materials approach (VCSGC [26]); other potential approaches from the ML literature include guidance and reward-tilting methods, discussed in Section 2.3.5. The hard route instead restricts the sampler's moves so the constraint holds by construction.

2.3.2 Constrained sampling with Kawasaki dynamics

Constraint-aware samplers have been around for some time, and fixed-magnetisation sampling in particular has a well-defined lineage. Kawasaki spin-dynamics [25] is very simple in nature. Instead of using single site flips, instead pick two sites carrying opposite spins and swap them. In Kawasaki's original form, the two swapped sites are nearest neighbours, though this is by no means necessary. The practitioner default (and the one used throughout this dissertation via the canonical-ensemble sampler in `mchammer` [71] a popular Monte Carlo tool for alloy configurations) swaps any unlike pair, which is preferable in problems where modes might be separated (Chapter 5). Since one up-spin becomes a down-spin and vice versa, the total magnetisation remains constant, and the chain only explores configurations with the composition it started from. The constraint holds by construction, with no penalty. Simple, but effective. The swap is indeed the simplest instance of a general idea, a move set matched to a conservation law, which we discuss in Chapter 5 using the language of Markov bases.

2.3.3 Cluster expansions and alloy thermodynamics

Materials scientists need ensemble averages for ordering transitions, short-range order and free energies of mixing, but evaluating each sampled configuration with traditional methods such as DFT is too costly. They instead use a surrogate model called a *cluster expansion*, first computing energies from DFT using a training set of configurations, and then fit a faster lattice model to them by regression, which is then given to the sampler. Our work is the last piece of that workflow.

Where the aforementioned Ising energy couples sites only through nearest-neighbour pairs, a cluster expansion keeps adding terms, longer-range pairs then triplets, quadruplets and so on, so we have a much more expressive model than a single pairwise coupling. For a configuration x on the lattice the energy takes the form:

$$E(x) = J_0 + \sum_{\alpha} m_\alpha J_\alpha \, \Pi_\alpha(x), \tag{2.9}$$

where E is energy per site in the integrated cluster expansion toolkit (icet) convention[1], α labels a family of site groups related by lattice symmetries, called an *orbit*: nearest-neighbour pairs form one family, diagonal pairs another; the multiplicity m_α counts its clusters per site. The correlation Π_α averages products over the orbit: binary spins suffice here, while multicomponent expansions use products of point-basis functions [71]. The total Boltzmann energy is $dE(x)$, the coefficients J_α are the effective cluster interactions (ECIs) [9], and the constant J_0 simply fixes the reference energy. This idea may be familiar already, and is akin to a polynomial-basis-expansion; the correlation functions Π_α play the role of basis features and the ECIs J_α are the coefficients, which are not derived by hand but fit by regression to the energies of a training set of configurations computed with DFT.

Practitioners rely on tools like icet [71], which forms one of the reference implementations for the whole pipeline. It's able to construct the cluster space, fit the ECIs to a set of DFT energies, and run Monte Carlo over the resulting expansion through its `mchammer` module (the particularly interesting part for us). It will form the materials side benchmark for our sampler throughout.

A nice aside is that we can make a cluster expansion generalisation for the Ising model we use throughout the thesis, which proves useful when we look to compare our samplers to those used in icet. Keeping only the nearest-neighbour pair orbit and no field term at $S = 2$, with occupations encoded as $x_i \in \{-1, +1\}$ (Ising spins), on the periodic square lattice the multiplicity is $m_{\mathrm{NN}} = z/2 = 2$, where $z = |\{\uparrow, \downarrow, \leftarrow, \rightarrow\}| = 4$ is the coordination number,

[1] they differ depending where you look, we opt for this one since we use icet as a reference

and Π_{NN} averages $x_i x_j$ over the $Nz/2 = 2N$ bonds. Multiplying (2.9) by the N sites thus gives $E_{\mathrm{tot}}(x) = \mathrm{const} + J_{\mathrm{NN}} \sum_{\langle ij \rangle} x_i x_j$, so $J_{\mathrm{phys}} = -J_{\mathrm{NN}}$ in this convention and (2.2) follows at $\sigma = \beta J_{\mathrm{phys}}/2$. Neat.

2.3.4 Sampling cluster expansions in practice

A cluster expansion on its own is only an energy; the quantities we're actually interested in are finite-temperature averages over the ensemble of configurations, so getting at any of them means sampling that ensemble, and how you sample depends on what the composition is doing.

If your composition is allowed to vary, the natural choice is the semi-grand-canonical ensemble, $\rho_{\mathrm{SGC}}(x) \propto \exp[-\beta(E(x) + \Delta\mu\, N_1(x))]$, with N_1 the count of one species (at $\Delta\mu = 0$ this is our unconstrained Boltzmann target). Here single-site flips are permitted and the system exchanges species with a reservoir at a fixed chemical-potential difference μ, so μ controls the expected composition while letting it fluctuate [26]. This is well suited to mapping out phase diagrams, where we'd like to sweep composition, but it is not suitable when the composition must be fixed. Composition can also jump across multi-phase regions as this chemical potential varies, making it hard to resolve intermediate compositions in finite simulations, something VCSGC addresses.

Variance-constrained semi-grand-canonical (VCSGC) sampling sits between the semi-grand above and the grand canonical below [26]. It augments the semi-grand with a quadratic penalty on the composition, rather than fixing composition exactly, reweighting the semi-grand by $\exp[-\kappa N(c(x) + \phi/2)^2]$, a quadratic well in the composition. The rationale for doing this is to capture small fluctuations around a composition of interest across multi-phase regions. This is the direct classical analogue and inspiration for our soft-constrained neural sampler in Chapter 4.

If we'd instead like a fixed composition, the classical way to achieve it is canonical Monte Carlo over the Kawasaki swaps of Section 2.3.2, accepted by Metropolis – here every move trades one species for another, so the composition is preserved by construction. This is the canonical (sometimes called grand canonical) ensemble provided by icet [71].

Special quasirandom structures (SQS) take a different approach [72]. Rather than sampling the finite-temperature ensemble at all, an SQS is a single small configuration optimised so that its short-range correlations match those of the ideal random alloy at infinite temperature. It respects the target composition exactly and is widely used to hand a representative disordered configuration to DFT, which makes it a natural competitor to ensemble sampling. Its limitation is that it returns only a single point estimate, one carefully chosen configuration, rather than samples from the finite-temperature distribution.

The shared disadvantage of these methods is that each is bespoke and non-amortised. The move set or objective has to be tailored to the constraint at hand, swaps for fixed composition, a chemical potential for variable composition, a correlation-matching optimisation for SQS, and every new target means another Monte Carlo run, or another optimisation. Closing that gap with a single learned sampler that handles the constraint natively and amortises across compositions and temperatures is what motivates our work on constrained neural samplers.

2.3.5 Constrained neural samplers

The way I like to think about existing methods is *when* they're able to hold the constraint. There are various options – encourage the constraint but don't require exactness, ensure it's adhered to only at completion, restored each step by projection, or held at every step along the trajectory. The swap sampler we develop will fall into the final category.

We'll refer to the first of these as a soft constraint, one that biases toward a target but doesn't fix it exactly. Discrete guidance tilts the transition rates towards a property [73], following the

composition	ensemble (measure)	materials sampler (move)	learned counterpart
free	SGC: $e^{-\beta(E+\Delta\mu N_1)}$, here $\Delta\mu = 0$	`mchammer` SGC, single-site flips	Ch. 3: DNFS, learned flip rates ($\lambda = 0$)
steered	VCSGC: $e^{-\beta E-\kappa N(c+\phi/2)^2}$	`mchammer` VCSGC, single-site flips	Ch. 4: same flips, penalised target (finite λ)
fixed	canonical: $e^{-\beta E}$ on $\{c = c_{target}\}$	`mchammer` canonical, unlike-pair swaps	Ch. 5: swap CTMC, learned pair rates (conditioned target)

Table 2.2. The ensemble ladder: the free and steered rows change the target measure over the same single-site moves; the fixed row also changes the move set and support. The soft row's parameters map exactly onto ours ($\kappa = \lambda$, $\phi = -2c_{target}$, Section 4.1 of Chapter 4).

continuous classifier and classifier-free forms [74, 75], and discrete adjoint matching fine-tunes a sampler through a reward [67]. A finite tilt is the neural analogue of VCSGC, and is exactly the idea we use in Chapter 4. Letting the composition float and pinning it down afterwards is the other way to achieve the same goal, the most natural of which is rejection sampling. In alloy thermodynamics, autoregressive samplers extend variational spin sampling [68] to semi-grand-canonical lattices [76, 77], where chemical potential controls a fluctuating composition. Another sampler we'll see a lot throughout this thesis is meta-dynamics discrete neural sampler (MetaDNS) – Du et al. [49] add an adaptive bias to their target energy function to encourage sampler exploration, and then reweight to the unconstrained base distribution after-the-fact.

The second builds a configuration from the ground up and lets the remaining budget of permitted site occupations decide which actions are allowed, meaning the constraint is only guaranteed to be satisfied at the terminal state. The main neural comparator we'll use for the constrained case is GFlowNets, which takes exactly this approach. These exhibit failure modes including credit assignment on long roll-outs [69, 35] and mode-coverage [78, 29]; though newer objectives have been proposed to tackle these issues [35, 79, 80]. JANUS [33], the closest fixed-composition neighbour to our work, does the same thing in a masked diffusion. It reveals species under a remaining-count budget, conditions on the composition, and learns from energy-derived labels on its own samples. On the projection side we have constrained discrete diffusion [81] and its protein-structure sibling [82], where the generator may leave the feasible set at a step and is projected back.

The final route is the classical and in my opinion the most satisfying one – using a move set that never changes the count, meaning every state along the trajectory is a complete configuration with the right composition. Kawasaki swaps (Section 2.3.2) and their intracluster generalisation [83] do this by hand; discrete spatial diffusion (DSD) [84] is the one neural precedent we know of, learning count-preserving particle hops as CTMC rates, though trained on images rather than energies. None of the energy-trained samplers of Section 2.2.3 uses such a move set; and it's something we must explore, since simply training on valid constrained examples is not sufficient enforce the count at generation [85].

Hybrid methods that combine the Markov chain and learn only its proposal do exist, including latent-space flows [86], swap policies for off-lattice mixtures [87] and count-conditioned proposals rejected before the Metropolis test [88]. They reuse the proposal but still run a chain per target, so don't gain the full amortisation advantage of the completely neural methods.

2.4 Summary and Research Gap

This chapter provided us with the foundational knowledge necessary to tackle this dissertation, and exposed a gap we'll aim to fill. That is, no energy-trained CTMC sampler holds a composition at every step of its trajectory (Table 2.1), which we will work towards by enforcing it on the rate matrix directly. The materials pipeline gives us a convenient multi-step approach to mirror with their three classical ensembles, which guide our trident-based approach (Section 2.3.4, Table 2.2).

3 The Unconstrained Baseline

Like a flow, we begin at the simple base and build toward our hard target. In this chapter we replicate DNFS on the unconstrained Ising model, chosen with our hard-constraint goal in mind – restricting moves to swaps at the rate matrix level. We choose DNFS specifically here since it gives us an explicit rate parametrisation in which to make that change, with a residual and architecture that expose its cost. The analogous ensemble on the materials side is SGC at zero chemical-potential difference, free of any composition constraints (Table 2.2).

3.1 Discrete Neural Flow Samplers

DNFS [29] builds upon LEAPS [30], which introduced learning a CTMC based sampler for Boltzmann distributions, albeit using a different estimator for the log normaliser. The two approaches share a lot of the same machinery: the annealed path, single-site move set, locally equivariant parametrisation, and most of the squared Kolmogorov residual. Indeed it is probably easier to say where they differ – DNFS exchanges the learned free-energy network of $\partial_t \log Z_t$ with a discrete Stein control variate, and adds its own locally equivariant transformer, ingredients we can combine to turn the theory of Section 2.2.2 into an optimisation problem.

Recall that our goal is to sample from our target distribution $\pi(x)$ but we don't have access to the normalising constant, so instead want to learn a CTMC following a prescribed path of distributions that ends at this target. Here we use the geometric path introduced in Section 2.2.1. Writing both the base and the target through their exponents, $\eta = e^{-E_0}$ and $\tilde{\pi} = e^{-E}$ with $E(x) = -x^T J x$ and β already inside J (Section 2.1.3), the path $p_t \propto \eta^{1-t}\tilde{\pi}^t$ is a linear interpolation of the exponents, $E_t(x) = (1-t)\,E_0(x) + t\,E(x)$. Our base is uniform, $E_0 = d\log 2$, so $Z_0 = 1$ and the path weight gives the absolute target normaliser. Every p_t on this path has a known unnormalised form, which allows us to work without knowledge of the normalising constant; since $p_t(x) = \exp[-E_t(x)]/Z_t$, the ratio $p_t(y)/p_t(x) = \exp[-(E_t(y) - E_t(x))]$ is Z_t-free. The time-derivative $\partial_t \log Z_t$ does not cancel – it enters through $\partial_t \log p_t(x) = -\partial_t E_t(x) - \partial_t \log Z_t$, the first term of the residual written below, and is where we hit issues with variance; which we navigate with the help of a discrete Stein control variate in Section 3.1.2.

3.1.1 Training objective

Defining the path p_t does not give us sufficient information to move a sample along it; for this we require the rate matrix R_t of Section 2.2.2, tied to the path by the Kolmogorov forward equation (2.7). This is only a consistency condition on R_t, and doesn't give us a formula for obtaining one. Instead, we parametrise it with a neural network and train that to satisfy our Kolmogorov consistency condition.

We define the residual of the Kolmogorov equation at (x, t) as

$$\delta_t(x; R_t^\theta) := \partial_t \log p_t(x) + \sum_{y \neq x} \left[R_t^\theta(y, x) - R_t^\theta(x, y)\,\frac{p_t(y)}{p_t(x)} \right], \tag{3.1}$$

which equals zero exactly when R_t^θ satisfies the Kolmogorov forward equation at that point. At fixed t, the ideal objective is the expected squared residual

$$L(R_t^\theta) := \mathbb{E}_{q_t(x)} \Big[\, \delta_t(x; R_t^\theta)^2 \,\Big], \tag{3.2}$$

where $q_t(x) > 0$ wherever $p_t(x) > 0$. The choice of distribution here is what helps us – we're not able to sample from p_t, which is the whole problem, but we need not. The zero of (3.2) is not

dependent on q_t; if it's satisfied then it's necessarily true that the residual is zero everywhere, everywhere including where q_t covers, whichever q_t that may be [29]. In DNFS this reference is the sampler's own rollout; the current rate matrix is simulated forward from p_0, the visited states fill a replay buffer, and training mini-batches are drawn uniformly from it, so the loss concentrates on the regions the evolving sampler reaches.

Once we have a trained network, our rate matrix gives us an amortised sampler. Simply draw $x_0 \sim p_0$, roll the CTMC forward on an Euler time grid, and re-weight using the accumulated path weight (3.10). This looks familiar, and indeed it is. This is a continuous-time learned-transport generalisation of annealed importance sampling [41, 30], with the MCMC kernel replaced by a learned transport and continuous time approximated by an Euler discretisation.

This is somewhat of an idealistic view of things, and a little more work needs to be done before we can obtain these samples. The loss $L(R_t^\theta)$ is intractable in its current form. How do we deal with $\partial_t \log Z_t$, and the $\mathcal{O}(S \times d)$ neighbour evaluations in $\sum_{y \neq x}$?

3.1.2 Variance reduction techniques

We tackle the first problem, the high variance of the $\partial_t \log Z_t$ estimate, with a discrete Stein control variate. Recall that $\partial_t \log p_t(x) = -\partial_t E_t(x) - \partial_t \log Z_t$; differentiating $Z_t = \sum_x \exp[-E_t(x)]$ gives $\partial_t \log Z_t = -\mathbb{E}_{p_t(x)}[\partial_t E_t(x)]$, with the two-line derivation in Appendix C. Since we are dealing with an expectation, the traditional move is to use Monte Carlo approximation:

$$\partial_t \log Z_t \approx -\frac{1}{K} \sum_{k=1}^{K} \partial_t E_t(x^{(k)}), \quad x^{(k)} \sim p_t. \tag{3.3}$$

This leaves us with two problems. Sampling from p_t is hard; p_t is an unnormalised Boltzmann distribution, exactly the type of thing we are trying to learn to sample from. In practice we could use short-run MCMC, but we'd like not to introduce bias if we can afford not to. Additionally, $\partial_t E_t(x) = E(x) - E_0(x)$ varies wildly across the state space, giving the estimator high variance even with correct samples. Plugging a noisy estimate of $\partial_t \log Z_t$ into the squared loss amplifies that noise further and can destabilise training.

How may we tackle this variance, you may ask? With a control variate, a classic Monte Carlo variance-reduction technique [29, Sec. 3.1]. Here we use the discrete Stein identity, which leans on the column-sum-to-zero property of the rate matrix. Its expectation is known to be zero, and at optimality it is perfectly anticorrelated with $-\partial_t E_t$, which is how we achieve our variance reduction.

The Stein quantity is defined as

$$\xi_t(x; R_t^\theta) := \sum_{y \neq x} \left[R_t^\theta(y, x) - R_t^\theta(x, y) \frac{p_t(y)}{p_t(x)} \right]. \tag{3.4}$$

The Stein identity states that $\mathbb{E}_{p_t(x)}[\xi_t(x; R_t^\theta)] = 0$ for any rate matrix R_t^θ satisfying the column-sum-to-zero property, making ξ_t a valid control variate with known (zero) expectation; with proof left to Appendix C. The variance-reduced estimator of $\partial_t \log Z_t$ is then

$$\partial_t \log Z_t \approx \frac{1}{K} \sum_{k=1}^{K} \left[-\partial_t E_t(x^{(k)}) + \beta\, \xi_t(x^{(k)}; R_t^\theta) \right], \quad x^{(k)} \sim p_t. \tag{3.5}$$

We set the control-variate coefficient $\beta = 1$ in (3.5) (separate from the inverse-temperature notation) and write $a_t^\theta(x) := -\partial_t E_t(x) + \xi_t(x; R_t^\theta)$. Full detail of its implementation during training can be found in Algorithm 1 in Appendix D.

Notice here that Equation (3.5) is written with $x^{(k)} \sim p_t$, which is not necessarily true during training, which uses samples produced by the model, with distribution q_t^θ – the pointwise identity $a_t^\theta - \partial_t \log Z_t = \delta_t$ only guarantees a zero-variance estimate when the residual vanishes, and away from that solution no guarantees are made (we can see bias and even additive variance effects). Appendix C.2 provides a bound for this bias, as well as an ideal-centred-loss identity; worth noting however that neither guarantees monotonic improvement during training.

3.1.3 Hollow and locally equivariant architecture

The second bottleneck from Section 3.1.1 is that evaluating the residual at a state x needs the rate matrix not only at x but at every single-site-flip neighbour; the inflow term $R_t^\theta(x, y)$ of (3.1) is the rate *out of* the neighbour y, i.e. a network evaluation at y. With only single-site flips admissible, the sum $\sum_{y \neq x}$ runs over the $d = D^2$ sites; we index a transition by the flipped site i and its new token τ, writing $R_t^\theta(\tau, i \mid x)$. In this notation the reverse (inflow) term $R_t^\theta(x_i, i \mid y)\, p_t(y)/p_t(x)$ is a rate evaluated at the neighbour $y = \mathrm{Swap}(x, i, \tau)$. Done naively, that is $\mathcal{O}(S \times d)$ forward passes per state, which is intractable as the problem scales. This can be bypassed using two results – reparametrise the rate matrix into a one-way form so a single scalar field carries both directions of every transition, then learn that field with a locally equivariant network so one evaluation at x returns it for every site and token at once.

We say that a rate matrix is *one-way* if $R(y, x) > 0 \implies R(x, y) = 0$, i.e. between any pair of states, transitions flow in a single direction only. For any rate matrix there is an equivalent one-way matrix generating the same marginal path [30, App. D, Lemma D.1], so we lose nothing by restricting to this form. The gain is that a one-way matrix is described by a single real-valued potential G_t^θ through

$$R_t^\theta(\tau, i \mid x) = \big[\, G_t^\theta(\tau, i \mid x) \,\big]_+ := \max\big(G_t^\theta(\tau, i \mid x),\, 0\big), \tag{3.6}$$

so a single signed scalar per edge encodes the rate in whichever direction is active and the network need only represent G_t^θ.[1]

Antisymmetry and local equivariance. Let $\mathrm{Swap}(x, i, \tau)$ be x with site i overwritten by τ, so the flipped neighbour is $y = \mathrm{Swap}(x, i, \tau)$. We say G_t^θ is *locally equivariant* if it is antisymmetric under this swap [30, 29]:

$$G_t^\theta(\tau, i \mid x) = -\, G_t^\theta\big(x_i, i \mid \mathrm{Swap}(x, i, \tau)\big). \tag{3.7}$$

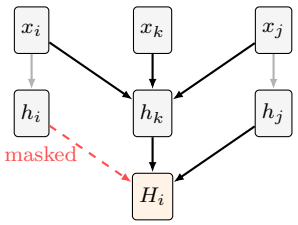
The inflow rate the residual needs at y, namely $[\, G_t^\theta(x_i, i \mid y)\,]_+$, equals $[\, -G_t^\theta(\tau, i \mid x)\,]_+$ by (3.7), so it is read straight off the same tensor $G_t^\theta(\cdot \mid x)$ already computed at x. No forward pass needs to be taken at a neighbour, and the per-state cost in network evaluations falls from $\mathcal{O}(S \times d)$ to $\mathcal{O}(1)$.

Putting the pieces together, the residual the locally equivariant transformer (leTF) actually evaluates is the single-site, one-way specialisation of (3.1): the neighbour sum $\sum_{y \neq x}$ becomes a sum over the d sites and their target tokens, and every rate is read off G_t^θ,

$$\begin{aligned} \delta_t(x; R_t^\theta) = \partial_t \log p_t(x) + \sum_{i=1}^{d} \sum_{\tau \neq x_i} \left(\big[\, G_t^\theta(\tau, i \mid x)\,\big]_+ - \big[\, -G_t^\theta(\tau, i \mid x)\,\big]_+ \frac{p_t(y)}{p_t(x)} \right), \\ y = \mathrm{Swap}(x, i, \tau), \end{aligned} \tag{3.8}$$

the first bracket being the outflow rate for $x \to y$ and the second the inflow $y \to x$ recovered from the same tensor through (3.7). For example, if one flip has $G_t^\theta(\tau, i \mid x) = -2$ and $p_t(y)/p_t(x) = 3$,

[1] For an arbitrary G_t^θ the two rectified rates of a pair need not be one-way; they are once G_t^θ is antisymmetric under the site swap.



layer 1 mixes x_i into h_k; layer 2 reads h_k into H_i. Masking h_i at the top leaves the two-hop path open, so H_i still depends on x_i. Causal stacks preserve the prefix/suffix dependencies that keep x_i out of the shifted read; a final mask alone can't.

Figure 3.1. Unrestricted mixing followed by a final mask leaves the path $x_i \to h_k \to H_i$ open. Removing the self-edges shown here would not close that path, so naively stacking hollow mixing layers fails too. Causal stacks preserve the dependencies required by the shifted read; Chapter 5 meets the same issue with two holes.

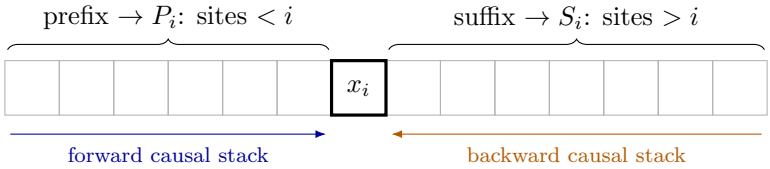


Figure 3.2. The one-hole decomposition behind the leTF. Flattened to a sequence, the lattice with site i removed is two intervals, and each is covered by a causal stack read one slot short of the hole: P_i has seen exactly $\{t, x_{<i}\}$, S_i exactly $\{t, x_{>i}\}$. Their fusion H_i is deep within each interval yet blind at i by construction, and one pass returns it for every i at once, which is what (3.9) reads out. Chapter 5 removes a second site, and the interval that opens between the two holes is the new object it has to build (Figure 5.3).

its forward rate is 0 and its reverse rate is 2. Its residual contribution is therefore $0 - 2 \times 3 = -6$, i.e. we're able to recover the inflow without evaluating the network at y. This is what we use for training; replacing $\partial_t \log Z_t$ in (3.8) by the detached control variate estimator and averaging the square over replayed (x, t) gives the implemented inner loss. In practice we clip the density ratio $p_t(y)/p_t(x)$, with the log cap of Table L.1, so that a large-ratio neighbour cannot dominate a mini-batch. This is a bias-variance trade-off, and the Stein identity need not hold.

Hollow architectures. So antisymmetry helps us achieve our efficiency goals, and the real work is building a network that satisfies it, where here we rely on hollowness. A network is hollow at site i if its hidden representation there, H_i, depends on the other sites x_{-i} but never on x_i itself [89]; reading the potential out bilinearly from such an H_i against learned token embeddings ω_τ,

$$G_t^\theta(\tau, i \mid x) = \big\langle\, H_i(x_{-i}, t),\ \omega_\tau - \omega_{x_i} \,\big\rangle, \tag{3.9}$$

makes the antisymmetry (3.7) hold by construction and the diagonal vanish automatically (swapping $x_i \leftrightarrow \tau$ leaves the hollow H_i untouched and flips the sign of $\omega_\tau - \omega_{x_i}$, and at $\tau = x_i$ the readout is zero). The one remaining question is how to make the hollow representation H_i expressive. The leTF of DNFS manages this by using forward and backward causal transformer stacks. Here causal means exactly the same as it does in the language model setting – slot i sees only slots $k \le i$, and stacking layers preserves that. The forward stream reads off just before i, and the backward just after it, providing two contexts blind to x_i which we can fuse to give a hollow H_i. It is this preserved ordering and shifted read that permit depth, argued diagrammatically in Figure 3.1. The result is equivariant to single-site token flips but not to permutations or translations, hence the term *local equivariance*. Something you'll notice is that the architecture doesn't take the spatial symmetry of the lattice into account – it is instead flattened to a one-dimensional sequence with positional encodings. We revisit this in both Chapter 5 and Section 6.3.

Figure 3.2 shows this construction pictorially – one hole, two intervals, each a causal stream, and the fused hollow context read out into all d flip rates in a single forward pass. Local equivariance makes evaluating the path-weight correction efficient, and allows us to recover

the importance-sampling ESS of Section 2.1.4. Rolling the learned CTMC forward produces a trajectory together with a path-integral importance weight, accumulated in log space:

$$\log w_t = \int_0^t \Big(\partial_s \log \tilde{p}_s(x_s) - \sum_y R_s(x_s, y) \, \frac{p_s(y)}{p_s(x_s)} \Big) \, ds, \tag{3.10}$$

whose integrand is the Kolmogorov residual evaluated along that trajectory, up to an x-independent term that cancels in the self-normalised weights [29]. A perfectly trained R_t^θ has zero residual, so every trajectory accumulates the same weight and the ESS attains its maximum of 1, up to time-discretisation error. The ESS is therefore both a sampler-quality diagnostic and an indirect read-out on training, and forms one of the headline metrics we use when it comes to evaluation, with the same *trust-but-verify* approach introduced in Section 2.1.4.

Closed-form channel. We can also add a known target feature to this potential, used by the critical 10×10 run below and throughout the constrained chapters, inspired by Zhu et al. [31]:

$$G(\cdot \mid x) = G_\theta(\cdot \mid x) + g(t) \, \Delta(\cdot \mid x), \qquad g(t) = g_0 + g_1 t, \tag{3.11}$$

where $\Delta(\cdot \mid x) = \log[\pi(y)/\pi(x)]$ is the log-ratio of the target itself, $\pi = p_1$, across the proposed move, with time entering through $g(t)$. Antisymmetry under move reversal preserves local equivariance, i.e. by using it we don't break our architectural constraints, and so we can add this to the network's output instead of having it try to learn it from the start. The learned scalars g_0, g_1 start at zero, preserving the matched backbone's initial output; at σ_c on the 10×10 lattice, they train to $g(1) \approx 0.7$–0.8 on every seed, so we see the sampler indeed using this closed-form we provide instead of relearning it.

3.2 Experimental Setup

Onto our experiments. The system throughout is the 2D Ising model of Section 2.1.3 on a periodic $D \times D$ lattice. We first study the 4×4 case, which is small enough to enumerate all 2^{16} states, and allows us to verify our sampler against a real enumerable "ground-truth". We then scale to 10×10 to match the headline figure from the DNFS paper at both the subcritical $\sigma = 0.1$ and critical couplings σ_c. The critical 10×10 run required a longer training budget (Table L.2), and a temperature curriculum (Section 3.3). The 4×4 runs need no such temperature curriculum, likely due to the smaller problem size being easier, and train at σ_c directly.

The sampler is a leTF trained on the Kolmogorov-residual objective with the Stein control variate, by the loop of Algorithm 1. Every setting is in Tables L.1 and L.2, beside the paper's own recipe for the same lattice; seeds are 42 to 45 throughout, and every evaluation draws 5000 samples, the size of the reference pool.

One thing to note about our buffer implementation relative to the paper – Algorithm 1 of Ou et al. [29] as printed only ever adds to the buffer, with no eviction; the buffer we use here is the bounded form (the last B outer cycles, line 5).

The 10×10 reference pools 100 independent Wolff cluster chains [22], each burned in for 500 clusters and recording 50 frames at intervals of 20 clusters; pooled $\hat{R} \leq 1.002$ at both couplings. The practitioner comparison uses eight `mchammer` semi-grand-canonical chains of 10^7 single-flip trials at $\Delta\mu = 0$, targeting the same unconstrained distribution.

3.3 Experimental Results

Our evaluation of the unconstrained sampler is guided by the principles of Section 2.1.4 – how well do we recover physical properties, and at what cost? Our model was built in five stages,

lattice	σ	F/d	E/d	S/d	ESS fraction
4×4	0.1	-3.6739 ± 0.0003	-0.455 ± 0.007	0.6438 ± 0.0014	0.9986 ± 0.0008
exact	0.1	-3.6786	-0.4561	0.6445	/
4×4	σ_c	-2.1844 ± 0.0006	-1.561 ± 0.004	0.2749 ± 0.0017	0.9864 ± 0.0036
exact	σ_c	-2.2014	-1.5656	0.2802	/
10×10	0.1	-3.6691 ± 0.0001	-0.427 ± 0.002	0.6484 ± 0.0004	0.9951 ± 0.0007
exact	0.1	-3.6727	-0.4282	0.6489	/
DNFS paper	0.1	-3.6709	-0.4271 ± 0.0010	0.6488	0.9985
10×10	σ_c	-2.1178 ± 0.0002	-1.474 ± 0.003	0.2839 ± 0.0011	0.9478 ± 0.0181
exact	σ_c	-2.1242	-1.4762	0.2855	/
DNFS paper	σ_c	-2.1120	-1.4743 ± 0.0068	0.2811	0.9685

Table 3.1. Thermodynamic evaluations: free energy, internal energy and entropy per site, and ESS fraction; mean ± sample standard deviation (SD) over seeds 42–45, 5000 draws each, fp32 evaluation. Exact references: enumeration at 4×4 and the Ferdinand–Fisher finite-lattice closed form at 10×10 [51]; paper rows from Ou et al. [29, Table 2]. E/d uses importance reweighting. The table uses energy units with $J_{\text{phys}} = 1$: its E is internal energy, while the path exponent uses $2\sigma E_{\text{phys}}$. $F/d = -\langle \log w \rangle / (2\sigma d)$ estimates the variational upper bound, with $S/d = 2\sigma(E/d - F/d)$. "/" not applicable. The paper labels its critical row $\sigma = 0.22305$, but its exact row matches σ_c: its printed $S/[2(E-F)] \simeq 0.2204$ also supports evaluation at σ_c. We retain the row under σ_c on this evidence.

ordered in such a way that each step fixed an issue the previous one carried. Reading the reference paper, for example, it is not immediately clear to a first-time reader what would happen if we used a non-locally equivariant architecture, or just how bad the variance might be. The ordering therefore was: bare multilayer perceptron (MLP), locally equivariant multilayer perceptron (leMLP), Stein control variate, locally equivariant convolutional neural network (leCNN), and the full leTF. We report only the final results for brevity; the paper's own ablations cover a similar progression (its Fig. 1 for the estimator, Fig. 3 for the architectures).

We present two results tables here, the first of which is in-line with the DNFS paper (Table 3.1), and a second which will form our standard evaluation table throughout the thesis (Table 3.2). The former includes the closed-form Ferdinand-Fisher thermodynamic properties and so gives us an idea of proximity to a ground-truth, even though enumerating all 2^{100} configurations is out of reach. In the latter we start as we mean to go on, comparing our larger lattice results to a long burned-in MCMC chain, here Wolff. At 4×4, Appendix I compares the sampler's unweighted configuration frequencies with exact enumeration.

We observe that our reweighted energies agree with the exact values to within a few thousandths per site. On the larger 10×10 lattice at criticality, our ESS is 0.948 ± 0.018 against the paper's 0.9685, within an acceptable tolerance to consider our implementation sound. Table 3.2 shows broad agreement with the reference, with correlation errors above the independent-draw floor at both couplings, especially at criticality (11.6 ± 1.9 against 6.9). The clearest advantage over `mchammer` is the critical magnetisation profile (8.2 ± 3.1 against 91 ± 59, at the floor); energy errors are similar and the correlation errors overlap.

Figure 3.3(b) checks final mode coverage. Across the four seeds, each magnetisation sign receives 50% of the importance-weighted mass within half a percentage point; 0.14–0.25% lies at zero.

Something which likely stands out in our results is the fairly punchy cost column. At criticality, neural sampling costs about 25,000 times the printed `mchammer` FLOP/es and 770,000 times Wolff's, excluding training. An interesting feature of the neural sampler however is that its cost barely moves with the coupling – the $2.1\times$ rise across the two columns is the critical recipe's doubled Euler grid, and on a matched grid it would be about 5%, against $85\times$ and $20\times$ for the chains. The fixed architecture and Euler grid explain that, though measurements at one lattice size do not establish a size-scaling advantage. Not all FLOPs are created equal however, as batching affects runtime, and amortisation spreads training cost across uses, though neither of these removes the mechanical sampling cost printed here.

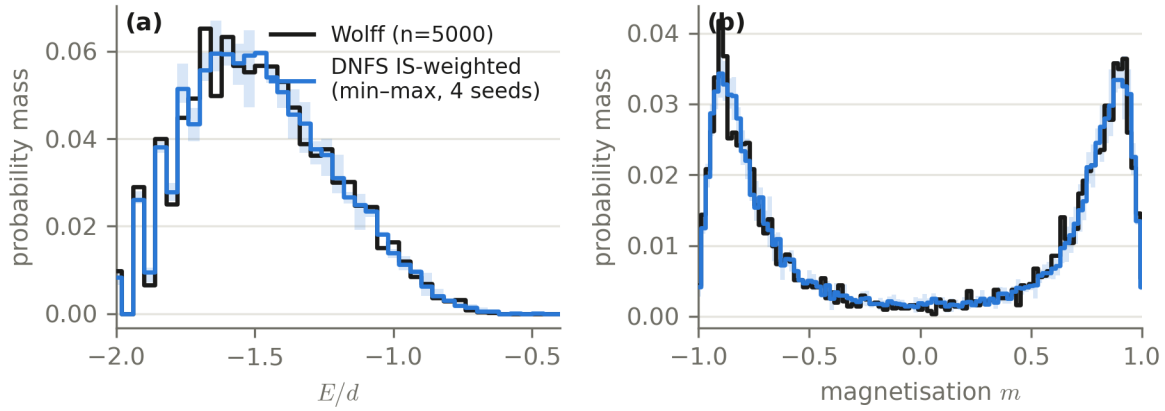


Figure 3.3. DNFS on the unconstrained 10×10 Ising at the critical coupling σ_c, against the pooled Wolff cluster reference: importance-weighted samples, mean and min–max band over four seeds. (a) Energy marginal on the exact energy levels, per site. (b) Magnetisation marginal: both Z_2 modes covered with equal mass in a single sampling pass. Total variation against the reference, beside the 5000-independent-draw resampling floor (200 replicates): 0.053 ± 0.005 (floor 0.029) on energy, 0.082 ± 0.003 (floor 0.049) on magnetisation.

	$\sigma = 0.1$					$\sigma = \sigma_c$				
Method	ESS↑	ΔMag↓	ΔCorr↓	EW_2 ↓	FLOP/es↓	ESS↑	ΔMag↓	ΔCorr↓	EW_2 ↓	FLOP/es↓
Reference (Wolff)	/	(5.7)	(7.5)	(0.8)	2.8×10^3	/	(10.2)	(7.0)	(1.0)	5.5×10^4
sampling floor at $N = 5000$	/	4.6	6.7	1.0	–	/	9.7	6.9	1.3	–
mchammer (SGC)	/	6.0 ± 0.8	8.6 ± 0.7	1.2 ± 0.3	2.0×10^4	/	91 ± 59	26 ± 19	1.7 ± 0.2	1.7×10^6
DNFS (leTF)	0.995 ± 0.001	6.9 ± 1.1	9.1 ± 0.7	1.3 ± 0.2	2.0×10^{10}	0.948 ± 0.018	8.2 ± 3.1	11.6 ± 1.9	1.4 ± 0.1	4.2×10^{10}

Table 3.2. Sampler evaluation on 10×10 unconstrained Ising. Columns: ESS fraction (2.3), magnetisation-profile and two-point-correlation absolute errors [31], energy-per-site Wasserstein-2 distance [32], and FLOPs per effective sample. The three errors are in units of 10^{-2}. DNFS: mean ± SD over four seeds, 5000 importance-reweighted draws each; `mchammer`: mean ± SD over eight chains. Parentheses give approximate reference uncertainty; the floor averages 200 replicates of 5000 independently resampled frames. See Appendix C.3 for definitions and Section 3.2 for chain recipes. At σ_c, ΔMag resolves the residual imbalance between the two Z_2 modes, whose exact mean profile is zero. "/" not applicable.

Two engineering additions were important for training stability, especially in the harder regimes, and will be carried with us throughout the thesis. Each was a development-time observation rather than a controlled ablation:

- **Learning-rate warmup** (500 steps). We observed that training is very sensitive to the initialisation, and without a warmup, seeds often died prematurely. Its addition allowed us to run more reliable multi-seed experiments.
- **Temperature curriculum.** Inspired by MDNS [31], we added a temperature curriculum to allow for smoother training. It was often the case that for larger lattice sizes, attempting to train directly at critical temperature was unstable and struggled to converge. We raise σ in stages from 0.10 up to σ_c, easing the learning rate down as we near σ_c, then hold at σ_c for the rest of training. Hyperparameter specifics can be found in Table L.2, as well as training curves displaying this behaviour in Appendix H.

3.4 Rejection Sampling

Before we embark upon an uncharted path toward natively constrained samplers, you might well wonder, what would it cost to request samples that satisfy a specific constraint from the one we already have? This is the type of question that motivates our work, fixed composition being a necessary requirement for alloys of given stoichiometry. The simplest way to reuse our trained

sampler is using it in rejection sampling, i.e. draw samples from it and only keep those that hit the composition we care about, discarding the remainder, reusing their original importance weights to estimate the conditional target. Table 3.3 demonstrates its cost, at 21–526 draws per retained sample in our 10×10 case. We observe that rejection becomes more expensive away from centre in the subcritical regime, whereas the opposite is true at criticality (though rejection overhead is generally higher otherwise). This table only measures proposal acceptance at one size however, and we claim no scaling law for learned samplers. It does however motivate changing the approach in the next chapters.

	$\sigma = 0.1$		$\sigma = \sigma_c$	
requested c	accepted	overhead	accepted	overhead
0.50	$4.75 \pm 0.32\%$	$21\times$	$0.21 \pm 0.06\%$	$488\times$
0.40	$2.36 \pm 0.23\%$	$42\times$	$0.19 \pm 0.03\%$	$526\times$
0.30	$0.24 \pm 0.05\%$	$408\times$	$0.28 \pm 0.11\%$	$357\times$
uniform spins, $c = 0.50$	7.96%, $13\times$			

Table 3.3. Rejection at fixed composition $c = n_+/d$ from the unconstrained 10×10 sampler: raw acceptance (mean $\pm$ SD over four seeds, 5000 draws each) and its reciprocal, the cost per retained draw, not per effective conditional sample. The uniform benchmark is $\binom{100}{50}/2^{100}$. At σ_c, the three cells contain 41, 38 and 56 retained draws, respectively, pooled over seeds.

3.5 Chapter Summary

This chapter established a working multi-seed implementation of DNFS and formed the basis from which we'll build throughout the remainder of the thesis. Our implementation recovers known thermodynamics closely, with only residual errors. The cost of inference in our neural sampler stays fairly flat across couplings, though at training-time, criticality needs extra effort. Rejection proposes a potential route to the exact composition, though its cost off the unconstrained sampler proves a little rich. This gives us sufficient motivation to look at both the target and move set in the following chapters.

4 Constrain with a Penalty

Our journey continues with a study of soft constraints, where composition is encouraged with a penalty rather than enforced by construction. Here, since the only change is biasing the target distribution, we are able to use the same unconstrained sampler of Chapter 3 (or indeed any of our surveyed energy-based samplers from Section 2.2.3). This approach forms the standard first move for various methods, many outside of the materials setting we find ourselves in. One we're likely all familiar with is KL-regularised preference fine-tuning of language model samples $\pi(x) \propto p_{\text{ref}}(x) \exp(r(x)/\tau)$, where a pretrained model p_{ref} is tilted by a reward, with τ setting the tilt's contribution [90]. Though we're using transformers, we aren't looking at language, instead using a similar idea applied to the Ising alloy. Our *reward* here is the negative quadratic composition cost, with λ replacing the role of $1/\tau$. In contrast to preference fine-tuning however, we won't be adjusting a pretrained sampler from Chapter 3, but training one from scratch on this new tilted target. We posit that the free sampler simply places too little mass near the requested composition for inference-time reweighting to work (Table 3.3), though this remains conjecture.

This type of soft penalty also appears in a couple of the neighbouring papers first met in Chapter 2, with varying applications – the quadratic penalty is the Moreau envelope of the hard indicator in constrained protein diffusion [82], and MetaDNS [49] instead uses an adaptive bias to encourage mode exploration in the sampler, then reweighting to the unconstrained target. JANUS applies the same quadratic tilt at inference, steering a pretrained alloy sampler to a prescribed Warren–Cowley short range order parameter with sequential Monte Carlo instead of retraining, and caps defect counts with a softplus penalty inside its target [33].

There is also direct precedent on the materials side, in the VCSGC ensemble [26]. A quadratic penalty on the composition (4.1) is added to the energy function to bias the distribution and allow us to trace the free energy $F(c)$ across compositions at fixed temperature, through multi-phase regions which would otherwise be missed by the SGC. These curves are used by practitioners for temperature–composition phase diagrams, and so a learned soft sampler is a direct neural replica of this approach, one we benchmark against, and exactly where we begin.

4.1 The Penalised Target

Writing the Ising energy through its coupling matrix as in Chapter 3, the penalised target is

$$\pi(x) \propto \exp\big(x^T J x - \lambda\, d\, (c(x) - c_{\text{target}})^2\big), \tag{4.1}$$

where λ controls constraint strength and d (the number of sites) makes the penalty such that a single λ stays comparable across lattice sizes (though not scaled by inverse temperature, i.e. λ means the same thing at every coupling). This is the VCSGC ensemble (Table 2.2); `mchammer` subtracts $\kappa N(c + \phi/2)^2$ from the log weight, much like our penalty. Setting $\phi = -2c_{\text{target}}$ completes the square to $\kappa N(c - c_{\text{target}})^2$, which is (4.1)'s penalty at $\kappa = \lambda$, $N = d$.

Annealing path. The sampler follows the same geometric path from the base η as Chapter 3,

$$\log \tilde{p}_t(x) = (1 - t)\, \log \eta(x) + t \left[\, x^T J x - \lambda\, d\, (c(x) - c_{\text{target}})^2 \,\right], \tag{4.2}$$

so at any intermediate time along the trajectory, the target distribution contains both coupling and penalty scaled by t, with the base still present at weight $1 - t$; this is discussed further in both the matched base Section 4.3.2 and the transfer Section 4.5.

Penalty width. Splitting the target distribution into a product of exponentials yields $\pi(x) \propto \exp(x^T Jx) \exp(-\lambda d\,(c - c_{\text{target}})^2)$, so our penalty multiplies the Ising target by this Gaussian envelope $\exp(-\lambda d\,(c - c_{\text{target}})^2)$ centred at c_{target} with width $1/\sqrt{2\lambda d}$. Though we experiment with different values of λ, the one we focus our headline results on is $\lambda = 50$, since on an 8×8 lattice this gives a width of 0.0125, just under one site of the 64, so a typical draw sits within a site of the target, though half the mass at 8×8 still lies off the exact slice (Table 4.5). This is similar in nature to the way VCSGC [26] chooses their penalty tightness, though the difference there is that a chain's acceptance only suffers at very large penalties, where in our learned case we pay for composition proximity in training variance even at small sizes, explored in Section 4.3. Something we must keep in mind throughout though is that this envelope is positive for every composition, and for any finite λ the distribution we are sampling from is not π_{hard}. Section 4.6.1 discusses how far off we remain after the soft training process.

4.2 Experimental Setup

Here we again run enumerable checks on 4×4 Ising, growing to 8×8, at both critical and subcritical couplings. This convention diverges from Chapter 3 (whose largest lattice size was constructed to match that of the DNFS paper) and will continue throughout our constrained journey. Here we use even scaling as it makes it easier to run constrained evaluations on integer sites (if we later scaled to 15×15 say, there is no $c = 0.5$).

Trained composition windows are $c_{\text{target}} \in \{0.25, 0.375, 0.50\}$ to allow every window to be an integer site count at $d = 64$, with $\{0.625, 0.75\}$ obtained free by the Z_2 $c \mapsto 1 - c$ mirror. $\lambda = 50$ is fixed, with no annealing schedule unless stated otherwise. We will refer to a "house" recipe throughout, which just refers to our standard setup. There are three main additions to Chapter 3's setup (Section 3.2): the exact-field channel of Section 4.3.1, a compiled model for efficiency, an exponential moving average of the weights evaluated alongside the raw ones.

When we call a model a *specialist*, we mean that this model was trained for one composition, and one coupling. A *conditioned sampler* is one per coupling, trained to amortise across compositions, over seventeen values one site apart from 0.25 to 0.5 at 8×8, the three windows at 4×4, drawn at the composition's own matched base. A *twin* means a model with only one change against the *house* recipe, so a twin pair differs by exactly one declared setting and helps to narrow down effects of making individual changes. Finally, the σ *ladder* reuses the baseline temperature-curriculum idea with a shorter schedule: seven stages from $\sigma = 0.1$ to σ_c over the first 30k steps. Every number is read from the fp32 importance-sampling evaluation of the final weights, at 5000 draws.

4.3 Tackling Penalty Variance

Before we move to our headline results of Section 4.4, we must first acknowledge and overcome the elephant in the room. Variance. The penalty enters the Kolmogorov residual with coefficient λ, and initial experiments (and the below theory) show that this share of the residual variance dominates in early training, and motivates our stabilisation attempts. We therefore explore three approaches to reducing the impact of that variance and ablate across them, carrying the positive cases to our final models.

We can put our finger on exactly where this variance comes from – rewriting the Kolmogorov residual δ as $\delta = \delta_I - \lambda\delta_P$, where δ_P is the centred penalty and δ_I includes the energy and Stein terms (Appendix E), we observe that for each fixed λ,

$$\text{Var}[\delta] = \text{Var}[\delta_I] + \lambda^2\ \text{Var}[\delta_P] - 2\lambda\ \text{Cov}[\delta_I, \delta_P]. \tag{4.3}$$

So, how does this look under our uniform base η? If we set $P = d(c - c_{\text{target}})^2$ and $\mu = 1/2 - c_{\text{target}}$, the binomial composition gives two terms (Appendix E):

$$\operatorname{Var}_\eta[P] = \underbrace{\mu^2 d}_{\text{mean offset}} + \underbrace{\tfrac{1}{8}\big(1 - \tfrac{1}{d}\big)}_{\text{fluctuation}}, \qquad \lambda^2 \operatorname{Var}_\eta[P] = \begin{cases} 308 & c_{\text{target}} = 0.5, \\ 10308 & c_{\text{target}} = 0.25, \end{cases} \tag{4.4}$$

at $\lambda = 50$, $d = 64$. The first term here is the base's mean sitting off the window, which collapses at the centre, while the second is the base's own composition spread, present at every c_{target}.

Fear not, however. We have a war chest full of interesting instruments we can leverage to help navigate these trying training conditions. The first is aforementioned and loosely inspired by the preconditioner of MDNS [31], first covered in Section 3.1.3, and explore further in Section 4.3.1, with a view to tackling the second term above. The second is something we will carry into Chapter 5, and that's deviating from the uniform base distribution to one that is closer to our target, making the transportation problem just a little easier. This is explored in Section 4.3.2 and works on the first term in the decomposition. The final one we tested is scaling λ itself, both by varying it in absolute terms and using an annealing schedule inspired by our temperature curriculum in Section 3.3, which we found exchanges proximity to the hard target for training stability and sampling efficiency (Table 4.1).

		ESS↑	
lever	setting	$\sigma = 0.1$	$\sigma = \sigma_c$
exact-field channel, $c_{\text{target}} = 0.50$	off	0.79 ± 0.13	0.005 ± 0.006
	on (house)	0.95 ± 0.01	0.71 ± 0.03
base, $c_{\text{target}} = 0.25$	uniform	0.23 ± 0.06	0.010 ± 0.015
	matched (house)	0.94 ± 0.05	0.81 ± 0.02
base, $c_{\text{target}} = 0.375$	uniform	0.81 ± 0.14	0.48 ± 0.15
	matched (house)	0.96 ± 0.01	0.71 ± 0.03
penalty strength, $c_{\text{target}} = 0.50$	$\lambda = 10$	0.998 ± 0.000	0.917 ± 0.010
	$\lambda = 50$ (house)	0.95 ± 0.01	0.71 ± 0.03
	$\lambda = 100$	0.24 ± 0.19	0.15 ± 0.12

Table 4.1. The three training instruments against their ablations, 8×8 Ising, $\lambda = 50$ unless stated, mean $\pm$ SD over seeds 42–45 from the raw final weights, the same runs as Table 4.3. The channel pair and the λ block are at the centre window, where the base is already matched; the base pairs are off centre, where the channel is on in both rows. Every λ that trains matches its predicted Gaussian envelope: std(c) at $\sigma = 0.1$ reads 0.027, 0.0125 and 0.009 for $\lambda = 10$, 50, 100 against the envelope width $1/\sqrt{2\lambda d} = 0.028$, 0.0125, 0.0088.

4.3.1 Closed-form residual of the penalty

For the target of (4.1) the response of the log-density to a single flip is closed form,

$$\Delta_i(x) \;=\; x_i\Big[-4\sigma h_i \;+\; 2\lambda\big(c_{\setminus i}(x) - c_{\text{target}}\big) \;+\; \tfrac{\lambda}{d}\Big], \qquad h_i = \textstyle\sum_j A_{ij}x_j, \quad c_{\setminus i}(x) = c(x) - \tfrac{1+x_i}{2d}, \tag{4.5}$$

with $c_{\setminus i}$ the composition excluding site i (derivation in Appendix E). Here $\Delta_i(x) = \log \pi(x^{(i)}) - \log \pi(x)$ for $x^{(i)}$ the state with site i flipped. The first term is the Ising part's response and, since a flip moves c by $\mp 1/d$, the last two together are the penalty's. Our prediction from (4.3) is that the penalty term will dominate variance, and we now know its closed form, so we can provide the network (4.5) through the channel (3.11) of Section 3.1.3 instead of forcing the network to try and learn it. Figure F.2 in Appendix F shows how this develops (4.5) along one trajectory. We observe that when the count is below target, every down-spin flip is favoured across the whole lattice, which reverses sign as we cross it.

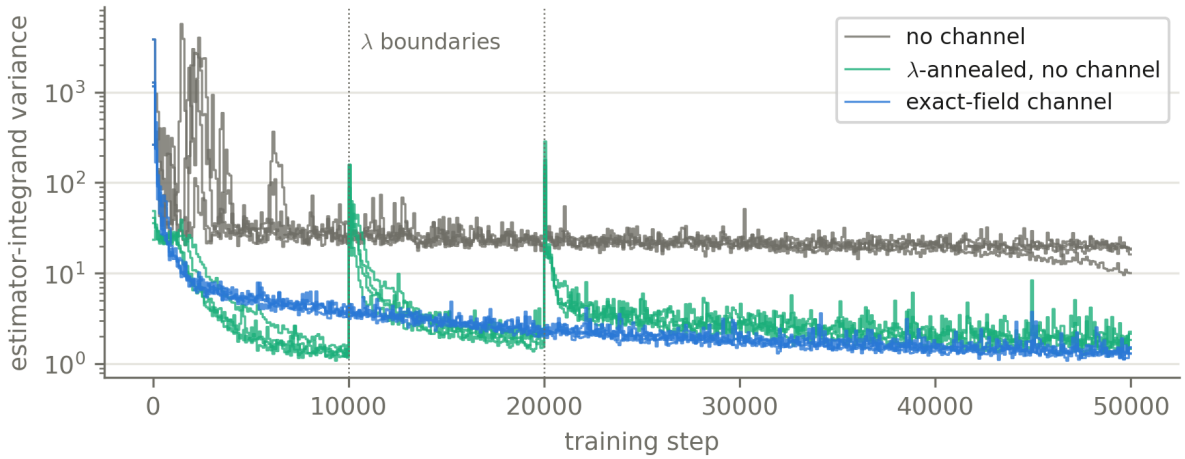


Figure 4.1. The penalty-variance shock and its three fates at the critical coupling, 8×8, $\lambda = 50$, $c_{\text{target}} = 0.5$: estimator-integrand variance over early training (rolling medians over 51 logged steps) for the channel-off control, the λ-annealed schedule and the exact-field channel. Final ESS: control 0.005 ± 0.006, all seeds denegerate; annealed 0.395 ± 0.21, three of four seeds, repaying at each λ boundary; channel 0.71 ± 0.03. Late plateaus: control 13 to 21, annealed 1.6 to 2.1, channel 1.3 to 1.4.

We indeed find that the channel helps overcome the training shock at criticality (Table 4.1), converting a set of degenerate samplers into healthy ones, and we also see a reasonable uplift subcritically. An interesting point to note here is that even the degenerate cases manage to hold the requested soft composition, so what they struggle to learn without the channel is the Ising physics. Figure 4.1 shows this effect too throughout the training process, where we have plotted the estimator-integrand variance (after taking into account the Stein control variate). The difference is night-and-day. We are able to achieve a respectable variance reduction by using λ annealing, though at each boundary point we see an unwieldy spike which is not observed in the channel case. The less shocks we provide to the training process, the better, and so our *house* recipe is almost decided.

4.3.2 A matched base for off-centre targets

The uniform base is a good choice for zero-field Ising, as well as one with constraint close to centre, since its up-count is Binomial$(d, \frac{1}{2})$ with most of the mass sitting at that central point.

Off-centre however, a natural replacement is a Bernoulli(p) base with $p = c_{\text{target}}$. This coincides with the uniform base at the centre and has log-density

$$\log \eta_p(x) = d \left[c(x) \log p + (1 - c(x)) \log(1 - p) \right] \tag{4.6}$$

which varies with composition, and removes the linear d term from the penalty's variance under the base (Appendix E). Along the geometric path $\partial_t^2 \log Z_t = \text{Var}_{p_t}(\log \tilde{\pi} - \log \eta)$, and integrating gives the Jeffreys divergence between base and target (Appendix E.2); at $c_{\text{target}} = 0.25$ on 8×8 the matched base cuts the penalty's contribution to it from 212.5 to 9.4 nats.

Across our ablations we saw that using the matched base repairs off-centre windows (Table 4.1), with increased impact as we stray further from the centre. Again, once we have observed this behaviour it makes complete sense – the Bernoulli$(\frac{1}{2})$ base puts binomially less mass near the requested composition than near the centre, which makes the transportation problem much harder than it needs to be.

4.3.3 Penalty strength

Something else we must factor in when applying a penalty of this form is λ itself, a hyperparameter. $\lambda = 50$ was chosen for a reason, but what's stopping us ramping that up to get our composition closer to the hard target? Table 4.1 gives an initial idea, and one you're probably expecting by now. Loosening λ helps to stabilise training, as we see when it's lowered to 10, though doing this causes us to stray further from π_{hard} which is less than ideal. Doing the opposite is harmful for a different reason, in that seeds start to collapse and training becomes more unstable. Doubling λ from 50 to 100 only narrows the penalty envelope by $\sqrt{2}$, while its contribution to the residual variance under the uniform base scales by $2^2 = 4$ – this scaling of width at $\lambda^{-1/2}$ versus variance contribution of λ^2 feels like a poor trade to me.

4.4 Headline Results

4.4.1 Enumerable check

Now onto our house experimental results with all these options decided. The 4×4 lattice results can be seen in Table 4.2. At $\sigma = 0.1$ every $\lambda = 50$ the observable errors are roughly comparable to the sampling floor, whereas at σ_c we already see some degradation in terms of sampling efficiency and fidelity. Perhaps surprisingly, the central composition $c = 0.5$ is the worst performer at criticality; all that work to save its neighbours! While the penalty itself is coupling-independent, we observe some degradation in sampling efficiency, as well as up to a doubling in observable errors when compared to the sampling floor. We attribute this to a combination of factors, the penalty and the harder-to-handle critical coupling, which also hurt training in many of our examples in Chapter 3 (and more to come). While it remains to be shown, we do expect that allowing the critical cells to train for longer than their subcritical counterparts would see an equalisation, albeit at slightly higher training cost.

We also see some deviation from the fully enumerated distribution – for the $c_{\text{target}} = 0.5$ specialist, Figure I.1 in Appendix I shows total-variation distance above the exact-draw reference at both couplings – about twice at $\sigma = 0.1$, and about five times at σ_c.

	$\sigma = 0.1$					$\sigma = \sigma_c$				
Method	ESS↑	ΔMag↓	ΔCorr↓	EW_2 ↓	FLOP/es↓	ESS↑	ΔMag↓	ΔCorr↓	EW_2 ↓	FLOP/es↓
exact enumeration (reference)	/	0	0	0	–	/	0	0	0	–
$c_{\text{target}} = 0.25$										
sampling floor at $N = 5000$	/	1.9	3.5	3.0	–	/	2.1	3.9	2.7	–
soft DNFS specialist	0.879 ± 0.086	2.7 ± 0.8	4.3 ± 1.1	5.0 ± 0.2	7.4×10^8	$0.602 \pm 0.341^{\ddagger}$	3.8 ± 2.9	7.2 ± 6.0	2.9 ± 1.2	2.3×10^9
$c_{\text{target}} = 0.375$										
sampling floor at $N = 5000$	/	2.3	3.8	3.3	–	/	2.6	4.3	2.8	–
soft DNFS specialist	0.845 ± 0.019	2.5 ± 0.5	5.0 ± 1.6	3.2 ± 0.7	7.6×10^8	0.706 ± 0.036	3.3 ± 0.3	6.3 ± 0.4	3.0 ± 0.1	9.2×10^8
$c_{\text{target}} = 0.50$										
sampling floor at $N = 5000$	/	2.3	3.6	3.5	–	/	2.6	4.2	3.2	–
soft DNFS specialist	0.913 ± 0.012	2.7 ± 1.1	4.7 ± 0.5	4.6 ± 1.0	7.1×10^8	0.526 ± 0.041	5.4 ± 1.8	8.0 ± 2.8	4.3 ± 1.8	1.2×10^9

Table 4.2. Evaluation table, 4×4 Ising under the composition penalty at $\lambda = 50$, the house recipe at the cross-chapter 4×4 budget of 10^4 training steps, seeds 42–45. Columns as Table 3.2; error columns in units of 10^{-2}. The floor row is the mean error of 5000 independent exact draws (multinomial draws from the enumerated target, 200 replicates). ‡One seed's raw final weights carry a weight tail (ESS 0.099; the same seed's exponential moving average (EMA) weights read 0.736, its siblings 0.69–0.84, and its observables match theirs).

4.4.2 The 64 site lattice

We then scale our targeted lattice to 8×8 and evaluate our learned sampler with the matched-base and exact field in Table 4.3, as well as in Figure 4.2.

We observe errors that are broadly comparable to the independent-draw benchmark, with several

	$\sigma = 0.1$					$\sigma = \sigma_c$				
Method	ESS↑	ΔMag↓	ΔCorr↓	EW_2 ↓	FLOP/es↓	ESS↑	ΔMag↓	ΔCorr↓	EW_2 ↓	FLOP/es↓
$c_{\text{target}} = 0.25$										
VCSGC reference	/	(2.0)	(2.4)	(0.6)	2.9×10^3	/	(1.0)	(1.3)	(0.3)	6.0×10^3
sampling floor at $N = 5000$	/	3.1	5.1	1.3	–	/	3.7	6.0	1.3	–
soft DNFS, matched base	0.94 ± 0.05	3.4 ± 0.4	5.2 ± 0.3	1.3 ± 0.2	2.7×10^{10}	0.81 ± 0.02	3.6 ± 0.9	6.9 ± 0.6	1.3 ± 0.1	3.2×10^{10}
$c_{\text{target}} = 0.375$										
VCSGC reference	/	(2.3)	(2.8)	(0.7)	3.3×10^3	/	(2.3)	(2.1)	(0.4)	6.7×10^3
sampling floor at $N = 5000$	/	3.5	5.9	1.4	–	/	4.3	6.9	1.4	–
soft DNFS, matched base	0.96 ± 0.01	4.3 ± 0.5	6.8 ± 0.8	1.7 ± 0.5	2.7×10^{10}	0.71 ± 0.03	4.8 ± 0.4	8.4 ± 0.8	1.5 ± 0.3	3.6×10^{10}
$c_{\text{target}} = 0.50$										
VCSGC reference	/	(3.0)	(2.4)	(0.7)	3.2×10^3	/	(2.1)	(2.6)	(0.3)	7.5×10^3
sampling floor at $N = 5000$	/	3.5	5.8	1.4	–	/	4.6	6.9	1.4	–
soft DNFS specialist	0.95 ± 0.01	4.6 ± 0.8	6.5 ± 0.6	1.4 ± 0.3	2.7×10^{10}	0.71 ± 0.03	5.9 ± 1.3	8.1 ± 1.0	1.6 ± 0.1	3.6×10^{10}

Table 4.3. Sampler evaluation on 8×8 Ising with the composition penalty $\lambda = 50$. Columns as Table 3.2; errors in units of 10^{-2}, mean ± SD over all seeds 42–45 using raw final weights. Reference: four `mchammer` VCSGC chains per cell at $\kappa = \lambda$, $\phi = -2c_{\text{target}}$. Parentheses give approximate reference uncertainty; each separate floor row averages 200 replicates of 5000 independently resampled frames. Pools contain 26,672 frames at $\sigma = 0.1$ and 266,672 at σ_c; only four independent chains underlie each uncertainty estimate (Appendix C.3). Off-centre specialists use the matched base of Section 4.3.2. "/" not applicable.

above it. At the critical centre, for example, magnetisation-profile error is 5.9 ± 1.3 against a floor of 4.6, and correlation error is 8.1 ± 1.0 against 6.9, all in units of 10^{-2}. These are however within the reference uncertainties, and so perhaps a better comparison are the marginals of Figure 4.2, which display agreement across compositions.

4.4.3 Against the VCSGC ensemble

Outside of the metrics reported in the evaluation tables, we're able to equal a matched-κ VCSGC chain from the materials-side benchmark on composition mean and spread, energy and short-range order at every window and both couplings (Figure F.1, Appendix F), with the marginals of Figure 4.2 at the chains' own floor. The free energy $F(c)$ curve that VCSGC exists to provide [26] is also attainable here, using the same weights derived from the theory of Section 4.6.2, removing the need for additional thermodynamic integrations which would be necessary in the classical case. We compare these with the hard sampler in Figure 5.11, with the soft-only curve in Appendix F.1.

4.5 Amortisation and Transfer

The headline results so far have been encouraging, though each sampler was trained specifically for one composition. One of the main draws of neural samplers is their ability to amortise – if we're required to train one per composition we don't gain much compared to the classical tools that are required to do the similar with their chains. In Table 4.4, we find that our amortised sampler, trained at the same budget as the specialist, performs almost as well as the specialist subcritically, though at σ_c without the temperature curriculum it degenerates. The addition of the σ ladder was again necessary to rescue the larger lattice size, as seen in Chapter 3. While it remains to be shown, we suspect that if we were to train the amortised sampler for more training steps (it therefore seeing the compositions with more volume), we would be able to achieve parity with the specialist. The trade-off between steps-per-specialist and steps-per-amortised samplers is not something that we studied in-depth – the natural question is, if we train for N compositions, do we need to multiply our specialist training steps by N, or some value smaller than N? Need we do this if we'd prefer to spend more on sampling than training, implying we can live with a lower ESS? Something to consider.

We also thought about zero-shot transfer, though the simple coupling trick we're able to use

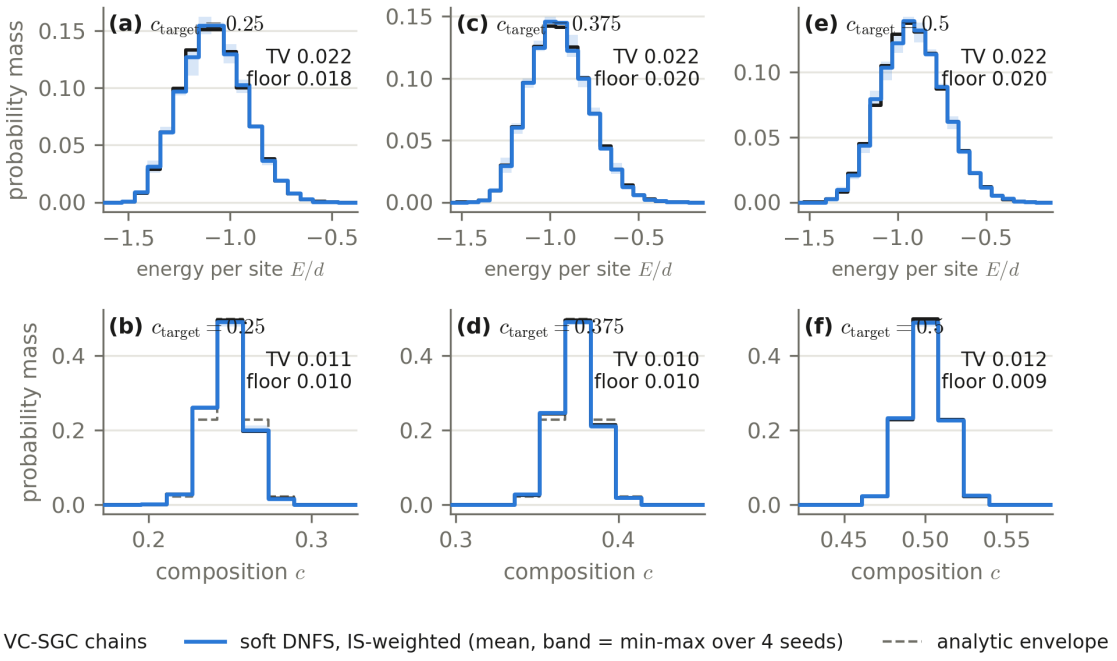


Figure 4.2. Critical 8×8 marginals at $\lambda = 50$, matched-base recipe: bare Ising energy per site (top) and composition (bottom) at each trained window. Black: pooled VCSGC reference; blue: importance-weighted DNFS mean with min–max band. Each panel reports seed-mean total variation against the reference and its 5000-independent-draw resampling floor (200 replicates), conditional on the pooled chains. Dashed: the analytic envelope $\propto \exp[-\lambda d(c - c_{\text{target}})^2]$, which omits the Ising composition marginal and its tilt toward half filling, most visible at $c_{\text{target}} = 0.25$.

in the hard case of Section 5.6.1 is not an option. Stopping early on (4.2) changes the penalty strength as well as the coupling, and the matched base contributes off centre. Re-centring a window's draws instead requires reweighting, something we experimented with and got poor yield and didn't explore further.

4.6 The Cost of Exact Composition

4.6.1 The Distributional Ceiling

So far we have mainly judged the soft constrained setting on its own merits; that we can indeed train one, and when we do it reproduces its target. It is also useful in replicating the materials side $F(c)$ curves and related metrics. If instead we're looking to leverage it to target the fixed-composition target, on its own, the sampler falls slightly short. At finite λ, the soft target is simply not the hard-constrained distribution, and even a perfect sampler of the incorrect distribution will return the wrong ensemble. We can however recover the right one, in theory.

Taking the limit in (4.1), the exponent is zero on the slice and tends to $-\infty$ off it, so

$$\lim_{\lambda\to\infty} e^{-\lambda d\,(c(x)-c_{\text{target}})^2} = \mathbf{1}[c(x) = c_{\text{target}}], \qquad \pi(x) \;\to\; \frac{e^{x^T Jx}\,\mathbf{1}[c(x) = c_{\text{target}}]}{\sum_{x'} e^{x'^T Jx'}\,\mathbf{1}[c(x') = c_{\text{target}}]}. \tag{4.7}$$

which is precisely the hard target of Chapter 5. Our learnings from Section 4.3 however tell us that this approach is neither practical nor plausible, and instead motivate rejection sampling.

4.6.2 Rejection Sampling

We can update our view on rejection sampling first seen in Chapter 3 since in theory we should be a little closer to the intended target once we take the penalty into account. If we condition

	$\sigma = 0.1$					$\sigma = \sigma_c$				
Method	ESS↑	ΔMag↓	ΔCorr↓	EW_2 ↓	FLOP/es↓	ESS↑	ΔMag↓	ΔCorr↓	EW_2 ↓	FLOP/es↓
4×4, $c_{\text{target}} = 0.25$										
specialist	0.879 ± 0.086	2.7 ± 0.8	4.3 ± 1.1	5.0 ± 0.2	7.4×10^8	$0.602 \pm 0.341^{\ddagger}$	3.8 ± 2.9	7.2 ± 6.0	2.9 ± 1.2	2.3×10^9
conditioned sampler	0.761 ± 0.208	2.0 ± 0.4	4.6 ± 0.8	3.5 ± 1.7	9.2×10^8	0.683 ± 0.149	3.1 ± 0.7	5.9 ± 0.8	3.7 ± 0.3	9.9×10^8
4×4, $c_{\text{target}} = 0.375$										
specialist	0.845 ± 0.019	2.5 ± 0.5	5.0 ± 1.6	3.2 ± 0.7	7.6×10^8	0.706 ± 0.036	3.3 ± 0.3	6.3 ± 0.4	3.0 ± 0.1	9.2×10^8
conditioned sampler	0.778 ± 0.142	2.8 ± 0.5	5.8 ± 0.3	3.3 ± 1.0	8.6×10^8	0.661 ± 0.044	3.3 ± 0.5	5.2 ± 0.9	3.6 ± 0.6	9.9×10^8
4×4, $c_{\text{target}} = 0.50$										
specialist	0.913 ± 0.012	2.7 ± 1.1	4.7 ± 0.5	4.6 ± 1.0	7.1×10^8	0.526 ± 0.041	5.4 ± 1.8	8.0 ± 2.8	4.3 ± 1.8	1.2×10^9
conditioned sampler	0.752 ± 0.159	4.9 ± 1.0	3.4 ± 0.6	3.9 ± 0.2	9.0×10^8	0.336 ± 0.032	7.1 ± 2.5	8.4 ± 2.2	5.7 ± 1.9	1.9×10^9
8×8, $c_{\text{target}} = 0.25$										
specialist, matched base	0.94 ± 0.05	3.4 ± 0.4	5.2 ± 0.3	1.3 ± 0.2	2.7×10^{10}	0.81 ± 0.02	3.6 ± 0.9	6.9 ± 0.6	1.3 ± 0.1	3.2×10^{10}
conditioned sampler	0.74 ± 0.07	4.2 ± 0.9	6.5 ± 0.6	1.4 ± 0.2	3.5×10^{10}	0.009 ± 0.005	†	†	†	†
σ ladder	/	/	/	/	/	0.39 ± 0.02	5.1 ± 1.4	10.3 ± 0.5	2.6 ± 0.3	6.5×10^{10}
8×8, $c_{\text{target}} = 0.375$										
specialist, matched base	0.96 ± 0.01	4.3 ± 0.5	6.8 ± 0.8	1.7 ± 0.5	2.7×10^{10}	0.71 ± 0.03	4.8 ± 0.4	8.4 ± 0.8	1.5 ± 0.3	3.6×10^{10}
conditioned sampler	0.88 ± 0.01	4.5 ± 0.9	6.7 ± 0.3	1.3 ± 0.1	2.9×10^{10}	0.005 ± 0.005	†	†	†	†
σ ladder	/	/	/	/	/	0.37 ± 0.11	8.0 ± 1.4	13.6 ± 1.4	2.0 ± 0.5	7.4×10^{10}
8×8, $c_{\text{target}} = 0.50$										
specialist	0.95 ± 0.01	4.6 ± 0.8	6.5 ± 0.6	1.4 ± 0.3	2.7×10^{10}	0.71 ± 0.03	5.9 ± 1.3	8.1 ± 1.0	1.6 ± 0.1	3.6×10^{10}
conditioned sampler	0.79 ± 0.05	4.7 ± 0.7	6.5 ± 0.4	1.4 ± 0.4	3.3×10^{10}	0.003 ± 0.003	†	†	†	†
σ ladder	/	/	/	/	/	0.25 ± 0.08	6.9 ± 1.3	14.5 ± 4.2	2.1 ± 0.7	1.1×10^{11}

Table 4.4. Conditioned samplers against specialists, $\lambda = 50$. Specialist rows, columns and conventions as Tables 4.2 and 4.3; all seeds use raw final weights. Each conditioned model trains at one coupling for the specialist's budget over three windows at 4×4 or seventeen compositions one site apart in $[0.25, 0.5]$ at 8×8, and is evaluated on the draws at each requested composition; 8×8 requests use their matched base. The σ-ladder rows add the temperature curriculum (Table F.1). ‡ marks the raw-weight tail described in Table 4.2; † marks degenerate importance weights, for which reweighted observables and cost per effective sample are withheld. "/" not applicable.

the soft target of (4.1) on the composition slice $\mathcal{C} = \{x : c(x) = c_{\text{target}}\}$ we get

$$\pi(x \mid x \in \mathcal{C}) = \frac{e^{x^T J x}\, e^{-\lambda d(c(x) - c_{\text{target}})^2}\, \mathbf{1}[x \in \mathcal{C}]}{\sum_{x' \in \mathcal{C}} e^{x'^T J x'}\, e^{-\lambda d(c(x') - c_{\text{target}})^2}} = \frac{e^{x^T J x}\, \mathbf{1}[x \in \mathcal{C}]}{\sum_{x' \in \mathcal{C}} e^{x'^T J x'}}, \tag{4.8}$$

which is precisely the hard-target of Chapter 5, for every λ, without needing to take the limit. All λ does it influence the acceptance rate. Since we already have the importance weights, all we must do is keep the draws that land on $\mathcal{C}$ and we have importance samples from π_{hard}. The results of doing so for the samplers in this chapter can be found in Table 4.5. Something we notice here is that the acceptance rate drops from nearly 1 in the 4×4 case to 0.5 in 8×8, scaling poorly with size (predicted overhead of $\sqrt{\pi d/\lambda}$ from the penalty envelope), and while we could combat that by trying to increase λ, in practice it's not quite that easy. The results are not quite as catastrophic as rejecting off the unconstrained sampler as in Table 3.3, though definitely not free, still costing discarded draws.

We can also use the same rejection identify to give us the canonical normaliser – $Z_{\text{can}} = Z_\lambda\, \pi_\lambda(c = c_{\text{target}})$, and so we can use the soft sampler to recover canonical free energy estimates (Appendix F.1). The same shortfalls apply though, in that our cost is the rejected mass that falls away from the hard target. We meet this idea again in Section 5.7, which compares this approach with the swap estimate and thermodynamic integration.

So rejecting off the soft sampler makes sense if you already have a trained soft sampler, and we recommend it in that case; though for exact count we'd argue it's not a substitute for building the constraint in. The issue is that you first have to overcome the training stability issues that come with the soft sampler, which requires various tricks and struggles to generalise beyond its trained window. The cost of rejection also grows with lattice size; all of this to say we're prompted to look for alternatives.

		$\sigma = 0.1$			$\sigma = \sigma_c$		
	envelope	accepted	ESS on slice	FLOP/es	accepted	ESS on slice	FLOP/es
4×4							
$c_{\text{target}} = 0.25$	0.92	0.91 ± 0.01	0.88 ± 0.07	8.1×10^8	0.89 ± 0.01	0.63 ± 0.24	1.5×10^9
$c_{\text{target}} = 0.375$	0.92	0.91 ± 0.01	0.85 ± 0.02	8.4×10^8	0.88 ± 0.01	0.71 ± 0.03	1.0×10^9
$c_{\text{target}} = 0.5$	0.92	0.91 ± 0.01	0.92 ± 0.01	7.7×10^8	0.88 ± 0.00	0.52 ± 0.03	1.4×10^9
8×8							
$c_{\text{target}} = 0.25$	0.50	0.48 ± 0.01	0.95 ± 0.04	5.6×10^{10}	0.48 ± 0.01	0.81 ± 0.03	6.6×10^{10}
$c_{\text{target}} = 0.375$	0.50	0.49 ± 0.00	0.96 ± 0.00	5.4×10^{10}	0.48 ± 0.01	0.71 ± 0.02	7.5×10^{10}
$c_{\text{target}} = 0.5$	0.50	0.50 ± 0.01	0.96 ± 0.01	5.3×10^{10}	0.48 ± 0.01	0.72 ± 0.03	7.4×10^{10}

Table 4.5. Rejection from the soft sampler onto the exact composition slice, $\lambda = 50$, using all seeds 42–45 and the 5000 frozen draws per seed behind Tables 4.2 and 4.3 (matched base off centre). Envelope: exact-slice mass of $\propto \exp[-\lambda d(c - c_{\text{target}})^2]$ normalised over $c = k/d$, a coupling-free acceptance prediction. Accepted: measured fraction of draws on the slice, mean ± SD over seeds. ESS on slice: the survivors' self-normalised ESS fraction with their original weights, targeting the fixed-composition ensemble by (4.8). FLOP/es charges every draw, including rejections.

4.7 Chapter Summary

What you take away from this chapter will depend on your application. If you are interested in sweeping your composition and not necessarily needing to hit it exactly, it's the right tool and indeed analogous to the one used by materials practitioners today. Our specialists trained at criticality are able to reproduce properties of the VCSGC target closely (Section 4.4.3), with slice weights enabling us to estimate canonical free energies without thermodynamic integration. Subcritically we also demonstrate amortisation across the sweep with one conditioned sampler, where the equivalent tooling would require one equilibrated chain per window. The shortfalls can then be attributed to a mismatch in expectations, where we are using a soft constraint to try to hit a composition exactly. While possible in theory, the only λ that would close the gap is only in the degenerate limit, which would likely require training trickery beyond our experimental compute budget.

There is an alternative to taking this limit, as we've seen above, in rejection sampling. The idea here is to train to a sufficiently large λ and then reject onto π_{hard}. Our rejected mass is then proportional to how large we could get this λ. We have seen that this is fine in small and simpler cases, but scales quite poorly with lattice size as we are forced to reject proportionally more draws. This motivates the next chapter, where instead of fiddling with penalty terms we build the constraints into the sampler by construction, never needing to reject a sample as we never leave composition slice in the first place.

5 Constrain by Construction

This chapter represents everything we have been building towards, and marks the last ensemble of Table 2.2 – the canonical ensemble. Here our goal is to learn a neural sampler that holds $c(x) = c_{\text{target}}$ exactly without ever straying from it, using the symmetry of swap moves to leave composition unchanged, rather than distorting the target. We have so far shown that learned neural samplers are able to handle the unconstrained problem (Chapter 3), and while the soft-constrained sampler of Chapter 4 was able to go head-to-head with its materials counterpart, rejection still required us to discard many of the samples we drew. We borrow tools first introduced sixty years ago, the swap of Kawasaki dynamics [25], and attempt to provide a neural solution which can battle its well-known MCMC failure modes. We first measure how Kawasaki slows near criticality, then introduce our *Manifold Preservation Principle*, and show how we can build a CTMC with learned swap rates ensure continued composition adherence across the entire trajectory.

5.1 The Canonical Target

5.1.1 From penalty to indicator

Hard constraints are used in cases where the constraint is a necessary requirement – for example, a fixed stoichiometry in an alloy cell or a conserved particle number. The soft samplers are useful in cases where the composition is allowed to fluctuate about a target (Section 4.4.3), but for a single fixed composition its finite-λ bleed doesn't give us what we need. Hard constraints instead settle this by ensuring every sample sits on the target composition by construction.

We can view this as a restriction of the unconstrained target π to the feasible set $\mathcal{C} \subseteq \mathcal{X}$ through an indicator:

$$\pi_{\text{hard}}(x) \propto \exp[-\beta E(x)]\, \mathbf{1}[x \in \mathcal{C}], \tag{5.1}$$

which for a fixed composition is the set $\mathcal{C} = \{x : c(x) = c_{target}\}$. We saw in Section 4.6.1 that taking $\lambda \to \infty$ in the penalised target (4.1) returns this very same indicator; the difference here being we need to take no such limit and land on it by construction.

There are several ways to enforce this constraint directly, one of which we have seen already in previous chapters. Rejection sampling draws samples from a proposal distribution q and rejects any that violate the constraints, with acceptance fraction $q(\mathcal{C})$. Table 3.3 measured this rejection at 21–526 proposals per retained sample for the unconstrained sampler at 10×10, a rate which depends both on coupling and composition. The waste is reduced if we instead reject off the soft sampler, but by no means perfect (Section 4.6.2). The issue here is that acceptance requires mass on $\mathcal{C}$ by definition – less mass means more rejection. A cleaner approach awaits.

Move-restriction is the classical answer, with Kawasaki swap dynamics [25] its standard Ising instance (Section 2.3.2); which for our purposes, means replacing the single-site flips of DNFS with two-site swaps. A CTMC realises move-restriction by masking, zeroing the rate-matrix entries that leave $\mathcal{C}$; but masking a single-site move set won't work here, since every single-site flip changes the composition. The fix is to change the move set, exchanging flips for swaps, up-spins change places with down-spins, and preserve total count throughout. This same idea, choosing only moves that preserve validity, is also used in generative Markov chains for molecules [91]. A change in move set means a change in architecture, one that's able to emit these swap rates efficiently. We'll meet that in (Section 5.2). We need never evaluate the indicator of (5.1) at all, and simply start our chain on $\mathcal{C}$, take moves that preserve composition, and you'll never leave. There is a third move which we have seen already in our related-work, which is the use of

budgeting in a constructive generator. A masked sampler that reveals one site at a time can force the remaining species once a count is exhausted; which is precisely how JANUS reaches fixed composition [33], how our GFlowNet comparator masks its actions (Section 5.4), and what Appendix G tests on MDNS. Its intermediate states carry masks and lie on no slice, so the count is a property of the finished configuration rather than of the dynamics, and doesn't preserve composition throughout the trajectory as in our swap case.

So swaps may keep us at the right composition, but do they connect every configuration in $\mathcal{C}$? How can we be sure we explore the entirety of π_{hard} and not just some small corner of it?

The argument is as follows – take two configurations in $\mathcal{C}$ with the same number of up-spins. Wherever they disagree, there must be an excess up-spin at one site and a missing up-spin elsewhere. We can simply swap those two spins to make the configurations agree at both sites, which doesn't change the count and hence they don't leave $\mathcal{C}$. Repeating this until the configurations are equal removes every mismatch, and so pair swaps connect any two configurations in the set. This is the idea behind a *Markov basis* – moves that preserve specified counts and connect all configurations with those counts [92], and indeed fixed species counts alone remain connected by unlike-species swaps, even with more than two species. It is worth noting however that while we assert that every configuration is reachable, we still need to learn the rates to transport that probability correctly.

5.1.2 Kawasaki dynamics and critical slowing-down

Before we propose our neural case, it's important to understand how the classical MCMC methods actually cope with the constrained, low-temperature regime materials problems live in. We have some options to explore here – the cluster algorithms we saw in Section 2.1.3 that beat critical slowing-down on the plain Ising, Wolff and Swendsen-Wang [22, 21], can change the composition when they flip a cluster, which rules them out as a comparator from the beginning. Count-conserving cousins do exist – geometric spin-exchange clusters based on Hamiltonian symmetries [45], with related geometric-cluster methods for off-lattice fluids [46], but our comparison uses the Kawasaki moves behind the canonical-ensemble sampler in `mchammer`, [71], a popular tool for sampling alloy configurations (Section 2.3.4). We use the non-local variant (Section 2.3.2), which allows swaps between distant sites. The idea is simple – propose a swap of two unlike sites and accept it with the usual Metropolis ratio. Since the swap only exchanges occupations, the composition $c(x)$ is left untouched at every step, so the hard constraint $c(x) = c_{\text{target}}$ holds by construction. Ours will by no means be the first attempt at combining these two sampling paradigms; a recent paper learns to accelerate exactly this fixed-composition chain and reports beating plain Kawasaki [88], though since it still runs a Metropolis-corrected Markov chain at sampling time, it's more of a hybrid-classical bridge that an all-out neural approach.

So, how does Kawasaki fare? Figure 5.1 demonstrates the critical slowing-down failure mode, and tracks the integrated autocorrelation time τ_{int} and the normalised ESS as we raise the coupling σ from the subcritical DNFS operating point $\sigma = 0.1$ up through the critical point σ_c. At the operating point the chain mixes quickly, decorrelating in under a sweep, but decorrelation takes longer as σ approaches σ_c. On the 16×16 lattice τ_{int} climbs from about 0.9 to 12 sweeps between $\sigma = 0.1$ and σ_c, so the normalised ESS drops from ≈ 1 (roughly one independent configuration per sweep) to ≈ 0.08, which worsens with size. From our runs at σ_c we see a growth of $\tau_{\text{int}} \sim D^{1.5}$, a cost which is paid on every chain roll-out, and one that a neural sampler can amortise in one training run. While we can't avoid critical slowing down completely with a neural sampler, we can indeed amortise this cost once, and in an ideal world (no spoilers) also amortise across constraints which removes the need to run bespoke chains, and will be our attempt at tackling the three MCMC failure modes of Chapter 1 to the hard constrained problem specifically.

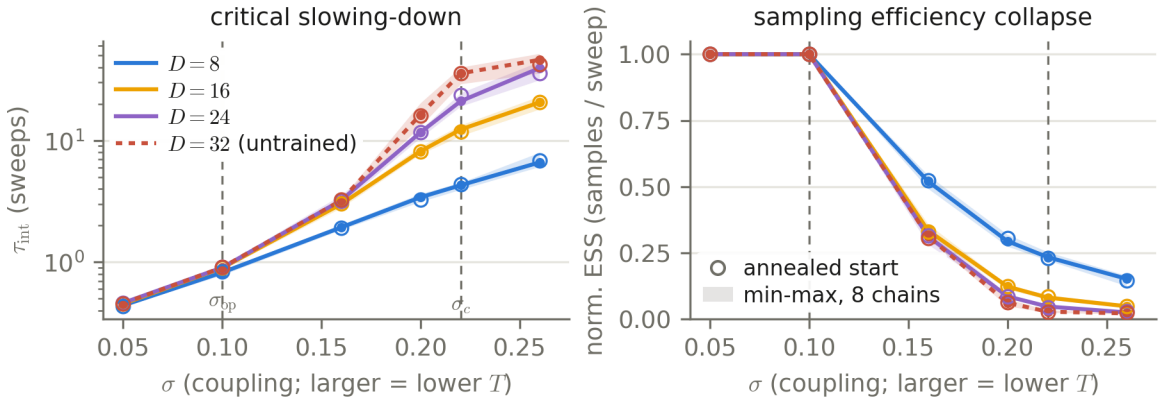


Figure 5.1. Critical slowing-down of Kawasaki dynamics on the hard-constrained ($c = 0.5$) Ising model: solid lines, $D \in \{8, 16, 24\}$, the sizes a neural sampler is trained at; dotted line, $D = 32$. Left: the integrated autocorrelation time τ_{int} of the energy per site, in sweeps of $d = D^2$ swap attempts; right, the normalised ESS per sweep, capped at 1. Every size spends the same budget per site, 30,000 measured sweeps after 3,000 of burn-in over 8 chains; open markers repeat every measurement from annealed initialisations walked up the same σ-ladder as the training curriculum.

5.2 A Neural Approach to Swaps

So, let's take everything we know from DNFS, and adapt it to composition-preserving swaps. We tackle this in four stages – use a base distribution and annealing path that live on the constraint manifold, a pair potential whose antisymmetry plays the role local equivariance played for flips, a closed form for the density ratios the residual needs, and a simulation scheme for a process whose moves, unlike flips, can conflict with one another. Let's call this method the *Manifold Preservation Principle*. Start every trajectory in $\mathcal{C}$, choose moves that preserve the constraint and connect $\mathcal{C}$, and learn their rates.

5.2.1 The on-manifold base and path

Write $\mathcal{C} = \{x : n_+(x) = N_A\}$ for the set of configurations at the requested composition, which we also call the composition slice, with $N_A = c_{\text{target}}\, d \in \{0, 1, \ldots, d\}$ up-spins. The base becomes the uniform distribution on the slice – sample by placing N_A up-spins on a uniformly random subset of the d sites, with density

$$\eta(x) = \binom{d}{N_A}^{-1}, \qquad x \in \mathcal{C}, \tag{5.2}$$

a constant on $\mathcal{C}$, with $\binom{d}{N_A}$ possible choices of the up-spin sites, each equally likely. The annealing path is the same log-linear Boltzmann interpolation as the unconstrained sampler, now restricted to the slice,

$$\log \tilde{p}_t(x) = (1 - t)\, \log \eta(x) - t\, \beta E(x), \qquad x \in \mathcal{C}. \tag{5.3}$$

After normalisation, this gives the uniform base at $t = 0$ and π_{hard} at $t = 1$. At intermediate times, $p_t(x) \propto \exp[-t\beta E(x)]$ on $\mathcal{C}$; as we increase t, the path gradually favours lower-energy configurations. A swap keeps both configurations on the same composition slice, where the uniform base assigns them equal probability, $\eta(y) = \eta(x)$. The base therefore cancels from their density ratio, leaving

$$\frac{\tilde{p}_t(y)}{\tilde{p}_t(x)} = \exp[-t\beta(E(y) - E(x))].$$

The unknown normaliser also cancels, as in the unconstrained case. Similarly, the uniform base contributes a constant to the path-weight integrand, here $\log \binom{d}{N_A}$ versus $d \log 2$. This cancels from normalised weights but is retained to recover the free energy (Section 5.7).

5.2.2 A pair potential antisymmetric under the swap

This section will feel very reminiscent of Section 3.1.3, and indeed it is. The main idea here is to replace the single-site flip emission of DNFS (Section 3.1.3) with paired swaps, and we'd like to leverage as much of that construction as possible. Recall that in the single-site case, the naive approach requires each state x to have its rate read from each of its d neighbours, and local equivariance (3.7) manages to cut this down to a single pass. The proposed swap approach has the same issue, though this time there are $N_A(d - N_A)$ distinct neighbours, one for each unlike-spin pair[1], so recovering their incoming rates without evaluating the network at those states becomes even more useful. Our construction uses a context for each pair that excludes both spin values, which we call *doubly blind*. Recovering the incoming rates this way leaves us with an architectural problem; how do we compute all of these contexts cheaply (Section 5.3)?

Let's start by writing the Kawasaki exchange of Section 2.3.2 as an operator, so let $\mathrm{Swap2}(x, i, j)$ denote the configuration x with the occupations of sites i and j exchanged. Each unlike-spin pair $i < j$ gives a neighbouring configuration, and the one-way parameterisation (3.6) carries over with a pair potential in place of the site-token one. The rate of the swap (i, j) from x is $[\, G^\theta_{\mathrm{swap}}(i, j \mid x)\,]_+$. We now replace the need for local equivariance (3.7) with antisymmetry under the swap itself,

$$G^\theta_{\mathrm{swap}}(i, j \mid x) = -\, G^\theta_{\mathrm{swap}}\big(i, j \mid \mathrm{Swap2}(x, i, j)\big), \tag{5.4}$$

This yields the incoming rate without another network evaluation, exactly as we required. The reverse rate the residual needs at the neighbour $y = \mathrm{Swap2}(x, i, j)$ is $[\, G^\theta_{\mathrm{swap}}(i, j \mid y)\,]_+ = [\, -G^\theta_{\mathrm{swap}}(i, j \mid x)\,]_+$ by (5.4), read off the tensor already computed at x with no forward pass at any neighbour. The two directions therefore use $[G]_+$ and $[-G]_+$ from the same score. Substituting these rates into (3.1) gives

$$\delta_t(x) = \partial_t \log \tilde{p}_t(x) - \partial_t \log Z_t + \sum_{i<j} \left(\big[\, G^\theta_{\mathrm{swap}}(i, j \mid x)\,\big]_+ - \big[\, -G^\theta_{\mathrm{swap}}(i, j \mid x)\,\big]_+ \frac{\tilde{p}_t(y_{ij})}{\tilde{p}_t(x)} \right), \tag{5.5}$$

with $y_{ij} = \mathrm{Swap2}(x, i, j)$. Inside the sum, the first term is the outgoing rate; the second is the incoming rate multiplied by the density ratio. Antisymmetry supplies the rates, while Section 5.2.3 supplies the ratios. A useful consequence of (5.4) is that same-spin pairs need no special handling, since if $x_i = x_j$ then $\mathrm{Swap2}(x, i, j) = x$, so antisymmetry forces $G^\theta_{\mathrm{swap}} = 0$ and the density ratio is 1, and the pair contributes exactly zero. The sum in (5.5) can run over all $\binom{d}{2}$ pairs with no masking.

We enforce (5.4) using the same idea as the hollow construction of Section 3.1.3, now excluding two spin values. The score of the swap (i, j) is read out as the two-site analogue of the bilinear readout (3.9): with ω_τ the learned embedding vector for spin value τ,

$$G^\theta_{\mathrm{swap}}(i, j \mid x) \;=\; \big\langle\, H_{ij}(x),\; \omega_{x_i} - \omega_{x_j} \,\big\rangle, \tag{5.6}$$

where to achieve antisymmetry, we force the pair context H_{ij} to be blind at *both* swapped sites – it may depend on the positions i and j and on every other occupation, but not on the occupation values x_i and x_j, i.e.

$$H_{ij}(x) = H_{ij}(x') \qquad \text{whenever } x_k = x'_k \text{ for all } k \notin \{i, j\}. \tag{5.7}$$

[1] We label each pair with $i < j$ so that it appears once in the residual. Including same-spin pairs gives $\binom{d}{2}$ terms; though we will later see that their contribution is zero.

The context can use the sites' positions because those positions stay fixed during a spin exchange. To see this, write $y = \mathrm{Swap2}(x, i, j)$, and notice the token difference reverses sign, $\omega_{y_i} - \omega_{y_j} = -(\omega_{x_i} - \omega_{x_j})$. Every spin the context can see is unchanged, so $H_{ij}(y) = H_{ij}(x)$. Hence

$$G^{\theta}_{\mathrm{swap}}(i, j \mid y) = \big\langle H_{ij}(y),\, \omega_{y_i} - \omega_{y_j} \big\rangle = -\big\langle H_{ij}(x),\, \omega_{x_i} - \omega_{x_j} \big\rangle = -\, G^{\theta}_{\mathrm{swap}}(i, j \mid x), \tag{5.8}$$

which is the required antisymmetry(5.4), by construction (the readout for same-spin pairs is zero since their embeddings are equal). Figure 5.2 shows this argument pictorially.

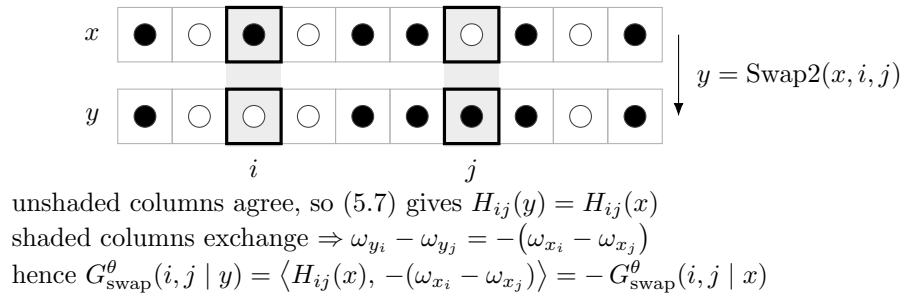


Figure 5.2. Swapping the spins at i and j leaves every other spin unchanged. The pair context excludes both shaded sites, so $H_{ij}(y) = H_{ij}(x)$. The embedding difference in (5.6) changes sign, so the swap score does too. This recovers the reverse rate without evaluating the network at y. If both spins are equal, the embedding difference and score are zero.

Everything else transfers similarly; the Stein control variate proof which was dependent on a relabelling identity is agnostic to the move set and continues to have zero estimator variance when the residual vanishes (Appendix C.4), and the path weights and ESS are computed as in Section 3.1.3, with the same limitations from finite samples and time discretisation. The training loop, rollout replay buffer and all, is the one described in Section 3.1.1 with the swap residual (5.5) in place of (3.8).

5.2.3 The swap log-ratio in closed form

Antisymmetry provides efficiency through a reduction in network evaluations, but to compute the residual we still need to evaluate the target-density ration at each swap. Building and evaluating every swapped configuration would require a $(B, \binom{d}{2}, d)$ tensor for a batch of B states, which becomes intractable as the problem scales. For the Ising target we can instead compute these ratios directly from the spins at i, j and their neighbouring spins in closed form.

If we write $h = Ax$, where A is the lattice adjacency matrix and $h_i = \sum_k A_{ik} x_k$ is the sum of the spins neighbouring site i, we're able to notice that if i and j are neighbours, each field includes the other spin. This implies that their shared bond has the same product $x_i x_j$ after the swap, so contributes no energy change, and we need not calculate it. We therefore exclude the swap partner from each field: $\tilde{h}_i = h_i - A_{ij} x_j$ and $\tilde{h}_j = h_j - A_{ij} x_i$.

With these *partner-excluded fields*, the derivation in Appendix K.4 gives the path log-ratio

$$\log \frac{\tilde{p}_t(y_{ij})}{\tilde{p}_t(x)} = t\,\sigma\,\Delta_{ij}(x), \qquad \Delta_{ij}(x) = 2(x_j - x_i)(\tilde{h}_i - \tilde{h}_j). \tag{5.9}$$

Exponentiating gives the density ratio required by (5.5),

$$\frac{\tilde{p}_t(y_{ij})}{\tilde{p}_t(x)} = \exp\big[t\sigma\Delta_{ij}(x)\big] = \exp\big[-t\beta\big(E(y_{ij}) - E(x)\big)\big]. \tag{5.10}$$

And so a swap that lowers the energy has ratio greater than 1 for $t > 0$. At $t = 0$ every ratio is 1, because the base is uniform; same-spin pairs also always give 1.

This trick is also used in Kawasaki for its ΔE, and indeed the same that `mchammer` uses per proposed swap, evaluated batched over every pair at once; one field computation $h = xA$ serves all $\binom{d}{2}$ pairs. We checked against the explicit neighbour evaluation and they agreed to within 1.3×10^{-5} or better across lattice sizes $D \in \{4, 8, 16\}$ and couplings from $\sigma = 0.1$ to 0.3, while removing the neighbour materialisation entirely (central processing unit (CPU) wall-clock $\times 5$ at $D = 4$ rising to $\times 97$ at $D = 16$). This also generalises to cluster expansions, since their energy is local in the same sense, so the two-site ΔE stays cheap on alloy energies too.

The closed-form channel. As we have just seen, partner-excluded fields depend neither on x_i nor on x_j, so swapping spins keeps $\tilde{h}_i - \tilde{h}_j$ consistent and flips only the sign of $x_j - x_i$. It therefore follows that $\Delta_{ij}(y_{ij}) = -\Delta_{ij}(x)$, which is familiar, and is the construction of (5.6) (a blind context multiplied by a spin difference). In the same way as the closed-form channels discussed in previous chapters, we may then add the known terminal log-ratio $\sigma\Delta_{ij}$ to a learned head without losing swap antisymmetry. For each stored pair $i < j$, the closed-form channel is

$$G(i, j \mid x) = G_{\text{head}}(i, j \mid x) + g(t)\,\sigma\,\Delta_{ij}(x), \qquad g(t) = g_0 + g_1 t, \tag{5.11}$$

where g_0, g_1 are zero-initialised learned scalars; nothing particularly new here. This energy difference tells us how the target favours one state over another, though doesn't determine the rates needed to get there, which still need to satisfy Kolmogorov.

5.2.4 Resolving swap conflicts

Something we have to think about with swaps that we could quietly ignore in the single-site flip case is the independence of moves. In the latter, flips are independent across sites, and so moves in the unconstrained and soft samplers can commute, i.e. applying them in any order yields the same final state. Swaps, however, can share a site and conflict. To see the issue, consider three sites with spins $(x_0, x_1, x_2) = (+1, -1, -1)$. Both $(0, 1)$ and $(0, 2)$ are valid swaps at this state, but they try to move the same up-spin. Applying both exchanges in different orders gives different results:

$$\begin{aligned} (+1, -1, -1) &\xrightarrow{(0,1)} (-1, +1, -1) \xrightarrow{(0,2)} (-1, +1, -1), \\ (+1, -1, -1) &\xrightarrow{(0,2)} (-1, -1, +1) \xrightarrow{(0,1)} (-1, -1, +1). \end{aligned} \tag{5.12}$$

After the first swap, the second exchanges equal spins and does nothing, so both proposals can't be simultaneously applied using the original spin values; the case where this isn't true is where the swaps are disjoint.

One event per step. The simplest way to avoid this conflict is to choose at most one swap per discrete time step. At state x and time t, write its rate and the sum of all outgoing rates as

$$q_{ij}(x, t) = \left[G^{\theta}_{\text{swap}}(i, j \mid x)\right]_+, \qquad \Lambda(x, t) = \sum_{i<j} q_{ij}(x, t). \tag{5.13}$$

Here Λ is the *total escape rate*: the instantaneous rate of leaving x by any swap. For example, if the only nonzero rates are $q_{01} = 2$ and $q_{02} = 1$, a step of length $\mathrm{d}t = 0.1$ assigns probabilities 0.2 and 0.1 to those swaps, leaving 0.7 for staying at x. In general, the Euler update draws once from

$$\begin{aligned} \mathbb{P}_{\text{Euler}}\big((i, j) \text{ fires} \mid x, t\big) &= q_{ij}(x, t)\,\mathrm{d}t, \\ \mathbb{P}_{\text{Euler}}\big(\text{stay} \mid x, t\big) &= 1 - \Lambda(x, t)\,\mathrm{d}t. \end{aligned} \tag{5.14}$$

These are only valid probabilities when $\Lambda\,\mathrm{d}t \leq 1$. In our implementation we clip these probabilities if they ever exceed 1 and retain a log of any occurrences to ensure our time grid isn't too coarse for the one-event method. We find that at small lattice sizes this restriction on the move-set never fires, though the same can't be said as we scale.

Several disjoint swaps per step. At larger sizes, allowing only one event can require a much finer time discretisation grid (Section 5.5.4), which makes training and inference more expensive. We instead use a *vertex-disjoint* matching update, which proposes each pair independently with probability $\min\{q_{ij}\,\mathrm{d}t, 1\}$, then retains a set in which no site occurs twice. We can then apply all swaps from this set together and preserve composition exactly since they do not conflict, but freezing rates and discarding proposals still only provides an approximation of the learned process. This approach is the approximation behind tau-leaping [93], where our matching step adds the vertex-disjoint filter. Note this is not a claim that the retained swaps themselves are independent. Again this is something we monitor throughout training as a diagnostic, to help us evaluate whether we need to increase our time grid even with this in place. We tested this on 8×8 checkpoints and saw no degradation in sampler efficacy, and used it by default on lattice sizes of 16×16 and above.

Algorithm 3 shows how these updates fit into training; Appendix D.1 gives the matching procedure and further discussion on local-error.

5.3 A Multi-Headed Approach

Almost all of this chapter so far has followed quite naturally from DNFS (Section 3.1.3), though you may be left wondering if the architecture transfers. We no longer need to emit only d flip rates, but $\binom{d}{2}$ swaps with blind contexts, and for this to scale we must be able to do so efficiently. We'll work from the most naive case to the heads we consider our "production" variants.

Blinded by brute-force. We start with a naive brute-force construction which masks both sites of every pair, $H_{ij} = \mathrm{body}\big(x \text{ with } i, j \text{ masked}\big)_j$, trivially blind, but costs $\mathcal{O}(d^2)$ backbone passes per state. While it's a good correctness check and a benchmark for other architectures, the number of backbone passes makes this option intractable for non-toy problems. For a slightly less naive method, recall that we already have an efficient architecture from DNFS (Section 3.1.3) which allows us to reduce the number of forward passes from d to 1 in the unconstrained case. A similar argument can be made in the pair case here, reducing number of forward passes from d^2 to d. Concretely, it reuses the backbone's single-site hollowness – one pass with anchor i masked returns representations $H_{ij} = \mathrm{body}\big(x \text{ with } i \text{ masked}\big)_j$ for every j at once, blind at i by the input mask and blind at j by the leTF's leave-one-out readout, so d single-masked passes returns all pairs. Since these passes are independent, we can batch them into one stacked forward to improve GPU utilisation, though it doesn't negate the need to evaluate d masked inputs. Table 5.2 compares measured gradient-update time and peak memory, excluding trajectory generation. We call this head the "mask-one" head.

One shot blindness. While d forward passes is certainly better than d^2, it simply won't be a sufficient saving once we get past small lattices, especially for efficiency maximalists. We'd therefore like to follow in the footsteps of the efficient architecture of DNFS, and design a head that allows us retrieve all $\mathcal{O}(d^2)$ blind contexts H_{ij} at once, in one pass. By (5.6), antisymmetry follows from this blindness, so if we succeed here, we're done. Something we need to keep in mind carries from the "two-hop" restriction of the unconstrained case (Figure 3.1). Shared features may only be deep if they exclude the two holes, else they won't adhere to the blindness requirement with this architecture, and is something that drives the head design that follows.

5.3.1 Blindness by order: the causal-stream heads

We begin by borrowing the backbone of the leTF from DNFS and seek to achieve blindness from ordering alone. We call these the *causal-stream heads*, after the prefix and suffix streams they

inherit from their ancestor, though due to the second hole, they produce an interesting third object we need to consider.

Three intervals. The reason a single pass is possible at all is the geometry of two holes (Figure 5.3). Removing sites i and j splits the flattened sequence into three intervals,

$$\underbrace{x_0 \cdots x_{i-1}}_{\text{prefix}} \; \boxed{x_i} \; \underbrace{x_{i+1} \cdots x_{j-1}}_{\text{band}} \; \boxed{x_j} \; \underbrace{x_{j+1} \cdots x_{d-1}}_{\text{suffix}}, \tag{5.15}$$

with two of the three summaries already present in the backbone; recall that the causal stacks of the leTF read one slot short of each hole,

$$P_i = P_i\big(t,\, x_{<i}\big), \qquad S_j = S_j\big(t,\, x_{>j}\big), \tag{5.16}$$

the forward stack's state at slot i (which, with the time token prepended, has seen exactly $\{t, x_0, \ldots, x_{i-1}\}$) and the backward stack's state at slot j after reversal (having seen exactly $\{t, x_{j+1}, \ldots, x_{d-1}\}$). Both are deep and fully mixed *within* their interval yet blind to everything at and beyond their hole, because causal masking composes across layers. We're then left with the third and new object, the *interior band*. It's an interval summary open at both ends, something we can't obtain from a causal sweep, and of which there are $\mathcal{O}(d^2)$, one per pair. Neither DNFS [29] nor the locally equivariant family of LEAPS [30] has a two-site analogue of its leave-one-out hollowness, and so the band offers an pair-based extension to both.

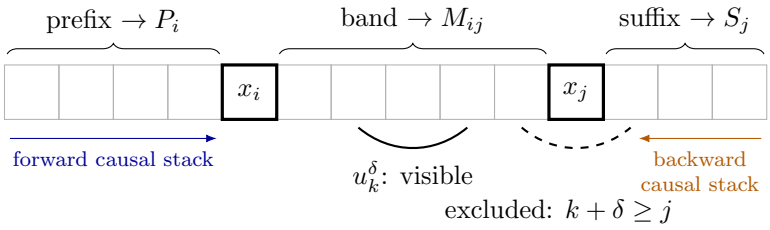


Figure 5.3. The three-interval decomposition (5.15) behind the one-pass heads, Figure 3.2 with a second hole. Removing the pair (i, j) splits the sequence into prefix, band and suffix. The prefix and suffix summaries P_i, S_j are the causal-stack states one slot short of each hole (5.16): deep, and blind by causality. The band M_{ij} is pooled from the local features (5.17) within the band itself (5.18): a bond feature u_k^δ touches sites k and $k + \delta$, so a bond feature is included only when both endpoints lie strictly between i and j.

Constructing the band features. We choose to construct local and shallow band features to satisfy the blindness requirement. With e_k the token embedding of x_k, each site contributes a *unary* feature (a learned function of one site) and each lattice-relevant offset $\delta \in \{1, D\}$ – row and column adjacency of the flattened $D \times D$ lattice (the bond families the Ising energy is built from) – contributes a *bond* feature (a learned function of the two sites together),

$$v_k = \phi_0(e_k), \qquad u_k^\delta = \phi_\delta\big(e_k, e_{k+\delta}\big), \tag{5.17}$$

with ϕ_0 and ϕ_δ small MLPs. The nonlinearity is necessary in ϕ_δ and decoration in ϕ_0 – the former allows the feature capture how the two occupations interact, including the spin product $x_k x_{k+\delta}$ in the Ising bond energy. For the latter we could just as easily store a mapping in an embedding table; here we use an MLP for implementation consistency.

For pair $i < j$, we retain site features inside the band and bond features whose two endpoints are both inside it:

$$\mathcal{V}_{ij}^0 = \{\, k : i < k < j \,\}, \qquad \mathcal{V}_{ij}^\delta = \{\, k : i < k \text{ and } k + \delta < j \,\}, \tag{5.18}$$

These sets exclude any feature that uses either swap value, so the retained band features are doubly blind (Figure 5.3). In the cluster-expansion terminology of (2.9), these are learned features on point and pair clusters [9].

Our decision to keep the band context local to the interior is not strictly necessary, and indeed you can include sites outside the band as long as the holes are left alone. We leave the exterior along for a couple of reasons. First, the exterior is already read deeply by the causal streams P_i and S_j, and adding a further shallow copy adds nothing. Second is that well, we tried it, and it proved our first point (Appendix J). Not only did it not help our sampler, it cost $\mathcal{O}(d)$ more terms per pair!

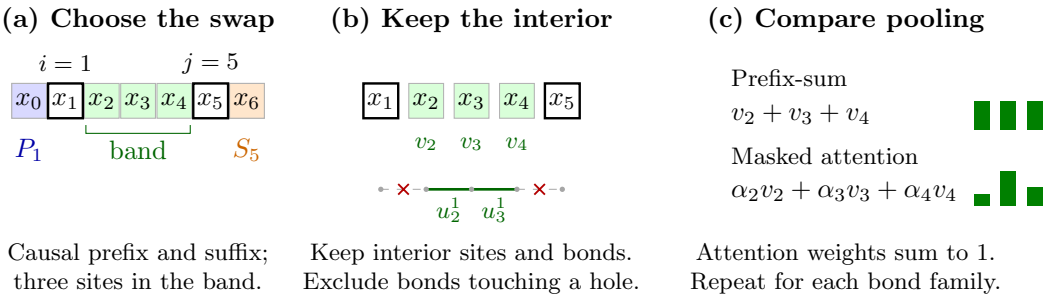


Figure 5.4. A worked example for the causal heads on a seven-site sequence, with swap sites $(i, j) = (1, 5)$ and bond offset $\delta = 1$. Both heads use the same prefix, suffix and permitted band features; they differ in how they pool the band. The weight bars are schematic. Each feature family is pooled separately, and the resulting band summary is combined with P_1 and S_5.

Weighting band contexts. We then propose two approaches to pooling the features, one naive and one learned. The former is the *prefix-sum* head, which adds the permitted features within each family. In Figure 5.4, $M^0_{1,5} = v_2 + v_3 + v_4$ and $M^1_{1,5} = u^1_2 + u^1_3$; these vectors are then concatenated. The name itself comes from how we compute the sums efficiently; with running totals $A_m = \sum_{k=0}^{m-1} v_k$, the site sum is

$$\begin{aligned} A_5 - A_2 &= (v_0 + v_1 + v_2 + v_3 + v_4) - (v_0 + v_1) \\ &= v_2 + v_3 + v_4. \end{aligned}$$

Subtracting the running total before the band from the total at its end removes the earlier sites, leaving exactly the permitted interior features. Once the running totals are available, this takes only one vector subtraction regardless of the band's length. The same subtraction works for each bond family.

The *masked-attention* head instead pools by masked softmax attention [94]. Each pair forms its query from the two position embeddings ρ_i, ρ_j alone – a query is shared machinery, so it must be blind for every pair, and positions always are – keys carry each term's content and position, and exclusion is applied *before* the softmax:

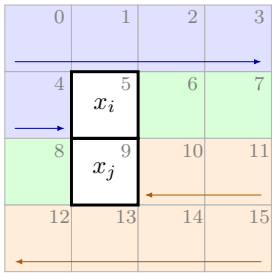
$$\alpha_{ij,k} = \operatorname*{softmax}_k \left(\tfrac{1}{\sqrt{A}} \left\langle W_q(\rho_i, \rho_j),\ W_k(v_k, \rho_k) \right\rangle + m_{ij,k} \right), \qquad M_{ij} = \textstyle\sum_k \alpha_{ij,k}\, v_k, \tag{5.19}$$

with A the query–key width and the mask $m_{ij,k} = 0$ for $k \in \mathcal{V}_{ij}$ and a finite (-10^9) floor to drive the post-softmax weight to 0. This initially feels like an improvement over the fixed weight approach above, with learned pooling weights seeing both content and position, and allows the band to attend to the interior occupations adjacent to its holes rather than smearing them into a total. It isn't free however, and costs $\mathcal{O}(d)$ per pair.

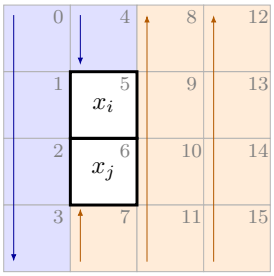
Figure 5.4 shows these constructions pictorially. We're swapping site 1 and 5, so the interior of the band contains $2, 3, 4$. We keep their unary site features v_2, v_3, v_4, and calculate permissible

bond features such that both endpoints remain inside the band. For the row-neighbour bonds ($\delta = 1$), this means that we're left with u_2^1 and u_3^1. None of the retained features nor the causal summaries P_1 and S_5 depend on the swapped sites, and so blindness is achieved. The only differences between the two approaches is the way we weight, as we just discussed.

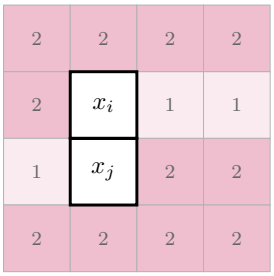
A richer exterior context. Did I really say one pass? While that might suffice, there may be no harm adding another. If you stare at Figure 5.3 for long enough, you might be left wondering – if we require blindness, computed by locality (i.e. within the band), how do we get any expressivity? Especially as the holes can be arbitrarily far apart, and the causal streams borrowed from DNFS only give us deep mixing of the exterior. Something I didn't focus too much on until slightly later in the project was the lattice geometry itself. We somewhat arbitrarily flatten it using row-major ordering, but we can just as equally sweep column-major too. Seen in Figure 5.5, this gives us a second pair of causal streams, deep and mixed, without having to look outside of the band directly. This is of course a best-case visualisation, the holes need not line up in the second ordering. It costs one extra backbone pass (though not quite twice the training and inference time), and no new parameters beyond a wider readout. So where these are concerned, let's call them the one(ish) pass heads.



(a) row-major sweep prefix → band ← suffix



(b) column-major sweep the same pair is adjacent: no band



(c) sweeps that read each site deeply one ordering leaves three sites band-only

Figure 5.5. The pair (i, j) are column neighbours, adjacent on the torus. (a) Under the row-major raster they sit $D = 4$ apart, so the context splits into a deep prefix (blue), a shallow band of three unrelated sites (green) and a deep suffix (orange); arrows show the direction each causal stack reads, the forward stack (blue) up to the first hole and the backward stack (orange) back to the second, with the band pooled rather than swept. (b) Under a column-major sweep the same pair is adjacent: the band is empty and every other site falls into a causal stream, deep and blind by construction. (c) Counting, for each site, how many of the two sweeps read it through a causal stream: with one ordering three sites are reachable only through the depth-one band; with two, none is. The second ordering adds depth over the band: prefix, band and suffix already cover every site except the holes.

Assembling the pair. The final piece of the causal-stream puzzle is then to assemble all of our features:

$$H_{ij} = \mathrm{LN}\Big(\mathrm{MLP}\big(P_i,\ S_j,\ M_{ij},\ \rho_i,\ \rho_j\big)\Big) + \tau_t, \qquad H_{ji} = H_{ij}, \tag{5.20}$$

with LN a layer norm and τ_t the backbone's time embedding. The position embeddings ρ_i and ρ_j do not change when the two occupations are exchanged, so they preserve blindness (5.7), and the label symmetry $H_{ji} = H_{ij}$ allows us to compute context for only $i < j$; only the token difference in (5.6) changes sign when the indices are reversed, which gives $G_{ji} = -G_{ij}$ in the readout. So, P_i and S_j exclude both swap sites by causality, and M_{ij} excludes the pair by pooling mechanism, i.e. our nonlinear MLP and layer norm combine the blind ingredients and receive no information about either swap value, so retain blindness trivially. Exchanging the spins leaves H_{ij} unchanged and reverses the token difference, which gives us (5.4), as required. So indeed we are able to retrieve all swap rates in one forward pass, though there is still the cost of computing the $\mathcal{O}(d)$ band terms, which we will explore more in Section 5.3.3.

We built, tested, and retired many members of this family along the way, many of which can be found in Appendix J. These include a bilinear interval exterior, global interior features, rotary position codes [95] and rollout resampling. The choice to exclude the extensive nulls and failures is driven by a desire to keep this already detailed chapter within an acceptable threshold.

5.3.2 A convolutional approach: the two-hole patch head

If you're think we had to jump through one too many hoops to achieve double blindness in the transformer, I'd agree. Blindness had to come from the sequence ordering, and it wasn't until I drew lattices before they were flattened into sequences that I started understand the problem. When you flatten a lattice, sites that used to be neighbours become far apart, and you lose the geometric prior you can leverage in your architecture. The problem started to feel more vision-based, and so I revisited the convolutional approach used briefly in Chapter 3 and borrowed from LEAPS [30]. Here blindness is not achieved from ordering, but using lattice geometry, with the same readout (5.6).

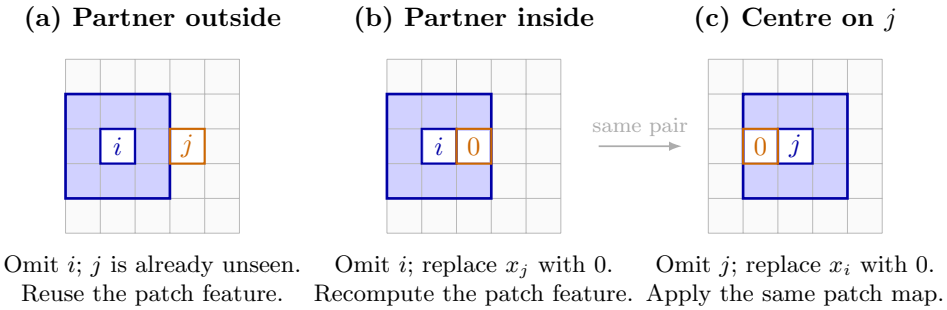


Figure 5.6. The patch head, illustrated with a 3×3 window ($R = 1$). Blue cells supply the local input; the centre is omitted. A partner outside the window needs no correction (a); inside, its value is replaced by zero before the patch map is applied (b). Centring the same window on j gives the other view of this pair (c). Both features are therefore independent of x_i and x_j. The head also adds pooled context with both sites removed, then combines the two summaries for the final readout.

I personally find this one easier to explain pictorially, as seen in Figure 5.6, though the mathematical theory underpinning it is below. The one-liner – hollow out a patch around each site, blank the swap partner too, and read the pair off the two patches.

Hollow patches. Around each site i, take a square patch of radius R and omit its centre, inspired by the zero-centre kernel of LEAPS [30]. Let $\mathcal{N}_R$ contain the remaining $(2R+1)^2 - 1$ offsets. The same small network φ maps each patch and the time t to a feature vector. For a swap pair (i, j), we also remove the partner's value if it lies inside the patch:

$$f_i = \varphi\big((x_{i+\delta})_{\delta\in\mathcal{N}_R},\, t\big), \qquad f_i^{(-j)} = \begin{cases} \varphi\big((x_{i+\delta})_{\delta\in\mathcal{N}_R}\,\big|_{\,x_j:=0},\, t\big) & \|j-i\|_\infty \le R, \\ f_i & \text{otherwise}, \end{cases} \tag{5.21}$$

Here φ is an MLP with one hidden layer, and distances and offsets are periodic across the lattice boundaries. If j is outside the patch, f_i already ignores both swap sites and can be reused. Otherwise, we mask x_j as 0 (distinct from either spin ± 1) and recompute the feature. Repeating this construction around j gives $f_j^{(-i)}$. The resultant features are independent of both x_i and x_j.

Pooled far field. To include information beyond the patch, we also pool over boxes of increasing size around each site. Each spin is embedded separately as ψ_k, then mapped by a learned linear projection W_l for that scale. We exclude both swap sites from each sum and add the pooled

features to the patch feature:

$$c^{l}_{i,(ij)} = \frac{1}{n_l} \sum_{k \in B_l(i)\setminus\{i,j\}} W_l\, \psi_k, \qquad C_i^{(-j)} = f_i^{(-j)} + \sum_l c^{l}_{i,(ij)}, \tag{5.22}$$

The box $B_l(i)$ has radius $r_l = 1, 2, 4, \ldots$, while $2r_l + 1$ fits within the lattice side; the final pool covers the whole lattice, and the denominator $n_l = |B_l(i)|$ is the original box size before excluding the swap sites. These sums can be computed once per site, then corrected for each pair by subtracting the excluded contributions, which works because each summand depends only on its own site's spin. The resulting summaries $C_i^{(-j)}$ and $C_j^{(-i)}$ therefore remain blind to both swap values.

Pair-side assembly. The final step is to combine the two summaries, the relative position of the sites, and the time:

$$z_{ij} = W_1\, C_i^{(-j)} + W_2\, C_j^{(-i)} + e(j-i) + \tau_t, \qquad H_{ij} = \rho\big(\mathrm{LN}(z_{ij})\big) - \rho\big(\mathrm{LN}(z_{ji})\big), \tag{5.23}$$

Where W_1 and W_2 are separately learned linear maps, e embeds the signed periodic displacement, τ_t encodes time, and ρ is a small MLP. Identical blindness arguments as posed in the transformer case apply here – layer normalisation and ρ act on inputs that already exclude both spin values, so neither can break it. Swapping x_i and x_j therefore leaves H_{ij} unchanged, and in the final readout (5.6), the token difference changes sign, giving the required reversal of the swap score. We subtract the two orderings in this way to give $H_{ji} = -H_{ij}$; the token difference flips sign similarly with this relabelling, so the score doesn't depend on which site is named first. In practice, we store it for $i < j$ and negate the flip-side of the diagonal.

In summary, the patch head is able to combine detailed *local* patches, alongside less-detailed information from larger regions. Its final network is able to learn nonlinear functions of these pooled summaries, and the result is a translation-equivariant network.

5.3.3 Engineering optimisations

Something we have to contend with in this swap-based approach is the pair-score assembly, which scales as $\mathcal{O}(d^2)$. This makes frequent and larger evaluations expensive; to combat this later runs reduced in-training evaluations from 5000 draws to 256 or 512 with `bfloat16` autocast, with final evaluations retaining the full sample count and `fp32` precision. We also experimented with `bfloat16` loss updates at 20×20 and above, inspired by mixed-precision training [96], with similar ESS across the tested three-seed comparisons at both couplings.

The use of fused attention and appropriately chunked evaluations helped reduce peak memory throughout training, as well as the use of micro-batched gradient accumulation for larger lattice sizes to avoid unnecessary out of memorys (OOMs). The latter also allowed us to experiment with using larger batches in training, inspired by the work of McCandlish et al. [97].

A nice guide by Tam [98] prompted the use of `torch.compile()` which drastically reduced training run times (and arguably should've been turned on from the start). On an A100-80GB at 16×16, compiling the $R = 1$ two-hole patch head gave a 2.2× faster gradient update (batch 128) and a 4.9× faster rollout (batch 512, 128 Euler steps), while satisfying numerical and gradient agreement with identical eager executions. This was used in later experiments for Chapter 4 too.

It's worth noting that the mask-one head's d passes need not imply a d-fold increase in wall-clock since batching changes GPU utilisation. We observed that mask-one costs about 10× the one-sweep attention update at 8×8, not the 64× we might otherwise predict (Table 5.2). Tables 5.1 and 5.2 summarise the family of heads as well as their measured update costs.

Table 5.1. Swap-head constructions and backbone passes per state. All swap heads use the readout (5.6) with context excluding both spin values; a second ordering doubles the band heads' passes. The GFlowNet comparator uses count-masked sequential generation.

head / comparator	construction	passes / steps
doubly-masked oracle	both holes masked at the input	$\binom{d}{2}$ passes
mask-one reference	one hole masked, the other excluded by the leTF readout	d passes
prefix-sum band	the raster ordering; interval sums over the band	1 or 2 passes
masked-attention band	the raster ordering; masked softmax over the band	1 or 2 passes
two-hole patch	lattice geometry: hollow patch, pooled far field	1 pass
GFlowNet policy	count-masked action space	d sequential steps

	8×8		16×16		20×20	
head	time (ms)	peak (GB)	time (ms)	peak (GB)	time (ms)	peak (GB)
mask-one (reference)	144	8.23	OOM†		—	
masked-attention, one sweep	14	1.19	160	17.71	449	42.91
two sweeps	17.7	1.41	190	20.18	516	48.65
prefix-sum, one sweep	11	0.95	117	13.92	257	33.63
two sweeps	14.7	1.17	148	16.38	324	39.37
two-hole patch, $R = 1$	6.03	0.43	61.5	5.80	92.3	13.98
$R = 2$	6.93	0.55	65	6.24	97.8	14.57
$R = 3$	8.49	0.72	71.3	7.07	107	15.87

Table 5.2. Time and peak GPU memory for one gradient update (loss, backpropagation and AdamW), excluding trajectory generation: batch 128, compiled heads, fp32, A100-80GB. Times are medians of 20 updates after five warm-ups, in a fresh process per configuration; memory includes model, batch and optimiser state. Masked-attention uses the separable contraction (Appendix K.3). †Mask-one exhausts memory at 16×16; "—" means not measured.

5.4 Experimental Setup

Runs use three seeds (42–44), with deviations from Section 3.2 in Tables L.1 and L.4. Three instead of four to allow for more experimentation across heads. Evaluations use 512 fp32 draws at 4×4, and 5000 above it. The main results tables report raw parameters through 8×8 and EMA at larger sizes. To keep the tables legible, we report results at σ_c only in the main body, with $\sigma = 0.1$ in Appendix K.1. Too many heads spoiled the table, or something like that. In many of the cases the subcritical results didn't allow us to disambiguate between the heads apart from on cost; where they do, we mention and point to the appendix. As previously mentioned, simulation uses one event per Euler step through 8×8 and vertex-disjoint matching above it (Section 5.2.4). Averaging and diagnostic settings are in Appendix L.4.1.

References are exact enumeration at 4×4, `mchammer` non-local Kawasaki at 8×8, and our own `numba`-compiled Kawasaki engine above it, both of which serve as our classical baselines. ESS, importance-reweighted observable errors and sampling cost follow Table 3.2. Parentheses give approximate whole-chain reference uncertainty; separate floors benchmark independent draws at the neural sample count (Appendix C.3). We add boldface to the "best" column entry for each of our swap heads, though this is mostly decoration and doesn't necessarily imply a winner.

We also use GFlowNets as a neural comparator, with parameter count roughly matching that of the attention head. We compare trajectory balance (TB) with a learned normaliser [69] and forward-looking detailed balance (DB) with exact prefix energies [80]; from 16×16 upward, TB excludes the normaliser from weight decay; forward-looking DB retains all three failed

seeds per coupling at 16×16 and was not run above it. Sampling cost includes rollout and weight evaluation, excluding training, using exact separable attention and key–value (KV)-cached GFlowNet evaluation. Training cost is priced separately for every cell in Table L.6. Chain cost includes burn-in and is divided by effective records. Appendix L.4.1 records all settings.

5.5 Experimental Results

This section takes a bit of a "survival of the fittest" approach, where many heads (even more than displayed in tables) were tried at 4×4, not all made it to 8×8, and so on. The first filter was of course fidelity, but as the lattices started to grow the computational cost became too much of a burden to carry each head forward. Figure 5.7 gives a *very* high-level summary of the results. As in Chapter 4, we first start by focussing on a single composition, though instead of then training additional specialists we move straight to amortisation. This is motivated by the fact that observed degradation away from centre was attributed to the base distribution, where here we match composition from the start by design.

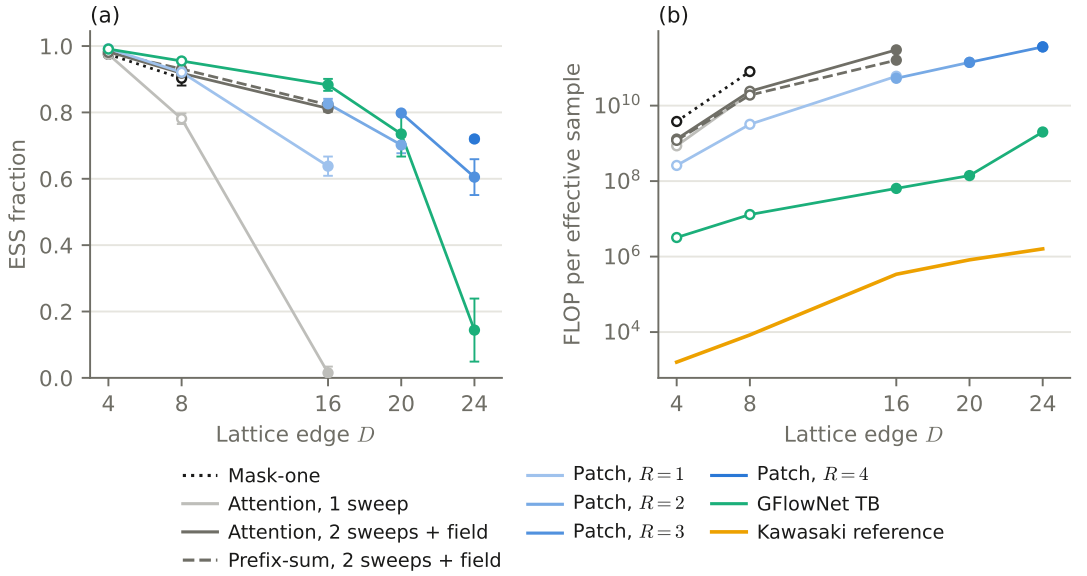


Figure 5.7. Scaling at the exact critical coupling: (a) ESS with error bars showing between-seed standard deviation; (b) sampling FLOPs per effective sample, (training cost in Table L.6). Selected swap heads and the GFlowNet trajectory-balance comparator are shown, with the certified Kawasaki chain as the classical baseline. In the legend, "field" means the exact-field channel. Open markers use raw parameters at 4×4 and 8×8; filled markers use EMA parameters at 16×16 and above. The collapsed one-sweep attention cell retains its ESS point but omits its cost. Values and seed conventions are those of Tables 5.3, 5.4 and 5.5.

5.5.1 Enumerable check

We start one last time by confirming our methodology is correct against a true ground-truth distribution. Here on the 4×4 lattice at $c_{\text{target}} = 0.5$, the manifold has $\binom{16}{8} = 12{,}870$ states, compared with the $2^{16} = 65{,}536$ of the unconstrained problem.

Table 5.3 shows observable errors broadly comparable to the sampling floor at both couplings against exact enumeration. Two pass heads and exact field variants are excluded for brevity. Something we're familiar with by now but still worth highlighting is the sampling cost, where each head remains more expensive in FLOPs per same than both Kawasaki and GFlowNets. Also worth highlighting that while all samples hit the requested target composition throughout the chapter, the nonzero magnetisation error in the tables measures the erorr on the row and column profiles, not on the aggregate count.

	$\sigma = \sigma_c$				
Method	ESS↑	ΔMag↓	ΔCorr↓	EW_2 ↓	FLOP/es↓
exact enumeration (reference)	/	0	0	0	–
sampling floor at $N = 512$	/	8.1	13.0	7.0	–
Kawasaki (mchammer)	/	3.1 ± 1.1	5.4 ± 2.0	4.5 ± 1.6	1.6×10^3
doubly-masked oracle†	0.967 ± 0.011	7.7 ± 0.3	12.1 ± 4.5	7.3 ± 2.6	2.9×10^{10}
mask-one head	0.975 ± 0.006	8.8 ± 2.0	11.5 ± 3.4	8.1 ± 2.9	3.8×10^9
masked-attention band, one sweep	0.975 ± 0.003	$\mathbf{6.0 \pm 2.3}$	12.9 ± 2.1	$\mathbf{5.7 \pm 2.2}$	8.6×10^8
prefix-sum band, one sweep	0.892 ± 0.118	10.0 ± 1.7	11.8 ± 3.1	7.9 ± 2.5	8.8×10^8
two-hole patch head	$\mathbf{0.992 \pm 0.006}$	6.3 ± 1.3	17.3 ± 2.8	8.9 ± 1.0	$\mathbf{2.6 \times 10^8}$
GFlowNet, trajectory balance	0.991 ± 0.002	7.4 ± 1.9	14.3 ± 4.1	8.1 ± 1.0	3.2×10^6
GFlowNet, forward-looking DB	0.972 ± 0.005	8.5 ± 2.5	13.1 ± 3.2	6.3 ± 1.5	3.2×10^6

Table 5.3. Sampler evaluation on 4×4 Ising at fixed composition $c_{\text{target}} = 0.5$. Neural rows use raw checkpoints and 512 draws per seed, reported as mean ± SD over three seeds. The reference is exact enumeration; the floor row gives the expected error at $N = 512$. Error columns are in units of 10^{-2}. Columns and cost conventions follow Section 5.4; "/" means not applicable and "–" unavailable. †The doubly-masked oracle is evaluated eagerly. Critical coupling only, two sweep and exact field variants excluded for brevity; full results in Table K.2.

We also check the unweighted configuration frequencies of the two-hole patch head in Figure I.1 in Appendix I. Its empirical TV lies within the exact-draw reference range at $\sigma = 0.1$, and modestly above it at σ_c (3.02–3.35% against 2.58–2.69%).

5.5.2 The 64 site lattice

In Table 5.4 we start to see the effect of adding a second pass to our causal-sweep heads at 8×8 ($d = 64$), where two-sweep masked attention is able to recover the ESS of the mask-one head at a fraction of the cost. The addition of the exact-field to the two-pass variants is marginal, but helpful in the right direction.

We also see the star of the neural-swap-head show starting to emerge at this size, with the two-hole patch head achieving comparable ESS at the lowest sampling cost among the family – about 25× fewer FLOPs per effective sample than mask-one at the critical coupling (Table 5.4), the cost side of which we expected from Table 5.2, though now we have the sampling efficacy too.

The 8×8 column of Figure 5.8 shows close agreement with the Kawasaki reference in the patch head's energy and half-magnetisation marginals, with discrepancies near the sampling floors. Example configurations appear in Figure K.1 in Appendix K.

5.5.3 Scaling the sampler, from 16×16 to 24×24

Onto one of the main goals of the thesis (Appendix A), to show that our composition preserving sampler can scale to larger lattices.

On the 16×16 lattice we start to see cracks forming, with the one-pass heads degenerating (training curves in Figure H.2), and a second pass substantially improves both heads, though the prefix-sum variant remains more sensitive to seed initialisations. The two-hole patch head again stands out among our proposed heads with comparable ESS and observable errors at a fraction of the cost. We also find that as the lattice size increases, increasing the patch radius R improves performance, perhaps naturally. There is still room for improvement on the error side though, since while the half-magnetisation and correlation of the $R = 2$ patch head are close to the sampling floor, its energy marginal starts to differ from the reference. Something else worth highlighting is the collapse of GFlowNet forward-looking DB at this size, all three seeds failing

	$\sigma = \sigma_c$				
Method	ESS↑	ΔMag↓	ΔCorr↓	EW_2 ↓	FLOP/es↓
Kawasaki (mchammer)	/	(2.7)	(2.8)	(0.5)	8.4×10^3
sampling floor at $N = 5000$	/	4.8	6.9	1.4	–
mask-one head	0.903 ± 0.022	5.1 ± 1.3	7.7 ± 0.9	1.5 ± 0.1	8.0×10^{10}
masked-attention band, one sweep	0.781 ± 0.016	6.3 ± 1.2	8.4 ± 1.3	1.6 ± 0.2	2.1×10^{10}
masked-attention band, two sweeps	0.908 ± 0.003	5.0 ± 1.6	8.8 ± 1.3	$\mathbf{1.4 \pm 0.4}$	2.4×10^{10}
masked-attention band, two sweeps + exact field	0.918 ± 0.005	4.7 ± 0.9	$\mathbf{6.3 \pm 0.0}$	1.7 ± 0.2	2.4×10^{10}
prefix-sum band, one sweep	0.735 ± 0.014	6.5 ± 1.1	8.4 ± 0.6	2.0 ± 0.1	1.7×10^{10}
prefix-sum band, two sweeps	0.889 ± 0.004	$\mathbf{3.6 \pm 1.6}$	8.1 ± 1.9	1.6 ± 0.1	2.0×10^{10}
prefix-sum band, two sweeps + exact field	$\mathbf{0.931 \pm 0.003}$	4.6 ± 0.3	6.4 ± 1.1	2.0 ± 0.1	1.9×10^{10}
two-hole patch head	0.921 ± 0.003	6.6 ± 0.9	7.4 ± 1.9	1.9 ± 0.2	$\mathbf{3.2 \times 10^9}$
GFlowNet, trajectory balance	0.955 ± 0.005	7.0 ± 1.1	7.7 ± 2.4	1.5 ± 0.2	1.3×10^7
GFlowNet, forward-looking DB	$0.767 \pm 0.024^{\dagger}$	6.7 ± 1.8	9.3 ± 0.4	1.6 ± 0.3	1.6×10^7

Table 5.4. Sampler evaluation on 8×8 Ising at fixed composition $c_{\text{target}} = 0.5$. Neural rows use raw checkpoints and 5000 draws per seed, reported as mean ± SD over three seeds. Error columns are in units of 10^{-2}. Parentheses give the Kawasaki reference's approximate uncertainty; the floor row gives the expected error at $N = 5000$. Columns and cost conventions follow Section 5.4; "/" means not applicable. †Doubling the forward-looking DB training horizon to 100k steps raises its critical raw ESS to 0.900 ± 0.019. Full results in Table K.3.

at both couplings (Table 5.5), while trajectory balance stays competitive on fidelity at lower sampling cost.

It is worth noting that this two-pass variant of the causal-stream heads actually came quite late in the development process, and many other repair strategies were attempted in the interim. Larger trajectory batches and changes to the control-variate estimator were a couple of examples. Table J.2 in Appendix J records these experiments.

Above 16×16 we opted to move forward only with the convolutional approach, mainly for compute reasons. At 20×20, both tested patch radii retain near-unit ESS and observable errors close to the sampling floors at $\sigma = 0.1$ (Table K.5), though the story is not quite the same at criticality (Table 5.5). Increasing the radius does improve the ESS, but energy and correlation errors remain above their sampling floors, so the larger patch improves sampling efficiency without resolving all distributional discrepancies. We see that the trajectory-balance GFlowNet survives and performs similarly in terms of ESS, with observable errors closer to the floor. Something that continues to be the case is its lower sampling cost. We also noticed when inspecting the training logs that its ESS was still rising at the end of the allocated training steps, which we as a training-budget artefact since we matched step-count across samplers.

The case is similar as we scale to reach our maximal lattice size at 24×24, where we only ran critical coupling in the interests of time and compute (Table 5.5). ESS fell slightly at $R = 3$, but again an increase in R managed to recover most of the drop, at a flat cost per effective sample. We were yet to increase the size of the network across our runs, and didn't here either. We suspect that this could be an interesting avenue to explore for better performance. The GFlowNet fails to continue its streak at this size, though as above we also noticed it was still improving as training terminated. We posit that with longer training it would match our implementation.

Figure 5.8 shows the four lattice sizes side-by-side and displays the above argument pictorially. The ϕ marginal tracks the reference at every size and stays near its floor, while the energy marginal drifts progressively off it as the lattice grows; we see the sampler keeping the reference's width but shifts toward higher energy, by about 0.02 per site at 20×20 and 24×24, which is the growing EW_2 column in the tables. We summarise this as the generator under-ordering slightly at criticality rather than collapsing.

	$\sigma = \sigma_c$				
Method	ESS↑	ΔMag↓	ΔCorr↓	EW_2 ↓	FLOP/es↓
16×16					
Kawasaki reference (thesis engine)	/	(5.7)	(5.1)	(0.2)	3.4×10^5
sampling floor at $N = 5000$	/	8.5	11.9	0.6	–
masked-attention band, one sweep	0.015 ± 0.019	236.2 ± 202.2	388.3 ± 271.0	24.8 ± 17.8	3.6×10^{14}
masked-attention band, two sweeps	0.759 ± 0.030	8.3 ± 1.6	27.0 ± 4.2	1.7 ± 0.1	3.2×10^{11}
+ exact field	0.812 ± 0.010	11.1 ± 2.0	$\mathbf{21.3 \pm 1.1}$	$\mathbf{1.5 \pm 0.1}$	3.0×10^{11}
prefix-sum band, one sweep	0.062 ± 0.041	48.9 ± 39.4	280.4 ± 72.2	26.9 ± 5.8	8.0×10^{12}
prefix-sum band, two sweeps†	0.732 ± 0.023	9.0 ± 2.2	24.5 ± 2.8	1.6 ± 0.1	1.9×10^{11}
+ exact field	0.823 ± 0.005	$\mathbf{7.4 \pm 2.1}$	24.4 ± 2.4	1.7 ± 0.1	1.6×10^{11}
two-hole patch head	0.638 ± 0.029	9.8 ± 2.9	23.8 ± 5.1	1.8 ± 0.1	6.0×10^{10}
$R = 2$	$\mathbf{0.826 \pm 0.015}$	9.6 ± 1.9	22.9 ± 2.2	1.8 ± 0.2	$\mathbf{5.3 \times 10^{10}}$
GFlowNet, trajectory balance	0.883 ± 0.018	9.0 ± 2.2	15.5 ± 2.7	0.7 ± 0.1	6.4×10^7
GFlowNet, forward-looking DB	0.001 ± 0.001	394.5 ± 157.6	461.1 ± 125.9	9.1 ± 3.3	6.2×10^{10}
20×20					
Kawasaki reference (thesis engine)	/	(8.7)	(7.0)	(0.2)	8.2×10^5
sampling floor at $N = 5000$	/	10.2	14.0	0.4	–
two-hole patch head, $R = 2$	0.702 ± 0.025	14.3 ± 3.6	35.2 ± 1.1	2.4 ± 0.1	$\mathbf{1.4 \times 10^{11}}$
two-hole patch head, $R = 3$	$\mathbf{0.798 \pm 0.004}$	$\mathbf{10.6 \pm 4.3}$	$\mathbf{32.3 \pm 1.8}$	$\mathbf{2.2 \pm 0.2}$	1.4×10^{11}
GFlowNet, trajectory balance‡	0.735 ± 0.068	10.5 ± 1.2	18.3 ± 1.6	0.5 ± 0.1	1.4×10^8
24×24					
Kawasaki reference (thesis engine)	/	(13.0)	(11.0)	(0.2)	1.6×10^6
sampling floor at $N = 5000$	/	12.2	16.4	0.3	–
two-hole patch head, $R = 3$	0.605 ± 0.054	17.1 ± 2.2	42.2 ± 1.1	$\mathbf{2.3 \pm 0.1}$	3.6×10^{11}
two-hole patch head, $R = 4$	$\mathbf{0.720 \pm 0.007}$	$\mathbf{15.9 \pm 2.4}$	$\mathbf{39.7 \pm 0.6}$	2.4 ± 0.1	$\mathbf{3.6 \times 10^{11}}$
GFlowNet, trajectory balance‡	0.144 ± 0.095	42.2 ± 11.1	51.3 ± 18.5	0.8 ± 0.2	2.0×10^9

Table 5.5. Sampler evaluation at the critical coupling across the production sizes, 16×16 to 24×24, at fixed composition $c_{\text{target}} = 0.5$: EMA checkpoints, 5000 draws per seed, mean ± SD over three seeds unless marked. Error columns are in units of 10^{-2}; parentheses give the Kawasaki reference's approximate uncertainty and the floor rows the expected error at $N = 5000$. The 24×24 cells are the 20×20 $R = 3$ recipe moved to the lattice (bf16 training, loss microbatch of 128, fp32 evaluation). Columns and cost conventions follow Section 5.4; "/" means not applicable and "–" unavailable. Full results for 16×16 and 20×20 in Tables K.4 and K.5. †Two-seed mean excluding one collapsed seed; the one-sweep masked-attention cell retains all three failed seeds (Appendix J). ‡Budget-matched comparator: trained for the same 100k updates as the swap heads, with training ESS still rising when the budget ended. Matched on updates, not compute: it used 25–55× fewer training FLOPs (Table L.6).

5.5.4 Visualising the trained generator

Now we have a trained swap generator (well, actually hundreds...), so the natural question follows, what does it look like? Figure 5.9 follows one draw from our uniform on-the-slice base from start to finish along the learned path. Larger regions of like spins form as the sampler exchange pairs of sites, while the composition itself stays consistent throughout. We also mark one site as an anchor to see how it evolves, and use a smaller than maximally trained lattice size to make the visualisation more straight-forward. A dynamic version of this can be found in the repo readme.

The learned swap rate (middle row) shows us how the network directs the exchanges, with each cell giving the rate for swapping that neighbour with the anchor site. Something you'll naturally notice is that many of these neighbours are far away in the lattice, a function of us not restricting to the nearest-neighbour swap variant. These are all the possible moves, and our simulator samples the ones that occur. Both the configuration and time are changing as we progress through the trajectory, with the network updating the rates as we go.

The bottom row connects us back to the closed-form log ration of Section 5.2.3, which may seem fairly abstract until now; the diagram makes this a little clearer. Positive values (blue)

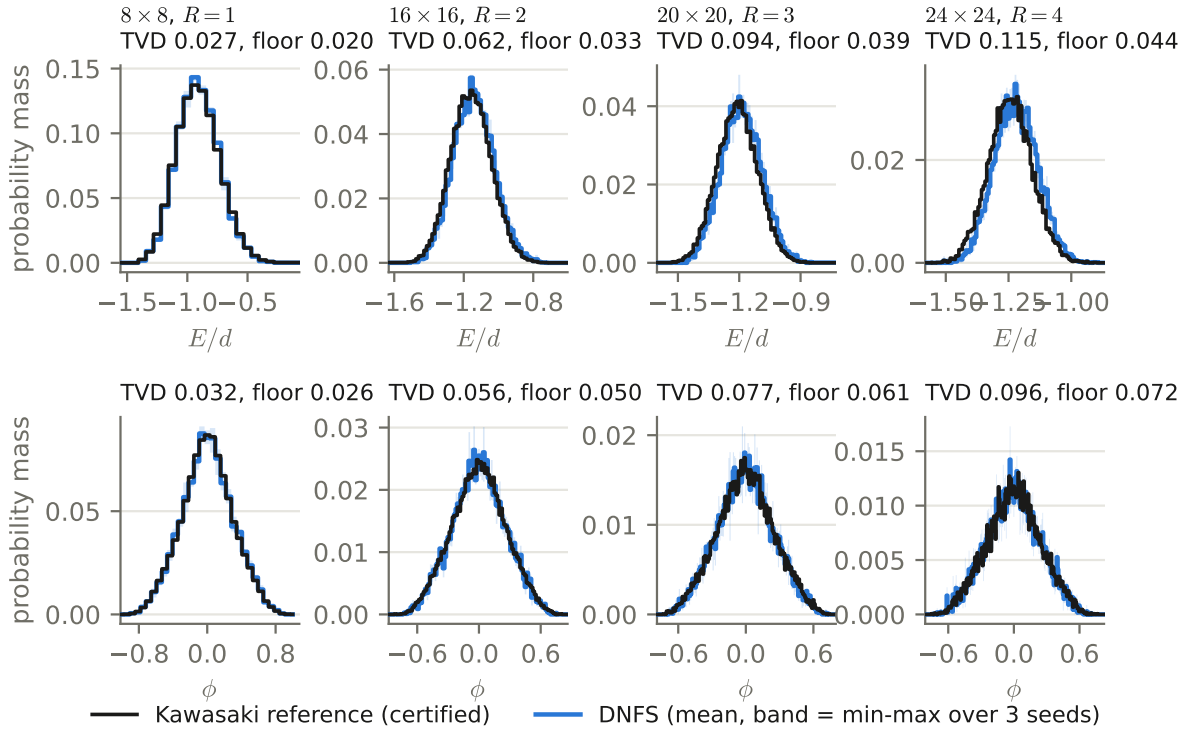


Figure 5.8. Hard results cells at σ_c and $c_{\text{target}} = 0.5$ across the four production rungs: the two-hole patch head at the radius its table prints, EMA parameters, importance-weighted mean and min–max band over three seeds, against the certified Kawasaki reference. Top: bare Ising energy per site on its exact levels; bottom: half-magnetisation $\phi = (m_{\text{left}} - m_{\text{right}})/2$ on its exact support, the Z_2-odd read that survives the constraint. Panel titles give the mean total variation against the reference beside the 5000-draw resampling floor (200 replicates); proximity to the floor does not establish equivalence.

tells us that the terminal target favours the swapped state, and the inverse is true for negatives (yellow). The two rows are related but you'll notice the learned rates are not simply rescaled energy preferences, some prominent rates in the final column correspond to states that actually increase energy. The residual trains the rates to transport the changing distribution, without restricting every move to lower the energy.

Across tested lattices, we notice total escape rate increases both with size and with coupling (Appendix K.5). We also see that above and including 16×16 lattices, the one-event diagnostic exceeds $\Lambda\, \mathrm{d}t = 1$ on some states, and so the use of the vertex-disjoint matching update which permits several disjoint swaps per step is warranted, preserving composition while avoiding the one-event restriction (Section 5.2.4).

5.6 Transfer and Amortisation

Up until this point, each trained sampler was specialised for one coupling at one composition, and the natural question again becomes, can this sampler be re-used? How much transfers zero-shot, and how well does amortisation training transfer?

5.6.1 Pathwise coupling transfer

On a fixed-composition slice, the uniform base contributes only a constant to (5.3). After normalisation, the intermediate distribution is therefore the canonical target at coupling $t\sigma_c$:

$$p_t(x \mid c) = \pi_{t\sigma_c}(x \mid c) \propto \exp[t\sigma_c x^T A x]\, \mathbf{1}[x \in \mathcal{C}_c], \qquad t^* = \frac{\sigma'}{\sigma_c}. \tag{5.24}$$

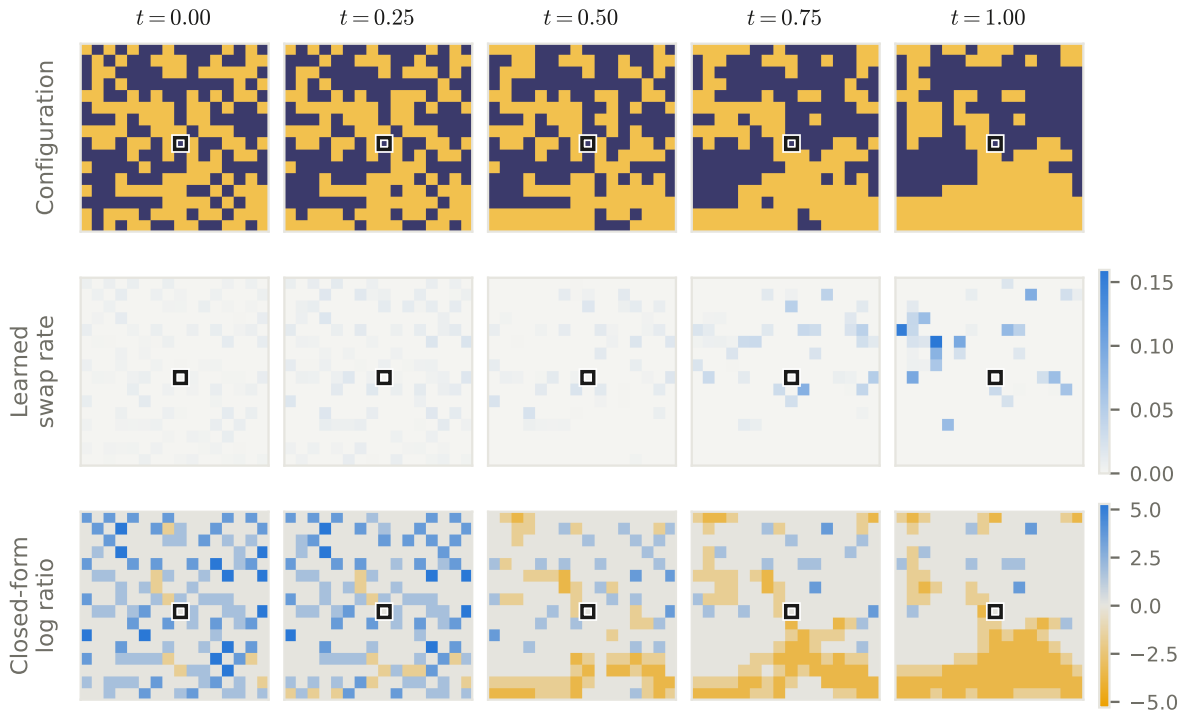


Figure 5.9. One raw 16×16 rollout of the two-hole patch head ($R = 2$, EMA parameters) at σ_c and $c_{\text{target}} = 0.5$, using 128 matching steps. Top: configurations at five sampler times, with gold denoting spin $+1$ and indigo -1. The same anchor site is boxed throughout. Middle: learned rates for swapping the anchor with each partner site; equal-spin pairs have zero rate. Bottom: the terminal log-density ratio $\sigma\Delta_{aj}(x)$ from (5.9); positive values (blue) indicate a lower-energy swapped state, negative values (gold) a higher-energy one. This is not a rate; the time-t path log ratio is $t\sigma\Delta_{aj}(x)$. Each field uses a fixed colour scale across columns.

So if we have a checkpoint trained at σ_c, we can stop it at t^* to target any coupling $0 \leq \sigma' \leq \sigma_c$ without the need to retrain, and evaluate the running weight integral to get several coupling estimates from one trajectory batch. This is the beauty of making the constraint a law of the entire trajectory; we may stop it arbitrarily, and it'll remain satisfied. The only downside to this method is that they would be correlated, since they share trajectories. This checkpointing method isn't unique to the hard case and indeed applies to the unconstrained sampler too, though as we saw in Section 4.5, not to the soft sampler.

We can see this at work in Table 5.6 where we evaluated a 16×16 sampler trained at $c_{target} = 0.5$, and can observe strong zero-shot coupling transfer. ESS improves as we stop further from the critical endpoint, reaching almost one near $\sigma' = 0.1$.

5.6.2 Amortising across compositions

Before moving to amortised training, we first wanted to see how far we could push our trained $c = 0.5$ sampler zero-shot, with results shown in Figure 5.10. This is rather mechanical in nature – simply change the base to start at your chosen composition and run inference on the sampler, seeing what you get back in effective samples. While our samples don't stray from the requested composition (indeed they're not able to), they're nevertheless not very useful – while the $8\times$ case is promising, on the larger 16×16 lattice at criticality, ESS fades as we move from the centre.

To improve upon these results we perform amortised training, looping through a handful of compositions uniformly, ensuring the base density matches in each rollout. Evaluation then works as you'd expect, scoring samples separately at each composition.

We find that the 8×8 case generalises well, beyond its trained compositions. Figure 5.10(a)

t^*	σ'	ESS↑
0.126	0.0278	1.000 ± 0.000
0.252	0.0555	0.999 ± 0.000
0.457	0.1006	0.997 ± 0.001
0.598	0.1319	0.992 ± 0.001
0.748	0.1648	0.978 ± 0.001
0.874	0.1926	0.945 ± 0.003
1.000	0.2203	0.824 ± 0.010

Table 5.6. Zero-shot coupling transfer at 16×16: a σ_c-trained two-hole patch sampler ($R = 2$, trained at $c = 0.5$) stopped at $t^* = \sigma'/\sigma_c$. ESS fractions use 5000 draws per seed and EMA checkpoints, reported as mean ± SD over three seeds. The $t^* = 1$ row is the untransferred checkpoint. Stops lie on the Euler grid; displayed times and couplings are rounded.

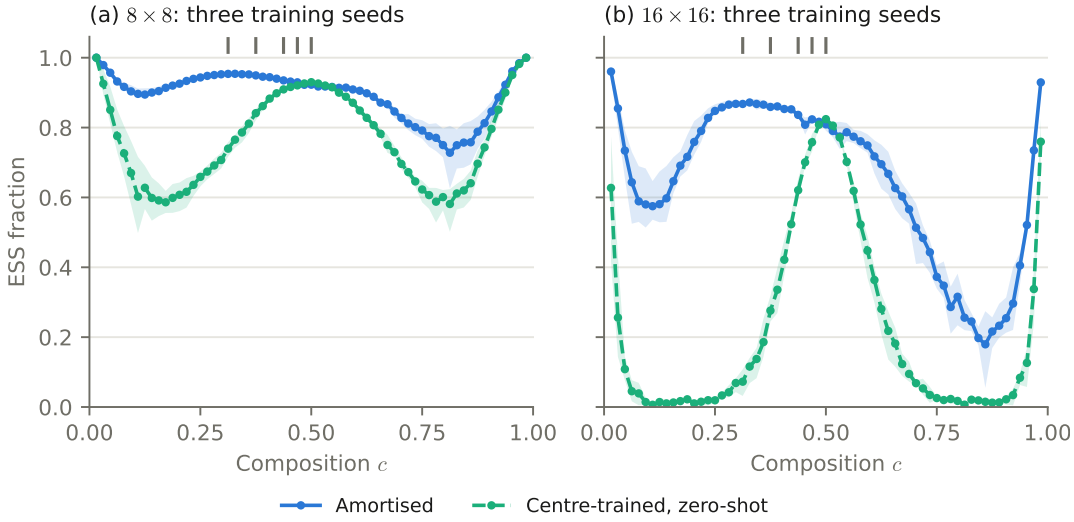


Figure 5.10. Composition transfer at σ_c: the two-hole patch head with $R = 1$ at 8×8 and $R = 2$ at 16×16, using EMA checkpoints and 5000 draws per requested slice. Blue: mixture-trained; green: centre-trained, transferred without retraining. Top ticks mark the five mixture-training compositions. (a) All 63 nontrivial slices at 8×8; (b) the same 63 compositions $c = k/64$, every fourth site, at 16×16. Lines show means and bands min–max over three training seeds for both models; the centre-trained curves use the same dense grid, of which Table K.1 prints the sparse subset. Measured compositions are points, with no values filled by spin-flip symmetry. Low-ESS specialist values have the weight-diagnostic limitations noted in Table K.1.

evaluates all 63 non-trivial composition slices, with the mean ESS staying above 0.72, across the curve, and weakest transfer and seed variance around $c_{target} = 0.75$. We could of course mirror the curve about the centre since the binary case is symmetric in $c \mapsto 1 - c$ (and perhaps you would in practice). The choice of the narrow amortised training range was primarily driven by this, as well as curiosity – would the sampler learn the symmetry somehow? Not quite.

The larger 16×16 case performs similarly within its amortised training range, sitting within the specialist seed spread at the centre, though transfer degrades further as we move away from that point. The same points about spin-flip symmetry apply here, and in practice we'd like to train the sampler across a broader mixture of compositions or use the $c \mapsto 1 - c$ mapping. With more time and compute we would've indeed done so, and the same point applies in terms of scaling our amortised training to the larger lattice sizes we trained earlier in the chapter.

Our original zero-shot probes for both couplings can be seen in Table K.1 of Appendix K. At 16×16, we notice that the specialist transfers well at the subcritical coupling, something we didn't observe on the soft sampler. One theory could be that the swap sampler is not only

learning the composition, but also the swap mechanism, though we undertook no interpretability experiments. The same certainly can't be said at criticality, as we see in Figure 5.10.

5.7 Free-Energy Curve

One of the primary reasons practitioners use the canonical-ensemble sampler is to return the free-energy curve across compositions to study phase boundaries, and a nice feature of our learned sampler, and indeed even better the amortised version of it, is that we can obtain the canonical free-energy curve from its trajectory weights. Each checkpoint supplies the whole measured curve, using a fresh batch of trajectories at each requested composition. No separate chemical-potential sweep or thermodynamic-integration ladder is needed, as you would need in the classical case.

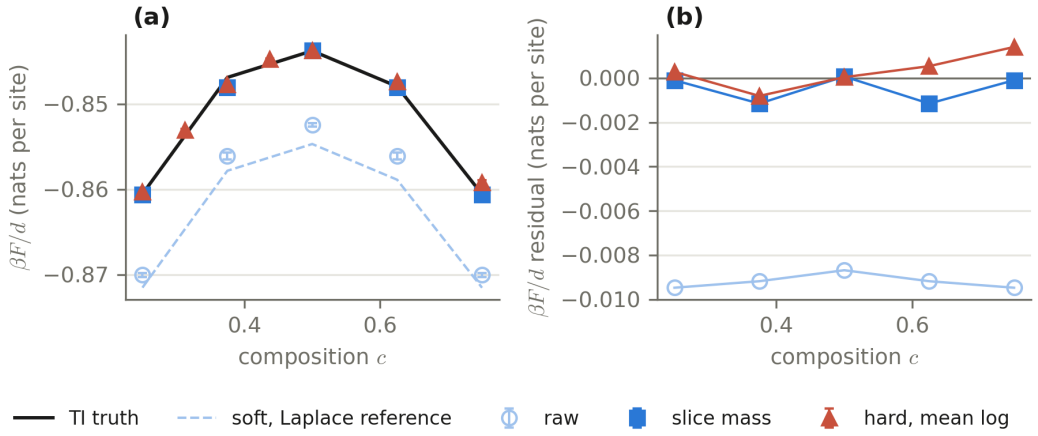


Figure 5.11. Canonical free energy in nats per site at 8×8, σ_c. (a) Thermodynamic integration (black); the soft sampler's mean-log-weight estimate (open circles) and canonical slice-mass estimate (squares); the hard sampler's mean-log-weight estimate (triangles). The dashed line approximates the soft-ensemble reference using a Laplace offset. Hard points use the amortised model of Section 5.6, three seeds and 5000 trajectories per composition per grid. Hard points use EMA checkpoints; soft points use raw checkpoints from four training seeds. Richardson extrapolation [99] uses grids of 128 and 256 points for hard, 256 and 512 for soft. Soft values above $c = 0.5$ are reflected using spin-flip symmetry. (b) Residuals against the canonical reference. Error bars combine within-batch Monte Carlo uncertainty and between-seed scatter.

On the lattice, the slice partition sum is $Z_c = \sum_{x \in \mathcal{C}_c} \exp[-\beta E(x)] = \exp[-\beta F(c)]$ [26, Sec. II.A, Eq. (4)], which is exactly the terminal normaliser estimated by our path weights: $\mathbb{E}[w] = Z_1/Z_0$ with $Z_0 = 1$ for the normalised base and $Z_1 = Z_c$ on the slice (Appendix E.2; Appendix C.1 reads the ESS off the same weights). The base's normaliser $\binom{d}{N_A}$ is known exactly and stays in the integrand (Section 5.2.1), so the weights estimate Z_c itself rather than Z_c up to a constant. We re-evaluate the same control-variate expression used for $\partial_t \log Z_t$ during training on B fresh trajectories at each composition. Integrating it along each path gives $\log w_b$, so

$$\widehat{\log Z_c} = \frac{1}{B} \sum_{b=1}^{B} \log w_b, \qquad \widehat{f}(c) = -\frac{\widehat{\log Z_c}}{d}. \tag{5.25}$$

Where $\widehat{f}$ is in nats per site. In the continuous-time formulation, $-\mathbb{E}[\log w]/d \geq \beta F(c)/d$, following the DNFS variational bound [29, Eq. (37)]. Our finite-batch estimate on an Euler grid need not retain that inequality, due to discretisation error.

Figure 5.11 demonstrates this on the 8×8 lattice, the size chosen to be able to fulfil our promises from Section 4.6.2, and overlay the results obtained by rejection on our soft sampler. We use Richardson extrapolation [99] to combat time discretisation error of the Euler approximation. We were indeed able to achieve our goal – one amortised hard-constrained sampler returns the curve to within marginal error, as does the soft-constrained specialist.

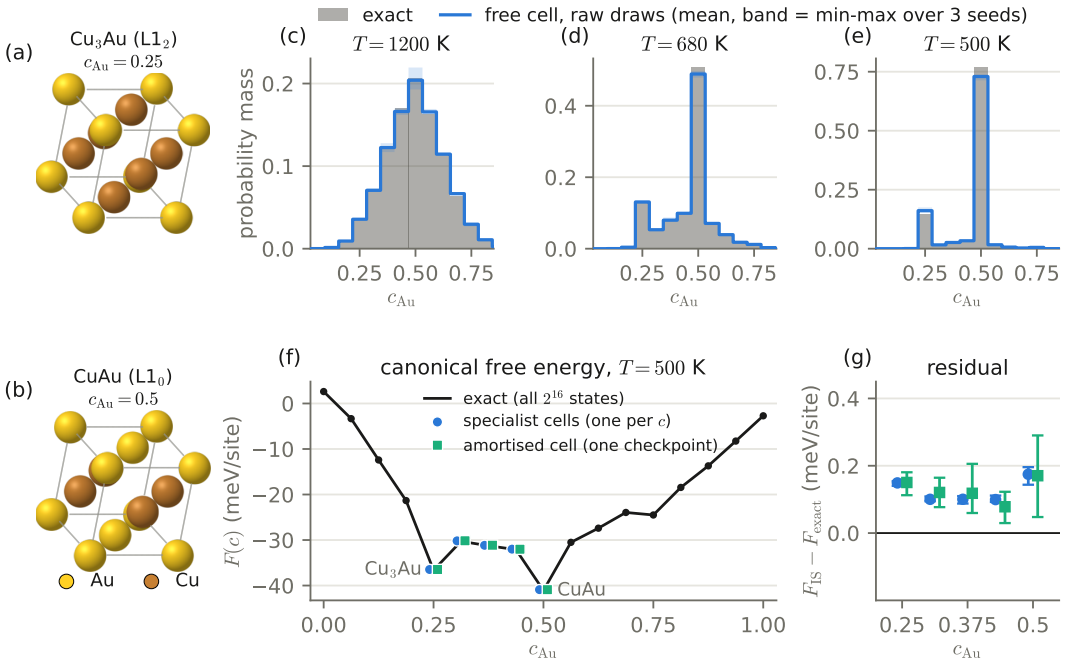


Figure 5.12. The Cu–Au cluster expansion on the 16-site cell, every panel against exact enumeration. (a, b) The two ordered phases, Cu_3Au ($L1_2$) and CuAu ($L1_0$). (c–e) Gold-fraction marginal of the free ensemble at 1200, 680 and 500 K: exact (grey) under the raw draws of the free sampler of Chapter 3 (mean over three seeds, band the min–max). (f) The canonical free energy per site at 500 K over all seventeen composition slices (black), with the importance-sampling read of the hard sampler: one specialist cell per composition (circles) and the composition-amortised cell of Section 5.6 (squares), one checkpoint whose draws are split by slice. (g) The residual of (f) against the exact value, min–max over three seeds. All sampler panels use raw final checkpoints.

5.8 From Binary Models to Binary Alloys

It's been a long time since we talked about the promised binary alloy cluster expansion of Section 2.3.3, and now's the time for us to finally tackle it. Here we borrow the Cu–Au cluster expansion released with MetaDNS [49] and set our trident to work.

Everything up to this point transfers directly, updating only the target distribution and, where used, the closed-form energy field. The uniform-slice base and geometric path both remain identical. We use the mask-one head to keep analogous architectures across our three samplers.

We use exact enumeration as our reference for Figure 5.12 and Table 5.7. The unconstrained sampler of Chapter 3 is able to track the composition marginal across the three temperatures, including the almost unimodal case at 500 K. The table also demonstrates good performance for the soft sampler relative to its penalised target, which still allows compositon to fluctuate. We also include the results from the MetaDNS paper for reference [49]. We're also able to recover the canonical free energies – to within 0.3 meV/site at the five compositions tested at 500 K.

Since we're gluttons for punishment, we also tried our hand at getting the two-hole patch head working on the 64 cite example, and while it was able to handle the face-centred cubic (FCC) geometry and achieved high ESS at 1200 K, it struggled as the alloy became more ordered (Table 5.8). At 680 and 500 K we observe that Cu_3Au draws lie close to $L1_2$ configurations despite low ESS, while CuAu draws remain far from $L1_0$ order. At 500 K they average about ten swaps from the nearest ordered variant. Various attempts were made to rescue the sampler here – a denser training-temperature ladder, doubled training budget and finer simulation grid, none of which helped. We posit that using a more ordered version of the uniform-slice base distribution may help, though this remains to be shown (Section 6.3).

Table 5.7. Sampler evaluation on the 16-site Cu–Au cell against exact enumeration. ESS fractions are raw (EMA in parentheses), mean ± SD over three seeds. ΔF is the importance-sampling free energy per site minus the exact value for each sampler's own ensemble, in meV/site, and uses raw-checkpoint weights. TV compares the raw composition marginal with the exact one. Hard specialists use one model per composition; the amortised model covers all five slices and is scored per slice. Soft rows target the penalised ensemble with $\lambda = 10$ and the flip channel. MetaDNS gives published free-ensemble ESS fractions from a single run.

free ensemble	ESS↑	ΔF	TV↓	MetaDNS
1200 K	0.952 ± 0.004 (0.971 ± 0.003)	0.0	0.018	0.90 (0.97)
680 K	0.908 ± 0.006 (0.935 ± 0.004)	0.1	0.023	0.83 (0.85)
500 K	0.838 ± 0.020 (0.880 ± 0.008)	0.2	0.039	0.85 (0.94)
fixed c, 500 K	specialist ESS↑	ΔF	amortised ESS↑	ΔF
Cu_3Au, $c = 0.25$	0.988 ± 0.007 (0.972 ± 0.003)	0.1	0.905 ± 0.032 (0.940 ± 0.003)	0.2
$c = 0.3125$	0.981 ± 0.008 (0.980 ± 0.003)	0.1	0.833 ± 0.043 (0.919 ± 0.021)	0.1
$c = 0.375$	0.873 ± 0.069 (0.960 ± 0.003)	0.1	0.758 ± 0.022 (0.846 ± 0.013)	0.1
$c = 0.4375$	0.920 ± 0.013 (0.942 ± 0.004)	0.1	0.623 ± 0.061 (0.707 ± 0.105)	0.1
CuAu, $c = 0.5$	0.875 ± 0.012 (0.886 ± 0.011)	0.2	0.590 ± 0.065 (0.655 ± 0.032)	0.2
soft, 500 K	ESS↑	ΔF	$\langle c \rangle$	
$c_{\text{target}} = 0.25$	0.967 ± 0.001 (0.965 ± 0.001)	0.2	0.255	
$c_{\text{target}} = 0.5$	0.819 ± 0.024 (0.847 ± 0.015)	0.2	0.499	

Table 5.8. Raw ESS fractions on the 64-site Cu–Au cell, min–max over three seeds. Hard rows use the two-hole patch head with two fcc neighbour shells; free rows use the flip head. MetaDNS gives published free-ensemble ESS fractions on the same cell from a single run.

T	hard, Cu_3Au	hard, CuAu	free	MetaDNS
1200 K	0.981–0.985	0.97–0.98	0.09–0.11	0.52 (0.79)
680 K	0.01–0.17	< 0.02	< 0.02	0.17 (0.40)
500 K	0.17–0.20	< 0.01	< 0.01	0.32 (0.98, CuAu only)

5.9 Chapter Summary

And so we end this chapter, and the thesis, with our trident fully equipped and prepared for battle. We were able to demonstrate that it is indeed possible to enforce our constraint as a law of the underlying move set, and pairing this with a suitable base distribution ensured that the law was never broken. Training this neural sampler was by no means an easy task, and required recycling, inspiration from adjacent papers, and at times, simply staring at a lattice. We managed to scale our sampler beyond the lattice sizes previously tackled by the unconstrained sibling in DNFS, up to those published in more recent neural sampling papers. Not only that, we were able to show some zero-shot transfer, amortisation, free energies across the entire curve, and training on a real FCC alloy. We make no claim that this is the only way to satisfy constraints, though in my (largely biased) opinion, I believe it to be the most beautiful.

6 Discussion and Conclusion

6.1 A Trident of Neural Samplers

Our journey began in Chapter 3 with the unconstrained case, with a replication of DNFS and a comparison to our first ensemble, the semi-grand at zero chemical potential. We took a swing at fixed composition sampling using this sampler as our basis, but felt a little dizzy when we saw the results. We then moved the target in the soft-constrained case in Chapter 4, motivated both as a neural equivalent to the VCSGC ensemble, and to get a little closer to the hard constrained target we so dreamed about. In the case of the former, it worked well, and was able to reproduce the observables as seen in the specialised tools. Not only that, we were able to use their weights as a route to the free-energy curve. Amortisation across compositions worked well subcritically, though criticality proved a challenge of its own. And, while we were able to get close to π_{hard}, it wasn't quite close enough.

Our final and main contribution in the thesis came in Chapter 5, with a learned sampler that built the requirement into its dynamics. By starting on the requested composition and only permitting swaps along the trajectory, the composition stays trivially satisfied, and our work then transferred to learning and evaluating the required rates. We toiled away, experimenting with an array of different heads, to take our required network forwards from d^2 to 1, a result worth celebrating in itself. On the transformer side, we ended up needing somewhere nearer to 2, and it was the late arrival of a convolutional approach of Section 5.3.2 that really closed things out. The beauty of the constrained sampler is the fact we can recover the canonical free energy directly and use it for materials side applications, particularly useful when the sampler amortises, as we were able to show.

The trident united forces in the final section of this thesis, Section 5.8, on a real energy of the Cu–Au cluster expansion. Our samplers were able to reproduce the relevant observables of their respective ensembles, with the hard again able to recover the free energies at their tested compositions. No less, the sampler also amortises. We were left wanting as we attempted to scale to the larger supercell, with the learned sampler struggling in the ordered phase. Not unique to our sampler and indeed comparable to performance seen in the published results of Du et al. [49]; it nonetheless remains a keen area of interest.

6.2 Scope and Limitations

Scope. Despite the title, no materials were discovered (or harmed) as part of this research project, and it has remained purely theoretical and computational, mainly focussed on the probabilistic inference and sampling side. While we hope our work can be used as a warm-start for practitioners in the field, this is yet to take place. Materials is simply one of a multitude of potential applications, and allowed for rich and interesting grounding throughout. Our work focussed mainly on compositional constraints on the Ising model, which can be seen as a generalisation of a binary alloy, with a small binary alloy example to wrap things up. Natural extensions to other conserved count constraints are argued rather than demonstrated here. We also only demonstrate our *Manifold Preservation Principle* on one sampler in the discrete neural family, with extensions to others reserved for future work (Section 6.3).

Hyperparameter soup. Anyone familiar with the ups and downs of training neural networks will understand the feeling of continually trying to tune toward optimal hyperparameters and seemingly getting nowhere. Changing the optimiser, increasing batch size, amending learning rate,

capacity, estimator of the log normaliser derivative, all to no avail. While I tried to make informed decisions about choices of hyperparameters, either from the literature or by running many small ablation experiments, I make no claim that these are the best choices. The search space is combinatorially large, and was a particular challenge when it came to the hard constrained case in Chapter 5, where I also had several heads to choose from (my fault, I suppose). Running small ablations is also not always fair, since sometimes even the perfect choice takes a while to bed in – Wu et al. [100] showed that short-horizon optimisations systematically favour the settings that look best early. I do believe my intuition got better as the project progressed, though I'm sure someone with fresh eyes and a different view would be able to make progress above and beyond the results found here. A random grid-search would also be a useful experiment to run, left here to someone with greater computational wallet than myself.

Computational cost. The rather bloated elephant in the room. It's something we've addressed many times throughout; the cost of both training and sampling in the neural case can prove slightly too far to the right tail for some to accept, especially the pair-based swap heads of Chapter 5. Even taking into account various architectural and engineering optimisations, the classical Kawasaki and neural reference of GFlowNets proved less expensive in FLOPs per effective sample, and the latter has mostly comparable performance should a neural alternative be necessary. Even at criticality, running the classical methods to burn-in can be done on my laptop CPU. Sure, it's a good CPU, but a CPU nonetheless. Not everyone has access to frontier-level graphics cards, and so the training of these models at size can become a blocker. While we were able to show that amortisation works well if you're willing to trade some training budget for sampling, we didn't attempt to train an amortised GFlowNet so aren't able to provide a contrasted argument there. Cost accounting is also not uniformly referenced in the wider literature, some include it, some don't, and those that do often differ in their measurement. For example, Blessing et al. [33], Du et al. [49] use energy evaluations or wall-clock per condition, which favours a learned sampler in cases where energy evaluation is more expensive than the network. Our accounting option sits at the other end of the spectrum, where network forwards will typically dominate. Better to be conservative, in my view.

A glimmer of light in an otherwise expensive abyss is the observation that FLOPs per effective sample in the neural case grows only marginally between subcritical and critical cases, where the classical baselines cost increase materially more. There is likely a point on the extrapolated curve where these two costs meet, at which point the larger cost of training the network may become significant. An interesting point to keep an eye on as these samplers grow.

Problem size. The largest problems we were able to cover in this thesis were a 24×24 Ising and a partial attempt on a 64-site Cu–Au cell. While the Ising dimensions are broadly in-line with those used in the sampling literature, and comparable to the size of *alloy* used by Sadigh and Erhart [26, Sec. IV] in the VCSGC paper, there is always room for upward improvement. On the unconstrained side, a recent paper was able to train a transformer-based sampler which reached 180×180, reporting roughly twenty times the effective sample size of the previous neural state of the art at 128×128 criticality [101]. Something to aim for.

6.3 Future Work

While I put everything I had into this project, there were certain interesting paths I simply didn't have time to tackle, left to future work. For fear of overspilling onto the next page, I'll simply list them: training the sampler on further FCC alloy configurations, further research into base distribution design, applying this same trajectory-level hard constraint to another neural sampler in the wider family, and as always, scaling. The thrill of the chase; closing one door opens more than you can imagine. Let's give the GPUs a rest for the time being, though.

Bibliography

[1] Department for Energy Security and Net Zero. Greenhouse gas reporting: conversion factors 2026, June 2026. URL https://www.gov.uk/government/publications/greenhouse-gas-reporting-conversion-factors-2026. Revised flat-format data, version 1.2, 31 July 2026.

[2] Noga Alon, Thomas F. Bloom, W. T. Gowers, Daniel Litt, Will Sawin, Arul Shankar, Jacob Tsimerman, Victor Wang, and Melanie Matchett Wood. Remarks on the disproof of the unit distance conjecture. *arXiv preprint arXiv:2605.20695*, May 2026. URL https://arxiv.org/abs/2605.20695.

[3] Terence Tao. A digestion of the Jacobian conjecture counterexample. Blog post, *What's new*, July 2026. URL https://terrytao.wordpress.com/2026/07/21/a-digestion-of-the-jacobian-conjecture-counterexample/.

[4] OpenAI. Ten advances in mathematics and theoretical computer science. OpenAI publication announcement, August 2026. URL https://openai.com/index/ten-advances-in-mathematics/.

[5] OpenAI. On the navier–stokes millennium prize problem. OpenAI publication announcement, September 2026. URL https://openai.com/index/navier-stokes-solution/.

[6] Dario Amodei. Machines of loving grace: How AI could transform the world for the better. Personal essay, October 2024. URL https://www.darioamodei.com/essay/machines-of-loving-grace.

[7] John Jumper, Richard Evans, Alexander Pritzel, Tim Green, et al. Highly accurate protein structure prediction with AlphaFold. *Nature*, 596(7873):583–589, 2021. doi: 10.1038/s41586-021-03819-2.

[8] Felix Wong, Erica J. Zheng, Jacqueline A. Valeri, Nina M. Donghia, Melis N. Anahtar, Satotaka Omori, Alicia Li, Andres Cubillos-Ruiz, Aarti Krishnan, Wengong Jin, Abigail L. Manson, Jens Friedrichs, Ralf Helbig, Behnoush Hajian, Dawid K. Fiejtek, Florence F. Wagner, Holly H. Soutter, Ashlee M. Earl, Jonathan M. Stokes, Lars D. Renner, and James J. Collins. Discovery of a structural class of antibiotics with explainable deep learning. *Nature*, 626(7997):177–185, 2024. doi: 10.1038/s41586-023-06887-8.

[9] J. M. Sanchez, F. Ducastelle, and D. Gratias. Generalized cluster description of multicomponent systems. *Physica A: Statistical Mechanics and its Applications*, 128(1-2): 334–350, 1984. doi: 10.1016/0378-4371(84)90096-7. URL https://doi.org/10.1016/0378-4371(84)90096-7.

[10] A. van de Walle and M. Asta. Self-driven lattice-model Monte Carlo simulations of alloy thermodynamic properties and phase diagrams. *Modelling and Simulation in Materials Science and Engineering*, 10(5):521–538, September 2002. ISSN 0965-0393. doi: 10.1088/0965-0393/10/5/304. URL https://iopscience.iop.org/article/10.1088/0965-0393/10/5/304.

[11] Ruopeng Zhang, Shiteng Zhao, Jun Ding, Yan Chong, Tao Jia, Colin Ophus, Mark Asta, Robert O. Ritchie, and Andrew M. Minor. Short-range order and its impact on the CrCoNi medium-entropy alloy. *Nature*, 581(7808):283–287, 2020. doi: 10.1038/s41586-020-2275-z.

[12] Nicholas Marcella, Jin Soo Lim, Anna M. Płonka, George Yan, Cameron J. Owen, Jessi E. S. van der Hoeven, Alexandre C. Foucher, Hio Tong Ngan, Steven B. Torrisi, Nebojsa S. Marinkovic, Eric A. Stach, Jason F. Weaver, Joanna Aizenberg, Philippe Sautet, Boris

Kozinsky, and Anatoly I. Frenkel. Decoding reactive structures in dilute alloy catalysts. *Nature Communications*, 13(1):832, 2022. doi: 10.1038/s41467-022-28366-w.

[13] Amil Merchant, Simon Batzner, Samuel S. Schoenholz, Muratahan Aykol, Gowoon Cheon, and Ekin Dogus Cubuk. Scaling deep learning for materials discovery. *Nature*, 624 (7990):80–85, 2023. doi: 10.1038/s41586-023-06735-9. URL https://doi.org/10.1038/s41586-023-06735-9.

[14] Claudio Zeni, Robert Pinsler, Daniel Zügner, Andrew Fowler, Matthew Horton, Xiang Fu, Zilong Wang, Aliaksandra Shysheya, Jonathan Crabbé, Shoko Ueda, Roberto Sordillo, Lixin Sun, Jake Smith, Bichlien Nguyen, Hannes Schulz, Sarah Lewis, Chin-Wei Huang, Ziheng Lu, Yichi Zhou, Han Yang, Hongxia Hao, Jielan Li, Chunlei Yang, Wenjie Li, Ryota Tomioka, and Tian Xie. A generative model for inorganic materials design. *Nature*, 639:624–632, 2025. doi: 10.1038/s41586-025-08628-5. URL https://doi.org/10.1038/s41586-025-08628-5.

[15] Lowik Chanussot, Abhishek Das, Siddharth Goyal, Thibaut Lavril, Muhammed Shuaibi, Morgane Riviere, Kevin Tran, Javier Heras-Domingo, Caleb Ho, Weihua Hu, Aini Palizhati, Anuroop Sriram, Brandon Wood, Junwoong Yoon, Devi Parikh, C. Lawrence Zitnick, and Zachary Ulissi. Open Catalyst 2020 (OC20) Dataset and Community Challenges. *ACS Catalysis*, 11(10):6059–6072, 2021. doi: 10.1021/acscatal.0c04525. URL https://doi.org/10.1021/acscatal.0c04525.

[16] CuspAI. Launching our 'AI Materials Foundry' — and $450 million Series B, 2026. URL https://medium.com/@CuspAI/launching-our-ai-materials-foundry-and-450-million-series-b-f5603d8259dd. Accessed 2026-08-11.

[17] Anjney Midha. Investing in Periodic Labs. Andreessen Horowitz, 2025. URL https://a16z.com/announcement/investing-in-periodic-labs/. Accessed 2026-08-11.

[18] Kyle Wiggers. This startup is using AI to discover new materials. TechCrunch, 2024. URL https://techcrunch.com/2024/02/21/this-startup-is-using-ai-to-discover-new-materials/. Accessed 2026-08-11.

[19] Will Grathwohl, Kevin Swersky, Milad Hashemi, David Duvenaud, and Chris J. Maddison. Oops I took a gradient: Scalable sampling for discrete distributions. In *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pages 3831–3841. PMLR, 2021. URL https://proceedings.mlr.press/v139/grathwohl21a.html.

[20] Ruqi Zhang, Xingchao Liu, and Qiang Liu. A Langevin-like sampler for discrete distributions. In *Proceedings of the 39th International Conference on Machine Learning*, volume 162 of *Proceedings of Machine Learning Research*, pages 26375–26396. PMLR, 2022. URL https://proceedings.mlr.press/v162/zhang22t.html.

[21] Robert H. Swendsen and Jian-Sheng Wang. Nonuniversal critical dynamics in Monte Carlo simulations. *Physical Review Letters*, 58(2):86–88, 1987. doi: 10.1103/PhysRevLett.58.86. URL https://doi.org/10.1103/PhysRevLett.58.86.

[22] Ulli Wolff. Collective Monte Carlo updating for spin systems. *Physical Review Letters*, 62 (4):361–364, 1989. doi: 10.1103/PhysRevLett.62.361. URL https://doi.org/10.1103/PhysRevLett.62.361.

[23] David J. Earl and Michael W. Deem. Parallel tempering: theory, applications, and new perspectives. *Physical Chemistry Chemical Physics*, 7(23):3910–3916, 2005. doi: 10.1039/B509983H. URL https://doi.org/10.1039/B509983H.

[24] Fugao Wang and D. P. Landau. Efficient, multiple-range random walk algorithm to calculate the density of states. *Physical Review Letters*, 86(10):2050–2053, 2001. doi: 10.1103/PhysRevLett.86.2050. URL https://doi.org/10.1103/PhysRevLett.86.2050.

[25] Kyozi Kawasaki. Diffusion constants near the critical point for time-dependent Ising models. I. *Physical Review*, 145(1):224–230, 1966. doi: 10.1103/PhysRev.145.224. URL https://doi.org/10.1103/PhysRev.145.224.

[26] Babak Sadigh and Paul Erhart. Calculation of excess free energies of precipitates via direct thermodynamic integration across phase boundaries. *Physical Review B*, 86(13): 134204, October 2012. ISSN 1098-0121, 1550-235X. doi: 10.1103/PhysRevB.86.134204. URL https://link.aps.org/doi/10.1103/PhysRevB.86.134204.

[27] Jacob Austin, Daniel D. Johnson, Jonathan Ho, Daniel Tarlow, and Rianne van den Berg. Structured Denoising Diffusion Models in Discrete State-Spaces. In *Advances in Neural Information Processing Systems*, volume 34, pages 17981–17993. Curran Associates, Inc., 2021. URL https://proceedings.neurips.cc/paper/2021/hash/958c530554f78bcd8e97125b70e6973d-Abstract.html.

[28] Aaron Lou, Chenlin Meng, and Stefano Ermon. Discrete Diffusion Modeling by Estimating the Ratios of the Data Distribution. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 32819–32848. PMLR, 2024. URL https://proceedings.mlr.press/v235/lou24a.html.

[29] Zijing Ou, Ruixiang Zhang, and Yingzhen Li. Discrete neural flow samplers with locally equivariant transformer. In *Advances in Neural Information Processing Systems*, volume 38, pages 127197–127237. Curran Associates, Inc., 2025. doi: 10.52202/085713-4238. URL https://proceedings.neurips.cc/paper_files/paper/2025/file/b8f21f324ff277ba26aed2e944b7576b-Paper-Conference.pdf. Poster. arXiv:2505.17741.

[30] Peter Holderrieth, Michael S. Albergo, and Tommi Jaakkola. LEAPS: A discrete neural sampler via locally equivariant networks. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 23397–23416. PMLR, 2025. URL https://proceedings.mlr.press/v267/holderrieth25a.html.

[31] Yuchen Zhu, Wei Guo, Jaemoo Choi, Guan-Horng Liu, Yongxin Chen, and Molei Tao. MDNS: Masked Diffusion Neural Sampler via Stochastic Optimal Control. In *Advances in Neural Information Processing Systems*, 2025. URL https://arxiv.org/abs/2508.10684. arXiv:2508.10684.

[32] Wei Guo, Yuchen Zhu, Xiaochen Du, Juno Nam, Yongxin Chen, Rafael Gómez-Bombarelli, Guan-Horng Liu, Molei Tao, and Jaemoo Choi. Discrete adjoint Schrödinger bridge sampler, 2026. URL https://arxiv.org/abs/2602.08243.

[33] Denis Blessing, Mouyang Cheng, Maximilian Schebek, Jutta Rogal, Mingda Li, Carles Domingo-Enrich, and Yuanqi Du. JANUS: A multi-modal foundation neural sampler for disordered materials, 2026. URL https://arxiv.org/abs/2608.19116.

[34] Emmanuel Bengio, Moksh Jain, Maksym Korablyov, Doina Precup, and Yoshua Bengio. Flow Network based Generative Models for Non-Iterative Diverse Candidate Generation. In *Advances in Neural Information Processing Systems*, volume 34, pages 27381–27394. Curran Associates, Inc., 2021. URL https://proceedings.neurips.cc/paper_files/paper/2021/hash/e614f646836aaed9f89ce58e837e2310-Abstract.html.

[35] Kanika Madan, Jarrid Rector-Brooks, Maksym Korablyov, Emmanuel Bengio, Moksh Jain, Andrei Cristian Nica, Tom Bosc, Yoshua Bengio, and Esmeralda S. Whitammer.

Learning GFlowNets from partial episodes for improved convergence and stability. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 23467–23483. PMLR, 2023. URL `https://proceedings.mlr.press/v202/madan23a.html`.

[36] Geoffrey E. Hinton. Training products of experts by minimizing contrastive divergence. *Neural Computation*, 14(8):1771–1800, 2002. doi: 10.1162/089976602760128018. URL `https://doi.org/10.1162/089976602760128018`.

[37] Aapo Hyvärinen. Estimation of non-normalized statistical models by score matching. *Journal of Machine Learning Research*, 6(24):695–709, 2005. URL `https://www.jmlr.org/papers/v6/hyvarinen05a.html`.

[38] David A. Levin and Yuval Peres. *Markov Chains and Mixing Times*. American Mathematical Society, Providence, RI, second edition, 2017. doi: 10.1090/mbk/107. URL `https://pages.uoregon.edu/dlevin/MARKOV/mcmt2e.pdf`. With contributions by Elizabeth L. Wilmer.

[39] M. E. J. Newman and G. T. Barkema. *Monte Carlo Methods in Statistical Physics*. Oxford University Press, Oxford, UK, 1999. ISBN 9780198517979.

[40] Giacomo Zanella. Informed proposals for local MCMC in discrete spaces. *Journal of the American Statistical Association*, 115(530):852–865, 2020. doi: 10.1080/01621459.2019.1585255.

[41] Radford M. Neal. Annealed importance sampling. *Statistics and Computing*, 11(2):125–139, April 2001. ISSN 1573-1375. doi: 10.1023/A:1008923215028. URL `https://doi.org/10.1023/A:1008923215028`.

[42] Arnaud Doucet, Nando de Freitas, and Neil Gordon, editors. *Sequential Monte Carlo Methods in Practice*. Springer, New York, 2001. doi: 10.1007/978-1-4757-3437-9.

[43] Ernst Ising. Beitrag zur Theorie des Ferromagnetismus. *Zeitschrift für Physik*, 31(1):253–258, 1925. doi: 10.1007/BF02980577.

[44] Lars Onsager. Crystal statistics. I. A two-dimensional model with an order-disorder transition. *Physical Review*, 65(3-4):117–149, 1944. doi: 10.1103/PhysRev.65.117.

[45] J. R. Heringa and H. W. J. Blöte. Geometric cluster Monte Carlo simulation. *Physical Review E*, 57(5):4976–4978, 1998. doi: 10.1103/PhysRevE.57.4976.

[46] Jiwen Liu and Erik Luijten. Rejection-free geometric cluster algorithm for complex fluids. *Physical Review Letters*, 92(3):035504, 2004. doi: 10.1103/PhysRevLett.92.035504.

[47] Jun S. Liu. *Monte Carlo Strategies in Scientific Computing*. Springer Series in Statistics. Springer, New York, NY, 2001.

[48] Augustine Kong. A note on importance sampling using standardized weights. Technical Report 348, University of Chicago, Department of Statistics, 1992.

[49] Xiaochen Du, Juno Nam, Jaemoo Choi, Wei Guo, Sathya Edamadaka, Junyi Sha, Elton Pan, Yongxin Chen, Molei Tao, and Rafael Gómez-Bombarelli. MetaDNS: Enhancing exploration in discrete neural samplers via well-tempered metadynamics, 2026. URL `https://arxiv.org/abs/2605.21722`.

[50] Sergios Agapiou, Omiros Papaspiliopoulos, Daniel Sanz-Alonso, and Andrew M. Stuart. Importance sampling: intrinsic dimension and computational cost. *Statistical Science*, 32(3):405–431, 2017. doi: 10.1214/17-STS611.

[51] Arthur E. Ferdinand and Michael E. Fisher. Bounded and inhomogeneous Ising models. I. Specific-heat anomaly of a finite lattice. *Physical Review*, 185(2):832–846, 1969. doi: 10.1103/PhysRev.185.832.

[52] Aditya Choudhary, Saaketh Desai, and Prasad Iyer. Boltzmann reinforcement learning for noise resilience in analog Ising machines, 2026. URL http://arxiv.org/abs/2602.09162.

[53] Ivan Kobyzev, Simon J. D. Prince, and Marcus A. Brubaker. Normalizing Flows: An Introduction and Review of Current Methods. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 43(11):3964–3979, November 2021. ISSN 0162-8828, 2160-9292, 1939-3539. doi: 10.1109/TPAMI.2020.2992934. URL http://arxiv.org/abs/1908.09257. arXiv:1908.09257 [stat].

[54] Frank Noé, Simon Olsson, Jonas Köhler, and Hao Wu. Boltzmann generators: sampling equilibrium states of many-body systems with deep learning. *Science*, 365(6457):eaaw1147, 2019. doi: 10.1126/science.aaw1147. URL https://doi.org/10.1126/science.aaw1147.

[55] Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow Matching for Generative Modeling. In *International Conference on Learning Representations*, 2023. URL https://arxiv.org/abs/2210.02747.

[56] Michael S. Albergo, Nicholas M. Boffi, and Eric Vanden-Eijnden. Stochastic Interpolants: A Unifying Framework for Flows and Diffusions. *Journal of Machine Learning Research*, 26(209):1–80, 2025. URL https://jmlr.org/papers/v26/23-1605.html.

[57] Qinsheng Zhang and Yongxin Chen. Path integral sampler: A stochastic control approach for sampling. In *International Conference on Learning Representations*, 2022.

[58] Francisco Vargas, Will Grathwohl, and Arnaud Doucet. Denoising diffusion samplers. In *International Conference on Learning Representations*, 2023.

[59] Francisco Vargas, Shreyas Padhy, Denis Blessing, and Nikolas Nüsken. Transport meets variational inference: Controlled Monte Carlo diffusions. In *International Conference on Learning Representations*, 2024.

[60] Michael S. Albergo and Eric Vanden-Eijnden. NETS: A non-equilibrium transport sampler, 2024. URL https://arxiv.org/abs/2410.02711.

[61] Sebastian Sanokowski, Wilhelm Berghammer, Haoyu Wang, Martin Ennemoser, Sepp Hochreiter, and Sebastian Lehner. Scalable discrete diffusion samplers: Combinatorial optimization and statistical physics. In *International Conference on Learning Representations*, pages 87053–87082, 2025. URL https://proceedings.iclr.cc/paper_files/paper/2025/hash/d87eadbe6114b768e76c5d5e8eb39388-Abstract-Conference.html. arXiv:2502.08696.

[62] Andrew Campbell, Joe Benton, Valentin De Bortoli, Tom Rainforth, George Deligiannidis, and Arnaud Doucet. A Continuous Time Framework for Discrete Denoising Models. In *Advances in Neural Information Processing Systems*, volume 35, pages 28266–28279. Curran Associates, Inc., 2022. doi: 10.52202/068431-2049. URL https://proceedings.neurips.cc/paper_files/paper/2022/hash/b5b528767aa35f5b1a60fe0aaeca0563-Abstract-Conference.html.

[63] Itai Gat, Tal Remez, Neta Shaul, Felix Kreuk, Ricky T. Q. Chen, Gabriel Synnaeve, Yossi Adi, and Yaron Lipman. Discrete Flow Matching. In *Advances in Neural Information Processing Systems*, volume 37, pages 133345–133385. Curran Associates, Inc., 2024. doi: 10.52202/079017-4239. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/f0d629a734b56a642701bba7bc8bb3ed-Abstract-Conference.html. arXiv:2407.15595.

[64] Wei Guo, Jaemoo Choi, Yuchen Zhu, Molei Tao, and Yongxin Chen. Proximal diffusion neural sampler. In *International Conference on Learning Representations*, pages 34373–34408, 2026. URL https://proceedings.iclr.cc/paper_files/paper/2026/file/3a0e2de215bd17c39ad08ba1d16c1b12-Paper-Conference.pdf. arXiv:2510.03824.

[65] Sergei Kholkin, Francisco Vargas, and Alexander Korotin. Sampling from energy distributions with target concrete score identity, 2025. URL https://arxiv.org/abs/2510.23106.

[66] Dongyeop Woo, Minsu Kim, Minkyu Kim, Kiyoung Seong, and Sungsoo Ahn. Energy-based generator matching: A neural sampler for general state space. In *Advances in Neural Information Processing Systems*, volume 38, pages 110247–110289. Curran Associates, Inc., 2025. doi: 10.52202/085713-3683. URL https://proceedings.neurips.cc/paper_files/paper/2025/hash/9fb5cf6a55de84a6c143a0d42ad90062-Abstract-Conference.html. arXiv:2505.19646.

[67] Oswin So, Brian Karrer, Chuchu Fan, Ricky T. Q. Chen, and Guan-Horng Liu. Discrete adjoint matching. In *International Conference on Learning Representations*, pages 32834–32868, 2026. URL https://proceedings.iclr.cc/paper_files/paper/2026/file/378dcc0bb9a61a4c1c00e19ad9890425-Paper-Conference.pdf. Poster. arXiv:2602.07132.

[68] Dian Wu, Lei Wang, and Pan Zhang. Solving statistical mechanics using variational autoregressive networks. *Physical Review Letters*, 122:080602, 2019. doi: 10.1103/PhysRevLett.122.080602.

[69] Esmeralda S. Whitammer, Moksh Jain, Emmanuel Bengio, Chen Sun, and Yoshua Bengio. Trajectory balance: Improved credit assignment in GFlowNets. In *Advances in Neural Information Processing Systems*, volume 35, pages 5955–5967. Curran Associates, Inc., 2022. doi: 10.52202/068431-0431. URL https://proceedings.neurips.cc/paper_files/paper/2022/hash/27b51baca8377a0cf109f6ecc15a0f70-Abstract-Conference.html.

[70] Mila AI4Science, Alex Hernandez-Garcia, Alexandre Duval, Alexandra Volokhova, Yoshua Bengio, Divya Sharma, Pierre Luc Carrier, Yasmine Benabed, Michał Koziarski, Victor Schmidt, Gian-Marco Rignanese, Pierre-Paul De Breuck, and Paulette Clancy. Crystal-GFN: sampling crystals with desirable properties and constraints, 2023. URL https://arxiv.org/abs/2310.04925. A shorter version appeared at the AI4Mat workshop, NeurIPS 2023.

[71] Mattias Ångqvist, William A. Muñoz, J. Magnus Rahm, Erik Fransson, Céline Durniak, Piotr Rozyczko, Thomas H. Rod, and Paul Erhart. ICET – A Python Library for Constructing and Sampling Alloy Cluster Expansions. *Advanced Theory and Simulations*, 2(7):1900015, 2019. ISSN 2513-0390. doi: 10.1002/adts.201900015. URL https://onlinelibrary.wiley.com/doi/abs/10.1002/adts.201900015.

[72] Alex Zunger, S.-H. Wei, L. G. Ferreira, and James E. Bernard. Special quasirandom structures. *Physical Review Letters*, 65(3):353–356, 1990. doi: 10.1103/PhysRevLett.65.353. URL https://doi.org/10.1103/PhysRevLett.65.353.

[73] Hunter Nisonoff, Junhao Xiong, Stephan Allenspach, and Jennifer Listgarten. Unlocking guidance for discrete state-space diffusion and flow models. In *International Conference on Learning Representations*, 2025. URL https://arxiv.org/abs/2406.01572. Poster. arXiv:2406.01572.

[74] Prafulla Dhariwal and Alexander Nichol. Diffusion models beat GANs on image synthesis. In *Advances in Neural Information Processing Systems*, volume 34, pages 8780–8794. Curran Associates, Inc., 2021. URL https://proceedings.neurips.cc/paper_files/paper/2021/file/49ad23d1ec9fa4bd8d77d02681df5cfa-Paper.pdf. arXiv:2105.05233.

[75] Jonathan Ho and Tim Salimans. Classifier-free diffusion guidance, 2022. URL https://arxiv.org/abs/2207.12598.

[76] James Damewood, Daniel Schwalbe-Koda, and Rafael Gómez-Bombarelli. Sampling lattices in semi-grand canonical ensemble with autoregressive machine learning. *npj Computational Materials*, 8:61, 2022. doi: 10.1038/s41524-022-00736-4. URL https://arxiv.org/abs/2107.05109. arXiv:2107.05109 [cond-mat.stat-mech].

[77] Xiaochen Du, Juno Nam, Sulin Liu, and Rafael Gómez-Bombarelli. Scaling autoregressive models for lattice thermodynamics, 2026. URL https://arxiv.org/abs/2603.14695.

[78] Esmeralda S. Whitammer, Salem Lahlou, Tristan Deleu, Xu Ji, Edward Hu, Katie Everett, Dinghuai Zhang, and Yoshua Bengio. GFlowNets and variational inference. In *International Conference on Learning Representations*, 2023. URL https://openreview.net/forum?id=uKiE0VIluA-.

[79] Dinghuai Zhang, Hanjun Dai, Esmeralda S. Whitammer, Aaron Courville, Yoshua Bengio, and Ling Pan. Let the Flows Tell: Solving Graph Combinatorial Problems with GFlowNets. In *Advances in Neural Information Processing Systems*, volume 36, pages 11952–11969. Curran Associates, Inc., 2023. doi: 10.52202/075280-0522. URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/27571b74d6cd650b8eb6cf1837953ae8-Abstract-Conference.html.

[80] Ling Pan, Esmeralda S. Whitammer, Dinghuai Zhang, and Yoshua Bengio. Better Training of GFlowNets with Local Credit and Incomplete Trajectories. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 26878–26890. PMLR, 2023. URL https://proceedings.mlr.press/v202/pan23c.html.

[81] Michael Cardei, Jacob K Christopher, Bhavya Kailkhura, Tom Hartvigsen, and Ferdinando Fioretto. Constrained discrete diffusion. In *Advances in Neural Information Processing Systems*, volume 38, pages 12383–12416. Curran Associates, Inc., 2025. doi: 10.52202/085713-0415. URL https://proceedings.neurips.cc/paper_files/paper/2025/file/124c1cc9f7750f512ed81cdbf2fd5bf5-Paper-Conference.pdf. arXiv:2503.09790.

[82] Jacob K. Christopher, Austin Seamann, Jingyi Cui, Sagar Khare, and Ferdinando Fioretto. Constrained Diffusion for Protein Design with Hard Structural Constraints. In *International Conference on Learning Representations*, 2026. URL https://openreview.net/forum?id=kkvqVRu2Zy. arXiv:2510.14989.

[83] Firas Hamze and Nando de Freitas. Intracluster moves for constrained discrete-space MCMC. In *Proceedings of the 26th Conference on Uncertainty in Artificial Intelligence (UAI)*, 2010. URL https://arxiv.org/abs/1203.3484. arXiv:1203.3484 [stat.CO].

[84] Javier E. Santos, Agnese Marcato, Roman Colman, Nicholas Lubbers, and Yen Ting Lin. Discrete spatial diffusion: Intensity-preserving diffusion modeling, 2025. URL https://arxiv.org/abs/2505.01917.

[85] Ahmadreza Azizi and Michel Pleimling. A cautionary tale for machine learning generated configurations in presence of a conserved quantity. *Scientific Reports*, 11:6395, 2021. doi: 10.1038/s41598-021-85683-8. URL https://arxiv.org/abs/2007.09764. arXiv:2007.09764 [cond-mat.stat-mech].

[86] Florentin Coeurdoux, Nicolas Dobigeon, and Pierre Chainais. Normalizing flow sampling with Langevin dynamics in the latent space. *Machine Learning*, 113(11–12):8301–8326, 2024. doi: 10.1007/s10994-024-06623-x. arXiv:2305.12149.

[87] Leonardo Galliano, Riccardo Rende, and Daniele Coslovich. Policy-guided Monte Carlo on general state spaces: application to glass-forming mixtures. *The Journal of Chemical Physics*, 161(6):064503, 2024. doi: 10.1063/5.0221221. URL `https://arxiv.org/abs/2407.03275`. arXiv:2407.03275 [cond-mat.soft].

[88] Yuya Kawamata, Yuichiro Nakano, and Keisuke Fujii. Divide-and-conquer neural network surrogates for quantum sampling: Accelerating Markov chain Monte Carlo in large-scale constrained optimization problems, 2026. URL `https://arxiv.org/abs/2604.20701`.

[89] Ricky T. Q. Chen and David Duvenaud. Neural Networks with Cheap Differential Operators. In *Advances in Neural Information Processing Systems*, volume 32, 2019.

[90] Tomasz Korbak, Ethan Perez, and Christopher L. Buckley. RL with KL penalties is better viewed as Bayesian inference. In *Findings of the Association for Computational Linguistics: EMNLP 2022*, pages 1083–1091, Abu Dhabi, United Arab Emirates, 2022. Association for Computational Linguistics. doi: 10.18653/v1/2022.findings-emnlp.77. URL `https://aclanthology.org/2022.findings-emnlp.77/`. arXiv:2205.11275.

[91] Ari Seff, Wenda Zhou, Farhan Damani, Abigail Doyle, and Ryan P. Adams. Discrete Object Generation with Reversible Inductive Construction. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 32, 2019. URL `http://arxiv.org/abs/1907.08268`. arXiv:1907.08268 [cs, stat].

[92] Persi Diaconis and Bernd Sturmfels. Algebraic algorithms for sampling from conditional distributions. *The Annals of Statistics*, 26(1):363–397, 1998. doi: 10.1214/aos/1030563990.

[93] Daniel T. Gillespie. Approximate accelerated stochastic simulation of chemically reacting systems. *The Journal of Chemical Physics*, 115(4):1716–1733, 2001. doi: 10.1063/1.1378322.

[94] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

[95] Jianlin Su, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. RoFormer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568:127063, 2024. doi: 10.1016/j.neucom.2023.127063.

[96] Paulius Micikevicius, Sharan Narang, Jonah Alben, Gregory Diamos, Erich Elsen, David Garcia, Boris Ginsburg, Michael Houston, Oleksii Kuchaiev, Ganesh Venkatesh, and Hao Wu. Mixed precision training. In *International Conference on Learning Representations*, 2018. URL `https://arxiv.org/abs/1710.03740`.

[97] Sam McCandlish, Jared Kaplan, Dario Amodei, and OpenAI Dota Team. An empirical model of large-batch training, 2018. URL `http://arxiv.org/abs/1812.06162`.

[98] Adrian Tam. Train a model faster with torch.compile and gradient accumulation. MachineLearningMastery.com, January 2026. URL `https://machinelearningmastery.com/train-a-model-faster-with-torch-compile-and-gradient-accumulation/`.

[99] Lewis Fry Richardson and J. Arthur Gaunt. The deferred approach to the limit. *Philosophical Transactions of the Royal Society of London. Series A*, 226:299–361, 1927. doi: 10.1098/rsta.1927.0008.

[100] Yuhuai Wu, Mengye Ren, Renjie Liao, and Roger Grosse. Understanding short-horizon bias in stochastic meta-optimization. In *International Conference on Learning Representations*, 2018. URL `https://arxiv.org/abs/1803.02021`.

[101] Piotr Białas, Piotr Korcyl, Tomasz Stebel, Adam Stefański, and Dawid Zapolski. Sampling

two-dimensional spin systems with transformers, 2026. URL `https://arxiv.org/abs/2604.27738`.

[102] Sourav Chatterjee and Persi Diaconis. The sample size required in importance sampling. *The Annals of Applied Probability*, 28(2):1099–1135, 2018. doi: 10.1214/17-AAP1326.

[103] David Ruppert. Efficient estimations from a slowly convergent Robbins–Monro process. Technical Report 781, School of Operations Research and Industrial Engineering, Cornell University, February 1988. URL `https://hdl.handle.net/1813/8664`.

[104] Boris T. Polyak and Anatoli B. Juditsky. Acceleration of stochastic approximation by averaging. *SIAM Journal on Control and Optimization*, 30(4):838–855, 1992. doi: 10.1137/0330046.

[105] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models, 2020. URL `http://arxiv.org/abs/2001.08361`.

Glossary of Acronyms

AI artificial intelligence. 1

CDF cumulative distribution function. 7

CPU central processing unit. 36, 55

CTMC continuous-time Markov chain. 2, 4, 7–10, 13–15, 18, 31

CV control variate. 81

DAG directed acyclic graph. 2, 10

DAM discrete adjoint matching. 10

DASBS discrete adjoint Schrödinger bridge sampler. 10

DB detailed balance. 43, 45–47, 82, 83, 90

DFT density functional theory. 3, 11, 12

DMALA discrete Metropolis-adjusted Langevin algorithm. 4

DNFS discrete neural flow sampler. 2, 7–10, 13–15, 17–21, 23, 26–28, 31–34, 37, 38, 40, 51, 53, 54, 72, 74, 76, 78, 86

DSD discrete spatial diffusion. 13

ECI effective cluster interaction. 11

EGM energy-based generator matching. 9, 10

EMA exponential moving average. 26, 43, 44, 47–51, 53, 72, 76, 78, 79, 81–84, 86–89

ESS effective sample size. 5, 6, 18–20, 24–27, 29, 30, 32, 33, 42–47, 49–53, 70, 71, 76, 78, 79, 81–83

FCC face-centred cubic. 52, 53, 55

FLOP floating-point operation. 6, 19, 20, 26, 27, 29, 30, 44–47, 55, 82, 83, 85, 89, 90

GFlowNet generative flow network. 2, 10, 13, 32, 43–47, 55, 82, 83, 86, 88, 90

GPU graphics processing unit. 6, 42, 43, 55

GWG Gibbs with gradients. 4

icet integrated cluster expansion toolkit. 11, 12

KFE Kolmogorov forward equation. 8–10

KL Kullback–Leibler. 10, 22

KV key–value. 44, 88

LEAPS locally equivariant discrete annealed proactive sampler. 4, 7–10, 14, 41

leCNN locally equivariant convolutional neural network. 19

leMLP locally equivariant multilayer perceptron. 19

leTF locally equivariant transformer. 16–20, 37, 38, 43, 87, 89

MCMC Markov chain Monte Carlo. i, 1–4, 6, 15, 19, 31, 32

MDNS masked discrete neural sampler. 4, 6, 7, 9, 10, 20, 24, 32, 78, 88

MetaDNS meta-dynamics discrete neural sampler. 13, 22, 52, 53, 88

ML machine learning. 1, 10

MLP multilayer perceptron. 19, 38, 40–42, 78, 81, 89

ODE ordinary differential equation. 7, 8

OOM out of memory. 42

PDNS proximal diffusion neural sampler. 9, 10

PINN physics-informed neural network. 9, 10

SB Schrödinger bridge. 10

SD standard deviation. 19–21, 24, 27, 30, 45–47, 50, 53, 76, 82, 83

SDDS scalable discrete diffusion sampler. 8

SGC semi-grand-canonical. 2, 13, 14, 20, 22

SMC sequential Monte Carlo. 4, 7, 86

SQS special quasirandom structure. 12

TB trajectory balance. 43

TV total variation. 7, 45, 53, 78, 80

VCSGC variance-constrained semi-grand-canonical. iii, 2, 10, 12, 13, 22, 23, 27, 28, 30, 54, 55, 76

ViT vision transformer. 78

A Initial Project Description

Discrete neural flow samplers with constraints for materials discovery

Neural flow samplers learn a transformation that maps simple base distributions to complex target distributions. This allows fast, one-shot sampling, unlike traditional methods like Markov chain Monte Carlo (MCMC) that require long chains and burn-in periods. In discrete domains, e.g., configurations of atoms, molecules, or spin systems, the space of valid states can be highly structured and constrained. This project addresses several open challenges in probabilistic modelling and generative learning over discrete spaces. Our group has previously developed the first neural flow sampler for sampling from unnormalised discrete distributions [10.48550/arXiv.2505.17741]. This project will build upon this work in two aspects: i) The first is adding constraints on the distribution, specifically, we will constrain the sampling to ensure that the total number of each class in the distribution is fixed. This problem maps directly onto the problem of sampling disordered solid solutions in materials chemistry and is essential for understanding the stability of novel materials for energy applications. ii) The second advance will be to scale the method to target higher-dimensional data distributions for practical applications. These technical challenges sit at the intersection of deep generative modelling, discrete optimisation, and statistical physics, and offer opportunities for novel algorithmic contributions.

The project has the following goals:

1. To develop a discrete neural flow sampler that supports hard constraints.
2. To train the model on a real-world dataset of disordered inorganic crystals.
3. To benchmark the approach against state-of-the-art approaches, including MCMC and combinatorial optimisation.

While this project is focussed on a chemical dataset, the method will have broad utility across many domains, from fundamental physics to drug discovery.

Representative references:

1. `https://doi.org/10.48550/arXiv.2305.12149`
2. `https://doi.org/10.48550/arXiv.1908.09257`
3. `https://doi.org/10.48550/arXiv.1905.10347`
4. `https://doi.org/10.48550/arXiv.1711.00937`
5. `https://doi.org/10.1088/2632-2153/ad6ca0`

Requirements:

Students should have strong programming skills in Python and familiarity with deep learning frameworks (e.g. PyTorch). A solid background in linear algebra, probability, and optimisation is essential. Prior experience with generative models, discrete mathematics, or scientific computing is highly beneficial, as is interest in applying AI to physical sciences.

B Kolmogorov Forward Equation Derivation

We work on a finite state space; the final log form requires $p_t(x) > 0$.

We seek the time evolution of the marginal $p_t(x)$. By the Markov property, condition on the state occupied at time t and apply the law of total probability over a single infinitesimal step of length Δt:

$$p_{t+\Delta t}(x) = \sum_y p_{t+\Delta t|t}(x \mid y)\, p_t(y). \tag{B.1}$$

Substitute the infinitesimal transition probability (2.6) introduced in Section 2.2.2,

$$p_{t+\Delta t|t}(x \mid y) = \mathbf{1}_{x=y} + R_t(x, y)\, \Delta t + o(\Delta t), \tag{B.2}$$

to give

$$p_{t+\Delta t}(x) = \sum_y \left[\mathbf{1}_{x=y} + R_t(x, y)\, \Delta t + o(\Delta t)\right] p_t(y). \tag{B.3}$$

The indicator collapses the first contribution to $\sum_y \mathbf{1}_{x=y}\, p_t(y) = p_t(x)$, leaving

$$p_{t+\Delta t}(x) = p_t(x) + \Delta t \sum_y R_t(x, y)\, p_t(y) + o(\Delta t). \tag{B.4}$$

Subtract $p_t(x)$ and divide through by Δt:

$$\frac{p_{t+\Delta t}(x) - p_t(x)}{\Delta t} = \sum_y R_t(x, y)\, p_t(y) + \frac{o(\Delta t)}{\Delta t}. \tag{B.5}$$

Taking $\Delta t \to 0$, the left-hand side is by definition $\partial_t p_t(x)$ and the remainder $o(\Delta t)/\Delta t \to 0$, which yields the Kolmogorov forward equation

$$\partial_t p_t(x) = \sum_y R_t(x, y)\, p_t(y). \tag{B.6}$$

Finally, separating the diagonal from the off-diagonal terms and using $R_t(x, x) = -\sum_{y \neq x} R_t(y, x)$ recovers the gain-loss form referred to in Section 2.2.2,

$$\partial_t p_t(x) = \sum_{y \neq x} \big[R_t(x, y)\, p_t(y) - R_t(y, x)\, p_t(x)\big], \tag{B.7}$$

that is, the rate of probability flowing into x from every other state minus the rate flowing out.

Dividing by $p_t(x)$ and using $\partial_t \log p_t(x) = \partial_t p_t(x)/p_t(x)$ gives the log form (2.8),

$$\partial_t \log p_t(x) = \sum_{y \neq x} R_t(x, y)\, \frac{p_t(y)}{p_t(x)} - \sum_{y \neq x} R_t(y, x), \tag{B.8}$$

in which the normaliser of a Boltzmann path cancels inside the ratio.

C Discrete Stein Control Variate

The identities below use exact density ratios and continuous-time dynamics. Active ratio clipping can break Stein cancellation; time discretisation also perturbs the path-weight identities.

The control variate exists to stabilise the estimate of $\partial_t \log Z_t$, so we first record the identity it estimates. Using $Z_t = \sum_x \exp[-E_t(x)]$ and differentiating,

$$\begin{aligned} \partial_t \log Z_t &= \frac{1}{Z_t} \partial_t \sum_x \exp[-E_t(x)] \\ &= -\frac{1}{Z_t} \sum_x \exp[-E_t(x)]\, \partial_t E_t(x) = -\mathbb{E}_{p_t(x)}[\partial_t E_t(x)]\,, \end{aligned} \tag{C.1}$$

which is the expectation whose naive Monte Carlo estimate Section 3.1.2 shows is too noisy to enter the squared loss. Recall the Stein quantity (3.4) from Section 3.1.2,

$$\xi_t(x; R_t^\theta) = \sum_{y \neq x} \left[R_t^\theta(y,x) - R_t^\theta(x,y)\, \frac{p_t(y)}{p_t(x)} \right]. \tag{C.2}$$

We show it has zero mean under p_t, which is what makes it a valid control variate. Taking the expectation and clearing the $p_t(x)$ in the denominator,

$$\begin{aligned} \mathbb{E}_{p_t(x)}\Big[\xi_t(x; R_t^\theta)\Big] &= \sum_x p_t(x) \sum_{y \neq x} \left[R_t^\theta(y,x) - R_t^\theta(x,y)\, \frac{p_t(y)}{p_t(x)} \right] \\ &= \sum_x \sum_{y \neq x} \Big[R_t^\theta(y,x)\, p_t(x) - R_t^\theta(x,y)\, p_t(y) \Big]. \end{aligned} \tag{C.3}$$

The two contributions in the double sum are exchanged by relabelling the dummy indices $x \leftrightarrow y$: in the second term,

$$\sum_x \sum_{y \neq x} R_t^\theta(x,y)\, p_t(y) = \sum_y \sum_{x \neq y} R_t^\theta(y,x)\, p_t(x), \tag{C.4}$$

which is identical to the first. The two therefore cancel, giving

$$\mathbb{E}_{p_t(x)}\Big[\xi_t(x; R_t^\theta)\Big] = 0 \tag{C.5}$$

where $p_t > 0$. This proves ξ_t is a valid control variate with known (zero) mean. Moreover, at optimality the Kolmogorov residual vanishes, $\delta_t(x; R_t^\theta) = 0$, so that

$$\xi_t(x; R_t^\theta) = -\partial_t \log p_t(x) = \partial_t E_t(x) + \partial_t \log Z_t \tag{C.6}$$

differs from the target summand $\partial_t E_t(x)$ only by the x-independent constant $\partial_t \log Z_t$. With an appropriately chosen coefficient the control variate then removes the estimator's fluctuation entirely, which is the zero-variance-at-optimality property quoted in Section 3.1.2 [29].

C.1 Weight moments and population ESS

For independent importance draws with finite first and second weight moments, the normalised statistic (2.3) converges to $\mathbb{E}_q[W]^2/\mathbb{E}_q[W^2]$ [48]. With $W = \pi/q$, π normalised and $q > 0$ on its support, $\mathbb{E}_q[W] = 1$ and

$$\mathrm{ESS}^{\mathrm{pop}} = \frac{1}{1 + \chi^2(\pi \| q)}, \qquad \chi^2(\pi \| q) := \mathbb{E}_q[(\pi/q - 1)^2]. \tag{C.7}$$

If $\log W \sim \mathcal{N}(m, v)$, then $\mathbb{E}[W] = e^{m+v/2}$ and $\mathbb{E}[W^2] = e^{2m+2v}$, so the same ratio evaluates to

$$\mathrm{ESS} \xrightarrow[K\to\infty]{} \frac{\mathbb{E}[W]^2}{\mathbb{E}[W^2]} = e^{-v} = e^{-\operatorname{Var}[\log W]}. \tag{C.8}$$

Approximate Gaussianity gives only an approximation to this last equality. The second moment governs importance-sampling requirements under the assumptions in Agapiou et al. [50]; Chatterjee and Diaconis [102] obtain a related criterion through $\exp[\mathrm{KL}(\pi\|q)]$. Path weights use the same moment ratio on path space, with the marginal relation needed below following by conditional expectation.

C.2 Control variate behaviour under the rollout samples

Training averages over the model's samples, with distribution q_t^θ, whereas (3.5) assumes target samples from p_t. Keep the body's $a_t^\theta = -\partial_t E_t + \xi_t$ and write $\delta_t = a_t^\theta - \partial_t \log Z_t$, suppressing its rate-matrix argument. Stein gives $\mathbb{E}_{p_t}[\delta_t] = 0$ for any admissible generator; $\delta_t \equiv 0$ is the stronger, pointwise Kolmogorov condition. For the covariance identity assume $q_t^\theta > 0$ wherever $p_t > 0$. Averaging under q_t^θ instead,

$$\mathbb{E}_{q_t^\theta}\left[a_t^\theta\right] - \partial_t \log Z_t \;=\; \sum_x \big(q_t^\theta(x) - p_t(x)\big)\,\delta_t(x) \;=\; -\operatorname{Cov}_{q_t^\theta}(w_t, \delta_t)\,, \qquad w_t := \frac{p_t}{q_t^\theta}, \tag{C.9}$$

using $\mathbb{E}_{q_t^\theta}[w_t\delta_t] = \mathbb{E}_{p_t}[\delta_t] = 0$ and $\mathbb{E}_{q_t^\theta}[w_t] = 1$. The estimate is therefore unbiased when $q_t^\theta = p_t$, and unbiased for every q_t^θ when the residual vanishes; otherwise its possible bias is bounded by Cauchy–Schwarz as

$$\left|\mathbb{E}_{q_t^\theta}\left[a_t^\theta\right] - \partial_t \log Z_t\right| \;\le\; \sqrt{\chi^2\big(p_t \,\|\, q_t^\theta\big)}\;\operatorname{sd}_{q_t^\theta}[\delta_t] \;\le\; \sqrt{\frac{1}{\mathrm{ESS}_t^{\mathrm{pop}}} - 1}\;\operatorname{sd}_{q_t^\theta}[\delta_t]\,, \tag{C.10}$$

Here W_t is the path weight of (3.10) stopped at t, $Q_{[0,t]}$ is the distribution of trajectories produced by the model from p_0, and $\bar{W}_t = W_t/\mathbb{E}_Q[W_t]$. The population fraction is $\mathrm{ESS}_t^{\mathrm{pop}} = 1/\mathbb{E}_Q[\bar{W}_t^2]$, assuming a finite second moment. Reweighting $Q_{[0,t]}$ by $\bar{W}_t$ gives terminal marginal p_t; hence $\mathbb{E}_Q[\bar{W}_t \mid X_t] = p_t(X_t)/q_t^\theta(X_t)$, and conditional Jensen gives the second inequality. ESS computed at $t = 1$ does not generally bound mismatch at an earlier time. The bound combines distribution mismatch and residual variation; it guarantees neither decreasing bias nor convergence. Finite empirical ESS can miss unvisited modes and can't replace the population quantity as a quality metric.

Write the population bias as $b_t = \mathbb{E}_{q_t}[a_t^\theta] - \partial_t \log Z_t$. Since $a_t^\theta - \partial_t \log Z_t = \delta_t$, ideal centring at $c_t = \mathbb{E}_{q_t}[a_t^\theta]$ gives

$$\mathbb{E}_{q_t}[(a_t^\theta - c_t)^2] = \operatorname{Var}_{q_t}(a_t^\theta) = \mathbb{E}_{q_t}[\delta_t^2] - b_t^2. \tag{C.11}$$

The implemented inner updates use a finite, frozen estimate and replay, so this is an ideal population identity. On a covering support, zero variance makes a_t^θ constant; its p_t expectation fixes that constant at $\partial_t \log Z_t$, forcing zero residual. An arbitrary stationary point need not have this property.

C.3 Reference uncertainty and sampling floors

Let $\widehat{\pi}$ be the full reference pool and D one of the reported discrepancies. The sampling floor is estimated as $B^{-1}\sum_{b=1}^{B} D(\widehat{\pi}_N^{(b)}, \widehat{\pi})$, with N frames drawn independently with replacement for each replicate. Enumerable references instead supply exact target draws and exact probabilities

in the second argument. Table floors and the baseline, soft and hard marginal-caption floors use $B = 200$; $N = 5000$ except the hard 4×4 comparison, which uses 512.

Reference uncertainty uses independent chains as the sampling units: partition the chains into two disjoint, as nearly equal as possible groups, pool each group, and compute $D(\widehat{\pi}_A, \widehat{\pi}_B)/2$, averaged over 64 seeded splits. For equally sized independent halves and a regular root-N statistic, the difference has twice the full-pool noise scale. Applying that factor to nonlinear distances is only an approximation, especially for Wasserstein distance on discrete support, connected correlations and unequal groups. We therefore label these parenthesised values approximate uncertainty rather than calibrated standard errors or confidence intervals.

The baseline pool has 100 chains with 50 retained frames each; the soft pools have four chains with 6668 or 66668 frames each. Four chains give only three distinct balanced partitions, so repeated splits do not create additional independent evidence. Chain splitting preserves serial correlation within each chain, but cannot diagnose bias shared by all chains. Independently resampling an empirical pool measures sampling noise conditional on it; it does not validate that pool as ground truth. Neither being below the mean floor nor overlapping a reference-uncertainty diagnostic constitutes a significance test.

C.4 Transfer to the Swap Generator

The Stein identity is move set agnostic. Let R_t^θ now be the swap generator – off-diagonal entries non-zero only between states of $\mathcal{C}$ differing by one unlike-pair swap – and let p_t be the on-slice path (5.3), strictly positive on $\mathcal{C}$. The Stein quantity is (3.4) with both state and neighbour restricted to $\mathcal{C}$, and its expectation is the same double sum as before,

$$\mathbb{E}_{p_t}\Big[\xi_t(x; R_t^\theta)\Big] = \sum_{x\in\mathcal{C}} \sum_{\substack{y\in\mathcal{C} \\ y\neq x}} \Big[\, R_t^\theta(y,x)\, p_t(x) - R_t^\theta(x,y)\, p_t(y) \,\Big] = 0, \tag{C.12}$$

by the identical relabelling $x \leftrightarrow y$. Closure on $\mathcal{C}$ ensures that no omitted term reaches off the slice. Zero variance at optimality follows by the same two lines as before: when the on-slice residual vanishes, $\xi_t(x) = -\partial_t \log p_t(x)$ differs from the target summand by the x-independent $\partial_t \log Z_t^{\mathcal{C}}$, with $Z_t^{\mathcal{C}} = \sum_{x\in\mathcal{C}} \tilde{p}_t(x)$ the on-slice normaliser.

The importance weight's dropped term is still a constant. The path-integral weight (3.10) is computed from the unnormalised path $\tilde{p}_s$ and, per the construction of Ou et al. [29], its integrand is the residual up to terms that are x-independent and therefore common to every trajectory, cancelling in the self-normalised weights. Three facts make the same bookkeeping go through on the slice. First, trajectories never leave $\mathcal{C}$ (closure again), so the indicator in π_{hard} is 1 at every state visited and never enters a ratio. Second, the normalising gap between the computable and the normalised path,

$$\partial_s \log \tilde{p}_s(x) - \partial_s \log p_s(x) = \partial_s \log Z_s^{\mathcal{C}}, \tag{C.13}$$

depends on s only, exactly as $\partial_s \log Z_s$ did on the full cube. Third, the base contribution inside $\partial_s \log \tilde{p}_s(x) = \log \tilde{\pi}(x) - \log \eta(x)$ is the constant (5.2), so $\log \eta$ joins the same x-independent bucket – the role the uniform base's $(1/2)^d$ played in the unconstrained weight, now played by $\binom{d}{N_A}^{-1}$. Every trajectory's log-weight is therefore shifted by one common function of time, the shift cancels in $w^{(k)}/\sum_l w^{(l)}$, and the ESS keeps its unconstrained reading: maximal when the residual vanishes on $\mathcal{C}$. The converse need not hold at a single terminal time: nonzero residuals can integrate to the same value along every trajectory. These statements and the cancellation of slice-dependent constants apply within each fixed composition.

D Training Algorithms

Algorithm 1 Shared DNFS training loop, following Algorithm 1 of Ou et al. [29]. Algorithms 2 and 3 specify the constrained variants.

Require: target path $\tilde{p}_t$, base η, time grid $0 = t_0 < \cdots < t_T = 1$, outer batch M, inner batch N, inner updates I, replay depth B, optimiser and simulation settings

for each outer cycle **do**
  apply any target and learning-rate stage change; clear replay when the target changes
  draw M base states and simulate all grid times under the current rates ▷ stop-gradient
  $c_{t_k} \leftarrow M^{-1} \sum_m a^\theta_{t_k}(x^{(m)}_{t_k})$ for every k ▷ Eq. (3.5), detached
  append $(x^{(m)}_{t_k}, k)$ to replay; retain the last B cycles
  for I inner updates **do**
    sample N replay rows uniformly; look up each row's c_{t_k} in the latest grid
    minimise $N^{-1} \sum (a^\theta_{t_k}(x) - c_{t_k})^2$, holding states and c_{t_k} fixed
    clip the gradient norm, take a scheduled AdamW step, and update the parameter EMA if enabled
  end for
end for

Algorithm 2 Matched-base soft amortisation: changes to Algorithm 1. The penalty λ and finite composition set $\mathcal{V}$ are fixed.

draw one $c \sim \mathcal{U}(\mathcal{V})$ per outer cycle; bind the target and model to c
use the product base $\eta_c = \text{Bernoulli}(c)^{\otimes d}$ ▷ $\mathbb{E}_{\eta_c}[n_+/d] = c$; the count fluctuates
roll out at this c and estimate c_{t_k} within this cycle's single-composition batch
store (x, k, c, c_{t_k}) in replay; inner batches use each row's stored composition and baseline
evaluate that row's penalty, base density and conditioned rates before the shared loss update

Algorithm 3 Hard swap training, including composition amortisation: changes to Algorithm 1.

choose a count N_A per trajectory: fixed for specialists, uniform over the declared set for amortisation
draw uniformly on that count's slice; use $\log \eta(x) = -\log \binom{d}{n_+(x)}$ for its conditional base
simulate with one-event or matching swap steps (Section D.1)
use swap scores $[G]_+$ and $[-G]_+$ for forward and reverse rates; clamp neighbour log-ratios at 30
group rollout rows by conserved count: $\mathcal{I}_n = \{m : n_+(x^{(m)}_0) = n\}$
for each time t_k and represented count n, set the detached baseline

$$c^{(n)}_{t_k} \leftarrow |\mathcal{I}_n|^{-1} \sum_{m \in \mathcal{I}_n} a^\theta_{t_k}(x^{(m)}_{t_k})$$

▷ swap integrand
if a declared count has no rollout rows, use the whole-batch slot mean for that count
append (x, k) to replay; retain the last B cycles
in each inner batch, read $n = n_+(x)$ and look up $c^{(n)}_{t_k}$ in the latest grid
use (5.5) with $\partial_t \log Z_t$ replaced by this detached baseline; apply the shared updates

Here a^θ_t is the control-variate integrand defined in Section 3.1.2; the naive-estimator ablations replace it with $\partial_t \log \tilde{p}_t$. Specialist soft runs use their declared base and the shared loop. A singleton $\mathcal{V}$ gives the same target family, but retains the amortised trainer's stored-baseline replay convention.

The hard amortised model reads composition from the state, without an extra conditioning input. Swaps preserve each slice's probability mass, so the residual on $\mathcal{C}_n = \{x : n_+(x) = n\}$ needs its own $\partial_t \log Z^{\mathcal{C}_n}_t$. The Stein identity (C.12) therefore gives $\mathbb{E}_{p_t(\cdot|\mathcal{C}_n)}[a^\theta_t] = \partial_t \log Z^{\mathcal{C}_n}_t$ separately on each slice. Algorithm 3 estimates these derivatives within each count group, whether the integrand is reused from the rollout or recomputed. Using the model's samples can still introduce the bias described in Appendix C.2 within each slice.

D.1 Swap simulation details

One-event update. For outgoing rates q_{ij} and $\Lambda = \sum_{i<j} q_{ij}$, the one-event step uses probabilities $q_{ij}\,\mathrm{d}t$ for each swap and $1 - \Lambda\,\mathrm{d}t$ for staying put when $\Lambda\,\mathrm{d}t \leq 1$. The implementation clips each pair probability at one and the stay probability at zero, then normalises the categorical distribution. If $\Lambda\,\mathrm{d}t > 1$, it therefore forces one event, changing the intended step probabilities.

Matching update. The matching step proposes pairs independently with probability $\min\{q_{ij}\,\mathrm{d}t, 1\}$, using rates at the start of the step. It assigns random priorities, accepts pairs with the highest priority at both endpoints, removes proposals touching accepted sites, and repeats for up to eight rounds. All retained swaps are applied together. The round limit can leave compatible proposals unused, so the result need not be maximal.

Discretisation error. Both updates are first-order approximations to the continuous-time process. At fixed lattice size, with bounded rates sufficiently regular in time, their local transition error is $\mathcal{O}(\mathrm{d}t^2)$ as $\mathrm{d}t \to 0$. For matching, conflicting proposals occur with probability $\mathcal{O}(\mathrm{d}t^2)$; retained swaps commute but their rates remain frozen during the step.

E Proof of Soft Constraint Variance

The soft constraint adds an extensive composition penalty to the Ising energy, so the (unnormalised) target becomes

$$\log \tilde{\pi}(x) = x^T J x - \lambda\, d\,\big(c(x) - c_{target}\big)^2, \tag{E.1}$$

with $c(x)$ the composition fraction, d the number of lattice sites, and λ the penalty strength (Section 4.1). DNFS anneals from a uniform base η along a path that is linear in the log-density, $\log \tilde{p}_t(x) = (1-t)\log \eta(x) + t \log \tilde{\pi}(x)$, so the time-derivative that drives the Kolmogorov residual is the t-independent endpoint difference

$$\begin{aligned} \partial_t \log \tilde{p}_t(x) &= \log \tilde{\pi}(x) - \log \eta(x) \\ &= \underbrace{x^T J x + d \log 2}_{=:\, I(x)} \;-\; \lambda\, \underbrace{d\,\big(c(x) - c_{target}\big)^2}_{=:\, P(x)}, \end{aligned} \tag{E.2}$$

which is exactly affine in λ: an Ising piece $I(x)$ and a penalty piece $P(x)$ (here $\log \eta \equiv -d \log 2$ for the uniform base on $\{-1,+1\}^d$). This decomposition is also true of the matched base of Section 4.3.2 unchanged: a product-Bernoulli $\log \eta_c(x)$ is affine in $c(x)$ and λ-independent, so it joins $I(x)$ and the penalty piece – and its explicit coefficient λ – is untouched.

Substituting into the residual (3.1) $\delta_t(x; R_t^\theta) = \partial_t \log p_t(x) + \xi_t(x; R_t^\theta)$ of Section 3.1.1, and using $\partial_t \log p_t = \partial_t \log \tilde{p}_t - \partial_t \log Z_t$ with $\partial_t \log Z_t = \mathbb{E}_{p_t}[\partial_t \log \tilde{p}_t] = \mathbb{E}[I] - \lambda\, \mathbb{E}[P]$,

$$\begin{aligned} \delta_t(x) &= \big(I(x) - \mathbb{E}[I] + \xi_t(x; R_t^\theta)\big) - \lambda\,\big(P(x) - \mathbb{E}[P]\big) \\ &=: \; \delta_I(x) - \lambda\, \delta_P(x), \end{aligned} \tag{E.3}$$

the decomposition quoted in Section 4.3, where $\delta_P(x) = P(x) - \mathbb{E}_{p_t}[P]$ is the centred penalty and δ_I collects the Ising residual and the rate-matrix term ξ_t.

For each fixed λ, the variance under any chosen reference law obeys

$$\mathrm{Var}[\delta_t] = \mathrm{Var}[\delta_I] + \lambda^2\, \mathrm{Var}[\delta_P] - 2\lambda\, \mathrm{Cov}[\delta_I, \delta_P]. \tag{E.4}$$

The additive constants $\mathbb{E}[I], \mathbb{E}[P]$ drop out of the (co)variances, so the penalty contributes the term $\lambda^2 \,\mathrm{Var}[\delta_P]$ with

$$\mathrm{Var}[\delta_P] = \mathrm{Var}[P(x)] = d^2\, \mathrm{Var}\big[(c(x) - c_{target})^2\big]. \tag{E.5}$$

Under the uniform base the up-count is Binomial$(d, \frac{1}{2})$, so with $\mu = \frac{1}{2} - c_{target}$

$$\mathrm{Var}_\eta[\delta_P] = \mu^2 d + \tfrac{1}{8}\big(1 - \tfrac{1}{d}\big), \tag{E.6}$$

bounded as d grows at the centre and linear in d at a fixed off-centre composition. At $\lambda = 50$, $d = 64$, the explicit contribution $\lambda^2 \,\mathrm{Var}_\eta[P]$ is 308 at the centre and 10308 at $c_{target} = 0.25$, falling to 175 under the matched base. For that base, writing $v = c_{target}(1 - c_{target})$, the exact coefficient is $\mathrm{Var}_{\eta_c}[P] = 2v^2 + v(1-6v)/d$.

For comparison, if $Y = c - c_{target} \sim \mathcal{N}(0, s^2)$ with $s^2 = 1/(2\lambda d)$, then $\mathrm{Var}(Y^2) = 2s^4$, giving $\lambda^2 \,\mathrm{Var}_\pi[P] \approx \lambda^2 d^2 (2s^4) = 1/2$. This one-half is a variance contribution under an approximate target law, not the composition $c_{target} = 1/2$. This requires a penalty-dominated envelope spanning many counts; interactions can change its centre and curvature. At the headline $d = 64$, $\lambda = 50$, the envelope width 0.0125 is smaller than the count spacing $1/64$, so the many-count

assumption is weak. At fixed finite d, the limit $\lambda \to \infty$ instead concentrates on minimum-penalty counts and this variance contribution tends to zero.

The explicit penalty contribution grows quadratically under a fixed reference distribution, explaining its large initial scale. This does not prove quadratic growth of the full residual variance: δ_I, its covariance with P, and the distribution of the model's samples can all depend on λ, including through the density ratios in ξ_t.

For an attainable count $dc_{target} \in \mathbb{Z}$, penalisation approaches the hard target as $\lambda \to \infty$. Structural enforcement avoids the explicit penalty term, without requiring a universal variance blow-up argument.

E.1 The flip response in closed form

Flipping site i sends $x_i \to -x_i$. In the Ising term $x^T Jx = \sigma \sum_{jk} A_{jk} x_j x_k$ every bond is counted twice, so the terms containing x_i are $2\sigma x_i h_i$ with $h_i = \sum_j A_{ij} x_j$, and the flip changes them by $-4\sigma x_i h_i$. The flip changes the up-count by $-x_i$, so $c \to c - x_i/d$ and the penalty changes by

$$-\lambda d\big[(c - x_i/d - c_{target})^2 - (c - c_{target})^2\big] = 2\lambda x_i\,(c - c_{target}) - \lambda/d, \tag{E.7}$$

using $x_i^2 = 1$. Substituting $c = c_{\setminus i} + (1 + x_i)/(2d)$, with $c_{\setminus i}$ the composition excluding site i, turns the constant $-\lambda/d$ into $+\lambda x_i/d$ and gives (4.5). Neither h_i nor $c_{\setminus i}$ depends on x_i, so flipping x_i changes only the sign of the response. A sign-odd response is exactly what the locally equivariant model can represent, which is why the channel fits inside it.

E.2 The matched base on the geometric path

Write $\tilde{\rho}$ for the unnormalised target of (4.1) and η for the base, so the path is $\log \tilde{p}_t = (1 - t)\log \eta + t \log \tilde{\rho}$. Its time derivative is the same function at every t,

$$\partial_t \log \tilde{p}_t(x) \;=\; \log \tilde{\rho}(x) - \log \eta(x) \;=:\; g(x), \tag{E.8}$$

and g is what every trajectory weight integrates. Three identities follow. First, the integrand of (3.10) is the Kolmogorov residual δ_t up to the state-independent $\zeta_t := \partial_t \log Z_t$, so

$$\log w \;=\; \log \frac{Z_1}{Z_0} \;+\; \int_0^1 \delta_t(x_t)\,dt \qquad \Longrightarrow \qquad \mathrm{Var}[\log w] \;=\; \mathrm{Var}\Big[\int_0^1 \delta_t(x_t)\,dt\Big]\,: \tag{E.9}$$

a perfectly trained rate matrix gives every trajectory the same weight under *any* base, provided x_0 is drawn from that base, since the accumulator starts at zero. Second, $\tilde{p}_t(x) \propto \eta(x)\, e^{tg(x)}$ is an exponential family in t with sufficient statistic g, so

$$\partial_t \zeta_t = \mathrm{Var}_{p_t}[g] \geq 0, \qquad \zeta_1 - \zeta_0 = \int_0^1 \mathrm{Var}_{p_t}[g]\,dt, \tag{E.10}$$

so the total rise of ζ_t is the integrated variance of g. Third, since $\zeta_1 - \zeta_0 = \mathbb{E}_\pi[g] - \mathbb{E}_\eta[g]$, that integral is the Jeffreys divergence $\mathrm{KL}(\pi \,\|\, \eta) + \mathrm{KL}(\eta \,\|\, \pi)$, whose $\mathrm{KL}(\eta \,\|\, \pi)$ term contains the expected penalty $\lambda \mathbb{E}_\eta[P]$: $\lambda d[(\frac{1}{2} - c_{target})^2 + 1/(4d)]$ under the uniform base against $\lambda c_{target}(1 - c_{target})$ under the matched base, the two values the chapter quotes after (4.6). This is one contribution to the divergence, not the whole of it, so the identity alone does not establish a lower training cost.

F Supplementary Soft Results

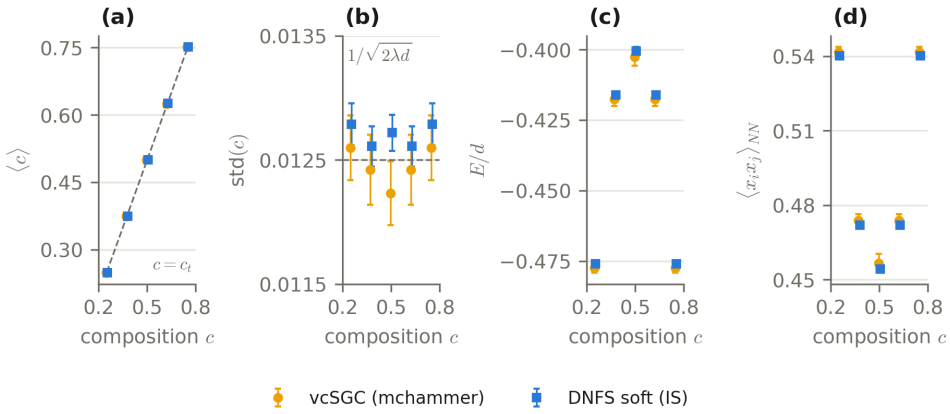


Figure F.1. The soft sampler against a `mchammer VCSGCEnsemble` at matched $\kappa = \lambda = 50$, across the composition windows at the critical coupling (8×8, the matched-base house recipe, every seed reported; at $\sigma = 0.1$ the same comparison agrees to a few 10^{-4} throughout). Importance-weighted DNFS estimates (blue squares) against the VCSGC reference (orange circles): (a) mean composition, (b) composition spread, (c) energy per site, (d) nearest-neighbour short-range order $\langle x_i x_j \rangle$, which with a single pair term and no field is an affine function of the energy and so repeats (c) rather than checking it independently. DNFS error bars are bootstrapped over the importance weights, the VCSGC bar is the spread over chains; the two series are offset by ± 0.004 in c for visibility, since at matched windows they otherwise overprint.

training at σ_c	draw set	ESS raw	ESS EMA
specialist, cold start	{0.50}	0.71 ± 0.03	0.81 ± 0.02
specialist, paired initialisation	{0.50}	0.68 ± 0.05	0.81 ± 0.02
conditioned, cold start	seventeen values	0.002 ± 0.001	0.005 ± 0.002
conditioned, cold start	three values	0.005 ± 0.004	0.003 ± 0.001
conditioned, paired initialisation	{0.50}	0.005 ± 0.003	0.005 ± 0.005
conditioned, request-dependent gain	three values	0.005 ± 0.002	0.006 ± 0.006
specialist, σ ladder	{0.50}	0.76 ± 0.02	0.84 ± 0.02
conditioned, σ ladder	seventeen values	0.24 ± 0.10	0.32 ± 0.10

Table F.1. Conditioning and curriculum controls at σ_c: 8×8, $\lambda = 50$, house recipe, ESS fraction at the centre request $c = 0.50$ from the frozen evaluation (the house table's σ-ladder cell reads the same model from its sweep frames, hence 0.25 there), mean $\pm$ SD over seeds 42–45. Paired initialisation draws the conditioner's parameters from a private random stream, so the conditioned model and the specialist share every trunk tensor at step zero. The request-dependent gain adds two scalars to the exact-field channel's time-gain, both of which stay at zero.

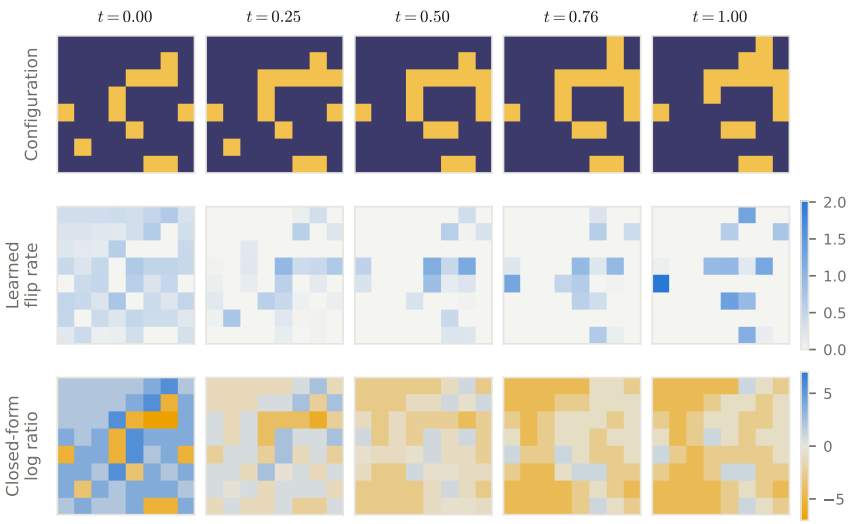


Figure F.2. One raw 8×8 rollout of the matched-base soft sampler at σ_c, $\lambda = 50$, $c_{\text{target}} = 0.25$ (raw parameters, 128 grid points). Top: configurations at five sampler times, gold spin $+1$, indigo -1. Middle: the learned flip rate at every site. Bottom: the closed-form response $\Delta_i(x)$ of (4.5); positive (blue) means the terminal target favours flipping that site. The draw starts four up-spins below target, so at $t = 0$ nearly every down-spin flip is favoured; once the count reaches target the same lattice turns against them. Each field uses a fixed colour scale across columns.

F.1 Canonical free energy from soft samples

The soft sampler estimates a penalised normaliser, whereas thermodynamic integration supplies the canonical free energy at fixed composition. On the requested slice the penalty vanishes, giving

$$Z_{\text{can}}(c_*) = Z_\lambda \, \pi_\lambda(c = c_*) = \mathbb{E}_Q[w \, \mathbf{1}\{c(x_1) = c_*\}].$$

The sample estimate averages over all trajectories, including the rejected ones. This is the normaliser version of Section 4.6.2; no continuum correction is required. Figure F.3 compares this estimate with thermodynamic integration. The soft and hard canonical estimates are compared directly in Figure 5.11.

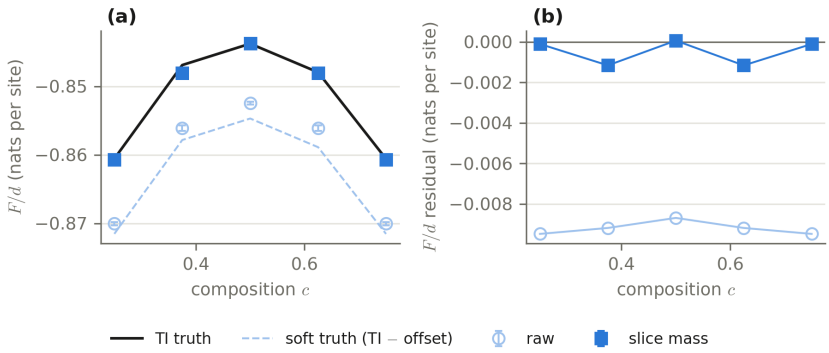


Figure F.3. Soft-to-canonical free energy at 8×8, σ_c and $\lambda = 50$, using the matched-base specialists. (a) Thermodynamic integration (ink), the soft sampler's mean-log-weight estimate $-\overline{\log w}/d$ (open circles), and the canonical slice estimate $-\log[N^{-1} \sum_{i=1}^{N} w_i \mathbf{1}\{c(x_i) = c_*\}]/d$ (squares). The dashed Gaussian-offset curve is an approximate guide for the penalised ensemble; it is not used by the slice estimator. (b) Residuals relative to thermodynamic integration after Richardson extrapolation on paired $n_e = 256/512$ grids. The canonical residual is within 0.0012 nats/site at the tested windows. Reflected points use Z_2 symmetry.

G Budget-Masked Reference Conditionals

A masked sampler can enforce a terminal composition by restricting each reveal to the remaining species budget. With m masked sites and b unassigned A-atoms, our reference chooses a masked site uniformly, then assigns A with probability b/m and B with probability $(m-b)/m$. For target count N_A and any reveal order, the assignment probabilities multiply to

$$\prod_{\text{steps}} \big(\text{assignment conditional}\big) = \frac{N_A!\,(d-N_A)!}{d!} = \binom{d}{N_A}^{-1}. \tag{G.1}$$

The denominators multiply to $d!$, and the A and B numerators to $N_A!$ and $(d-N_A)!$. Summing over the $d!$ equally likely reveal orders gives a uniform terminal law on the composition slice. This constant reference contribution cancels in self-normalised trajectory weights; the learned assignment probabilities must still enter those weights. JANUS uses this budgeted reveal for its fixed-composition rungs, setting its conditioning to the rung composition and reading $G(n)$ from the resulting path weights [33]. The initial start we took here predated the publication of that paper.

We adapt MDNS's conditional preconditioner [31] to the budget using the approximate Ising logit difference

$$\ell_i^{(0)} = \log\frac{b}{m-b} + 4\sigma f_i, \qquad 0 < b < m.$$

Here f_i sums neighbour spins, imputing masked neighbours at $(2b-m)/m$. At $\sigma = 0$ this is the exact urn conditional. Exhausted budgets force the remaining species. The energy-dependent term is an approximation.

Table G.1. MDNS-style MLP probes at $\sigma = 0.223$: ESS fractions, min–max over seeds 42–44, after 10k log-variance updates and 65,536 evaluation trajectories per seed. Constrained rows use $c = 0.5$ and the budget preconditioner. EMA decay is 0.9999, with warm-up only at 8×8. The MLP replaces the paper's vision transformer (ViT).

Lattice	Target	Raw ESS	EMA ESS
4×4	Unconstrained control	0.977–0.982	0.493–0.507
4×4	Fixed composition	0.853–0.857	0.521–0.527
8×8	Fixed composition	0.137–0.157	0.153–0.162

The constrained runs record no composition violations. At 8×8, EMA ESS misses the pre-set 0.5 threshold, although the weighted energy-marginal TV is 0.014–0.016 against a chain reference. Feasibility transfers; efficient sampling at scale remains unresolved.

This construction avoids swap enumeration, using d reveals with at most $2m$ outgoing assignments. For DNFS, it would also change the path through complete configurations. Applying its Kolmogorov residual (3.1) to the natural masked path requires probabilities of partial states, which sum Boltzmann weights over feasible completions. These sums are generally intractable: a different tractable path or estimator would be needed. The probes here instead use MDNS's path-measure objective.

H Training Curves

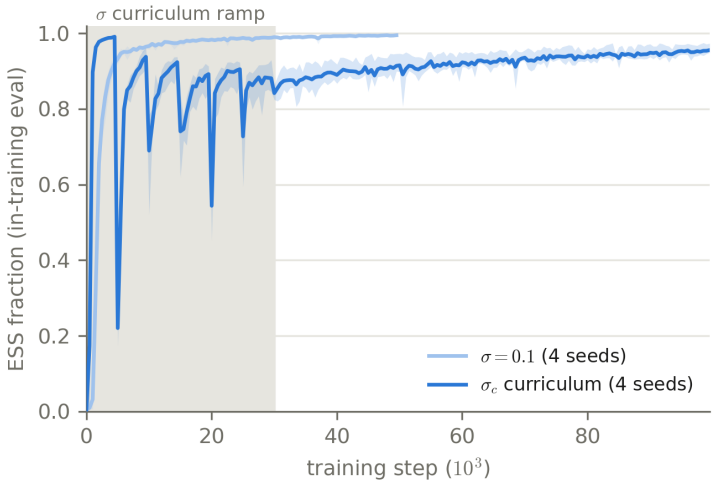


Figure H.1. Unconstrained 10×10 training: $\sigma = 0.1$ (light, 50k steps) and the σ_c curriculum (dark, 100k), four seeds each, mean and min–max band. During the shaded curriculum ramp the in-training ESS is evaluated against the current coupling; the critical family's in-training ESS uses 512 draws. Final evaluations appear in Table 3.2.

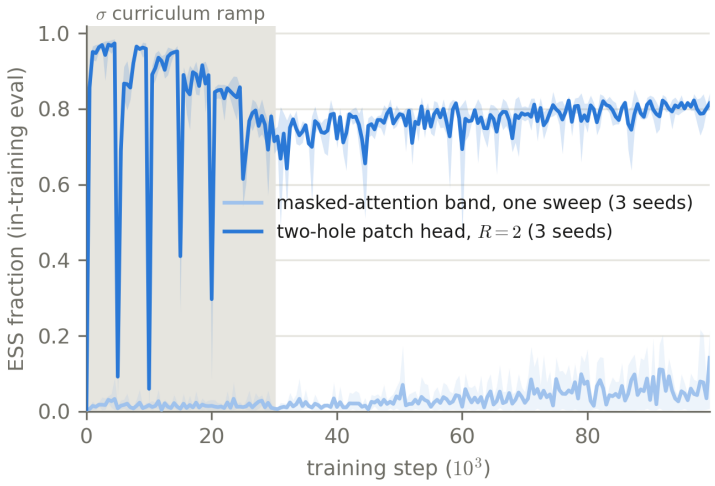


Figure H.2. Hard 16×16 training at $c = 0.5$: two-hole patch head, $R = 2$ (dark), and masked-attention band, one sweep (light), three seeds each, mean and min–max band over 100k steps. The shaded ramp changes the diagnostic's target coupling. The patch head sustains higher in-training ESS; these diagnostics differ from the EMA evaluations in Table K.4.

I Validation at the Enumerable Size

Figure I.1 compares unweighted configuration counts with exact-draw fluctuations. Soft-sampler TV exceeds the reference at both couplings; hard-sampler TV is within it at $\sigma = 0.1$ and modestly above at σ_c. Finite-grid error remains included.

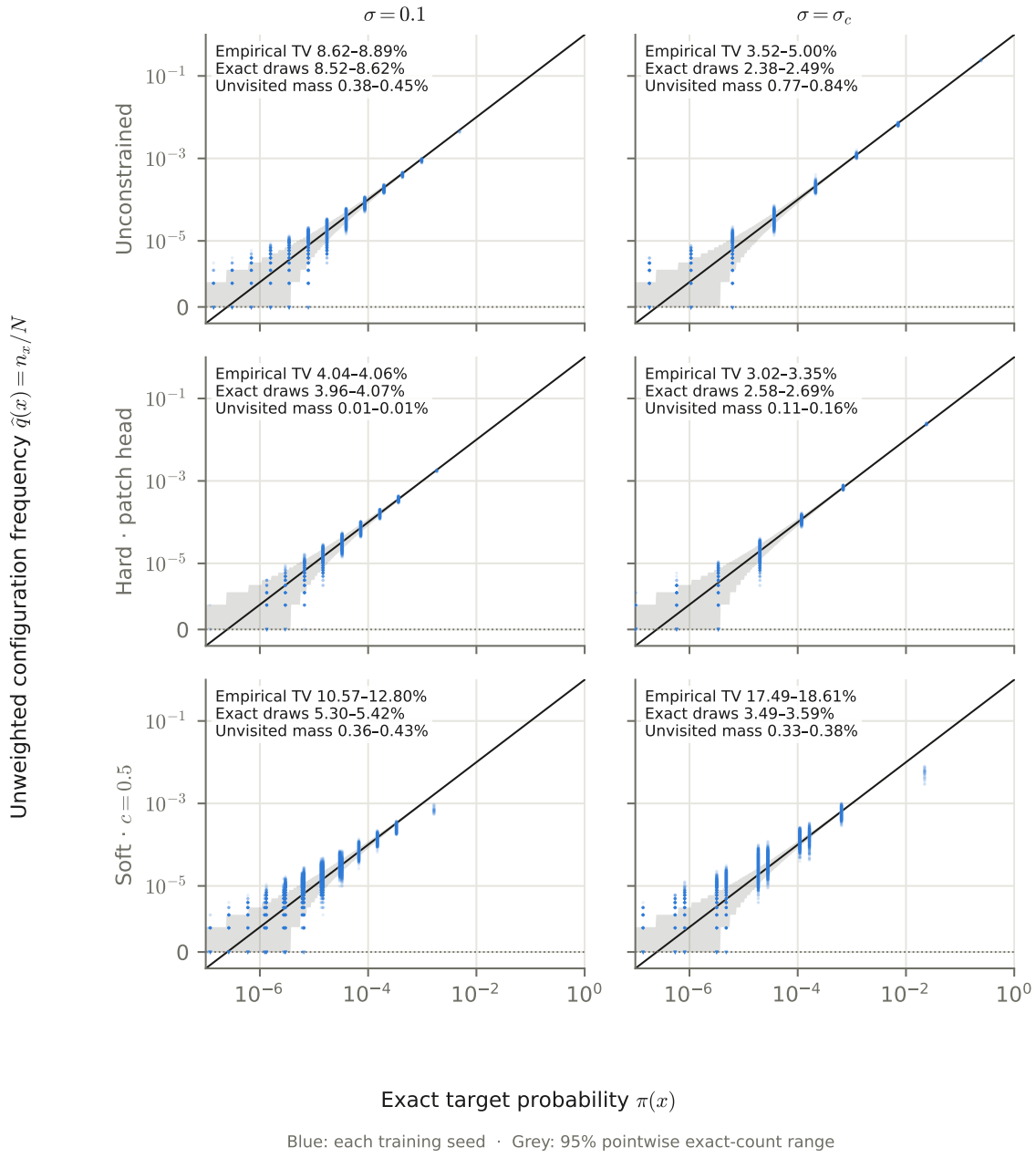


Figure I.1. Configuration-frequency calibration at 4×4, using 10^6 fresh draws per raw final checkpoint. Each blue point is one configuration in one training seed; seeds are not pooled. Unconstrained and soft runs use seeds 42–45; the hard two-hole patch uses 42–44. Soft targets have $c_{\text{target}} = 0.5$ and $\lambda = 50$; hard targets fix $n_+ = 8$. Each run retains its recorded Euler grid. The diagonal is $\hat{q} = \pi$; grey shows pointwise 95% binomial count intervals for exact sampling at the same N, not a simultaneous band. Unvisited states occupy the labelled zero row without a pseudocount. The display includes $\pi(x) \geq 10^{-7}$; all states enter the reported empirical TV and unvisited target mass. Empirical TV and unvisited-mass ranges span training seeds; "Exact draws" gives the central 95% range of empirical TV across 200 multinomial replicates. These diagnostics use no importance weights.

J Neural Swap Graveyard

Table J.1 records six exploratory comparisons that were not carried into the final recipe. The interval head's bilinear option uses only one ordering, whereas the separate factorised head supports multiple causal orderings. The former therefore cannot replace the two-sweep interval head without changing its information.

For the factorised construction, a per-site global sum offered a cheap context that excludes both exchanged values:

$$H_{ij} = \underbrace{\sum_{r=1}^{R} a_r(P_i, i) \odot b_r(S_j, j)}_{\text{bilinear: exterior}} + \underbrace{\rho\Big(\mathrm{LN}\big(\textstyle\sum_k \psi_k - \psi_i - \psi_j\big)\Big)}_{\text{global: interior}} + \tau(t), \tag{J.1}$$

The rank-R bilinear term combines causal summaries. In the global term, ψ_k depends only on site k and its position, so subtracting $\psi_i + \psi_j$ makes the sum blind to the two holes.

Table J.1. Exploratory head and training comparisons. ESS fractions are raw unless marked EMA; the resampling row reports event counts and weight variance. The last row pools diagnostic runs.

arm	what changed	rung	reading
bilinear interval exterior, rank 8	pair MLP → bilinear product	8×8	0.758 vs 0.735 for the interval head, one seed; the implemented option uses one ordering
per-site global interior	band → $\sum_k \psi_k - \psi_i - \psi_j$	4×4, σ_c	one low-ESS seed of three (0.53 against siblings ≥ 0.75)
whole-lattice window	exclusion $i < k < j \to k \notin \{i, j\}$	4×4 / 8×8	higher mean at the gate (0.982 vs 0.975); overlapping seed ranges one rung up (0.831±0.030 vs 0.846±0.022, EMA)
periodic rotary code, $p \in \{1, 2\}$	backbone position code	4×4	inside the parent's seed spread at both p
rollout resampling, $\tau \in \{0.3, 0.6\}$	measure of the replay buffer	8×8	fires 8 and 34 times in 500 cycles; per-site weight variance within 1.5% of the parent
compiled factorised chassis at σ_c	`torch.compile` on the head	4×4	5 of 12 seeds collapse vs 0 of 21 eager (Fisher $p = 0.003$)

Table J.2. Selected historical 16×16 trials at the legacy coupling $\sigma = 0.223$, $c = 0.5$, training seed 42 and 5000 evaluation draws. These precede the multi-seed production comparisons at σ_c. All use trajectory batch 512 and 128 grid points. The factorised head uses two causal orderings; CV denotes the Stein control-variate estimator from the start of training.

Recipe	Updates	Raw ESS	EMA ESS
Factorised, width 128, depth 3, naive estimator	50k	0.007	0.014
Same, learning rate 3×10^{-4} throughout	50k	0.072	0.149
Factorised, width 32, depth 2, CV	70k	0.257	0.381
Factorised, width 128, depth 3, CV	70k	0.319	0.430
Same, learning rate 3×10^{-4} throughout[†]	70k	0.377	0.525
Two-hole patch, $R = 1$, CV	70k	0.501	0.625
Same, with exact-field channel	70k	0.557	0.583
Two-hole patch, $R = 2$, CV	70k	0.752	0.760

[†]This run changed execution venue during training, so its difference from the parent cannot be attributed solely to the learning rate.

K Supplementary Hard Results

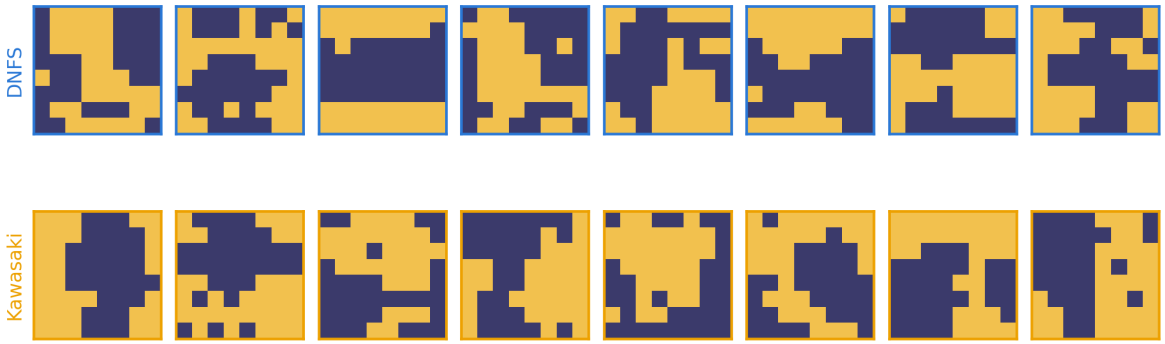


Figure K.1. Fixed-composition draws on the 8×8 lattice at σ_c, $c = 0.5$; gold tiles are spin $+1$, indigo spin -1. Top, blue frames: eight draws from the two-hole patch head of Table 5.4, selected by systematic resampling on their importance weights, approximating the target distribution. Bottom, amber frames: the final configuration of eight of the certified Kawasaki reference chains. Every tile on both rows holds exactly 32 up-spins. Quantitative marginal comparisons appear in the 8×8 column of Figure 5.8.

		16×16			8×8, σ_c	
c	N_A (16×16)	zero-shot, σ_c	zero-shot, $\sigma' = 0.1006$	amortised	zero-shot	amortised
0.25	64	$0.019 \pm 0.006^{\dagger}$	0.779 ± 0.055	0.848 ± 0.013	0.659 ± 0.019	0.943 ± 0.001
0.375	96	$0.276 \pm 0.031^{\dagger}$	0.927 ± 0.008	0.859 ± 0.007	0.841 ± 0.014	0.949 ± 0.002
0.5	128	0.824 ± 0.010	0.997 ± 0.001	0.809 ± 0.014	0.930 ± 0.002	0.923 ± 0.002

Table K.1. Composition transfer: ESS fractions from 5000 draws per seed and EMA checkpoints, mean ± SD over three seeds unless marked. Zero-shot samplers were trained at $c = 0.5$; amortised samplers uniformly on five slices spanning 0.3125–0.5, so $c = 0.25$ is extrapolation. Training follows Section 5.6. The 8×8 sparse probes use different draws from the dense series in Figure 5.10; the 16×16 rows are the corresponding points of that figure's dense grid. †These cells depart substantially from the log-normal approximation $\mathrm{ESS}/N \approx \exp(-\mathrm{Var}[\log w])$.

K.1 Evaluation tables at both couplings

	$\sigma = 0.1$					$\sigma = \sigma_c$				
Method	ESS↑	ΔMag↓	ΔCorr↓	EW_2 ↓	FLOP/es↓	ESS↑	ΔMag↓	ΔCorr↓	EW_2 ↓	FLOP/es↓
exact enumeration (reference)	/	0	0	0	–	/	0	0	0	–
sampling floor at $N = 512$	/	7.2	10.9	6.5	–	/	8.1	13.0	7.0	–
Kawasaki (mchammer), run long	/	2.0 ± 0.5	3.1 ± 0.8	3.1 ± 0.5	6.2×10^2	/	3.1 ± 1.1	5.4 ± 2.0	4.5 ± 1.6	1.6×10^3
doubly-masked oracle†	0.997 ± 0.001	8.5 ± 3.5	11.6 ± 0.2	6.6 ± 1.5	2.9×10^{10}	0.967 ± 0.011	7.7 ± 0.3	12.1 ± 4.5	7.3 ± 2.6	2.9×10^{10}
mask-one head	0.997 ± 0.000	8.0 ± 3.9	12.2 ± 1.1	7.1 ± 1.5	3.7×10^9	0.975 ± 0.006	8.8 ± 2.0	11.5 ± 3.4	8.1 ± 2.9	3.8×10^9
masked-attention band, one sweep	0.997 ± 0.001	6.9 ± 1.5	11.2 ± 2.5	6.7 ± 2.1	8.4×10^8	0.975 ± 0.003	$\mathbf{6.0 \pm 2.3}$	12.9 ± 2.1	$\mathbf{5.7 \pm 2.2}$	8.6×10^8
masked-attention band, two sweeps	0.997 ± 0.003	$\mathbf{5.4 \pm 2.7}$	10.7 ± 1.4	6.5 ± 0.6	1.2×10^9	0.983 ± 0.009	9.6 ± 4.9	11.2 ± 5.0	7.1 ± 2.8	1.3×10^9
masked-attention band, two sweeps + exact field	0.999 ± 0.000	6.3 ± 2.6	9.7 ± 0.2	6.0 ± 0.3	1.2×10^9	0.983 ± 0.002	7.8 ± 4.4	10.3 ± 1.7	5.9 ± 1.3	1.3×10^9
prefix-sum band, one sweep	0.997 ± 0.001	7.2 ± 3.6	10.4 ± 2.1	5.8 ± 1.4	7.7×10^8	0.892 ± 0.118	10.0 ± 1.7	11.8 ± 3.1	7.9 ± 2.5	8.8×10^8
prefix-sum band, two sweeps	0.998 ± 0.001	7.1 ± 3.1	10.5 ± 3.1	$\mathbf{5.4 \pm 1.3}$	1.2×10^9	0.963 ± 0.030	8.1 ± 5.6	13.1 ± 3.6	8.1 ± 2.5	1.2×10^9
prefix-sum band, two sweeps + exact field	0.999 ± 0.000	6.0 ± 2.9	12.1 ± 2.7	5.5 ± 0.1	1.2×10^9	0.981 ± 0.003	8.3 ± 3.7	$\mathbf{9.7 \pm 0.5}$	7.3 ± 1.4	1.2×10^9
two-hole patch head	$\mathbf{1.000 \pm 0.000}$	6.4 ± 3.1	$\mathbf{9.1 \pm 2.0}$	5.8 ± 0.5	$\mathbf{2.6 \times 10^8}$	$\mathbf{0.992 \pm 0.006}$	6.3 ± 1.3	17.3 ± 2.8	8.9 ± 1.0	$\mathbf{2.6 \times 10^8}$
GFlowNet, trajectory balance	0.997 ± 0.001	5.3 ± 0.7	12.3 ± 3.2	6.3 ± 0.5	3.2×10^6	0.991 ± 0.002	7.4 ± 1.9	14.3 ± 4.1	8.1 ± 1.0	3.2×10^6
GFlowNet, forward-looking DB	0.993 ± 0.003	5.4 ± 1.6	12.5 ± 3.2	6.6 ± 0.5	3.2×10^6	0.972 ± 0.005	8.5 ± 2.5	13.1 ± 3.2	6.3 ± 1.5	3.2×10^6

Table K.2. Sampler evaluation on 4×4 Ising at fixed composition $c_{\mathrm{target}} = 0.5$. Neural rows use raw checkpoints and 512 draws per seed, reported as mean ± SD over three seeds. The reference is exact enumeration; the floor row gives the expected error at $N = 512$. Error columns are in units of 10^{-2}. Columns and cost conventions follow Section 5.4; "/" means not applicable and "–" unavailable. †The doubly-masked oracle is evaluated eagerly.

	$\sigma = 0.1$					$\sigma = \sigma_c$				
Method	ESS↑	ΔMag↓	ΔCorr↓	EW_2 ↓	FLOP/es↓	ESS↑	ΔMag↓	ΔCorr↓	EW_2 ↓	FLOP/es↓
Kawasaki (mchammer), certified reference	/	(0.8)	(1.2)	(0.4)	3.3×10^3	/	(2.7)	(2.8)	(0.5)	8.4×10^3
sampling floor at $N = 5000$	/	3.6	5.8	1.4	–	/	4.8	6.9	1.4	–
mask-one head	0.995 ± 0.000	$\mathbf{3.4 \pm 0.1}$	7.0 ± 0.6	1.6 ± 0.2	7.2×10^{10}	0.903 ± 0.022	5.1 ± 1.3	7.7 ± 0.9	1.5 ± 0.1	8.0×10^{10}
masked-attention band, one sweep	0.984 ± 0.001	3.7 ± 0.2	6.2 ± 0.5	1.7 ± 0.4	1.7×10^{10}	0.781 ± 0.016	6.3 ± 1.2	8.4 ± 1.3	1.6 ± 0.2	2.1×10^{10}
masked-attention band, two sweeps	0.995 ± 0.001	3.7 ± 0.3	6.7 ± 0.1	1.5 ± 0.1	2.2×10^{10}	0.908 ± 0.003	5.0 ± 1.6	8.8 ± 1.3	$\mathbf{1.4 \pm 0.4}$	2.4×10^{10}
masked-attention band, two sweeps + exact field	0.995 ± 0.001	3.9 ± 0.6	6.3 ± 0.2	1.4 ± 0.1	2.2×10^{10}	0.918 ± 0.005	4.7 ± 0.9	$\mathbf{6.3 \pm 0.0}$	1.7 ± 0.2	2.4×10^{10}
prefix-sum band, one sweep	0.836 ± 0.214	3.7 ± 0.1	7.3 ± 1.2	1.8 ± 0.3	1.7×10^{10}	0.735 ± 0.014	6.5 ± 1.1	8.4 ± 0.6	2.0 ± 0.1	1.7×10^{10}
prefix-sum band, two sweeps	0.995 ± 0.000	3.8 ± 0.5	6.8 ± 0.6	$\mathbf{1.3 \pm 0.1}$	1.8×10^{10}	0.889 ± 0.004	$\mathbf{3.6 \pm 1.6}$	8.1 ± 1.9	1.6 ± 0.1	2.0×10^{10}
prefix-sum band, two sweeps + exact field	0.996 ± 0.001	3.9 ± 0.4	6.3 ± 0.3	1.6 ± 0.1	1.8×10^{10}	$\mathbf{0.931 \pm 0.003}$	4.6 ± 0.3	6.4 ± 1.1	2.0 ± 0.1	1.9×10^{10}
two-hole patch head	$\mathbf{0.999 \pm 0.000}$	4.0 ± 0.4	$\mathbf{5.7 \pm 0.7}$	1.6 ± 0.2	$\mathbf{3.0 \times 10^9}$	0.921 ± 0.003	6.6 ± 0.9	7.4 ± 1.9	1.9 ± 0.2	$\mathbf{3.2 \times 10^9}$
GFlowNet, trajectory balance	0.994 ± 0.002	3.8 ± 1.0	5.8 ± 0.5	1.6 ± 0.5	1.3×10^7	0.955 ± 0.005	7.0 ± 1.1	7.7 ± 2.4	1.5 ± 0.2	1.3×10^7
GFlowNet, forward-looking DB	0.973 ± 0.004	3.8 ± 0.8	6.0 ± 0.1	1.5 ± 0.3	1.3×10^7	$0.767 \pm 0.024^{\dagger}$	6.7 ± 1.8	9.3 ± 0.4	1.6 ± 0.3	1.6×10^7

Table K.3. Sampler evaluation on 8×8 Ising at fixed composition $c_{\text{target}} = 0.5$. Neural rows use raw checkpoints and 5000 draws per seed, reported as mean ± SD over three seeds. Error columns are in units of 10^{-2}. Parentheses give the Kawasaki reference's approximate uncertainty; the floor row gives the expected error at $N = 5000$. Columns and cost conventions follow Section 5.4; "/" means not applicable. †Doubling the forward-looking DB training horizon to 100k steps raises its critical raw ESS to 0.900 ± 0.019.

	$\sigma = 0.1$					$\sigma = \sigma_c$				
Method	ESS↑	ΔMag↓	ΔCorr↓	EW_2 ↓	FLOP/es↓	ESS↑	ΔMag↓	ΔCorr↓	EW_2 ↓	FLOP/es↓
Kawasaki (thesis engine), certified reference	/	(0.9)	(1.7)	(0.1)	7.4×10^4	/	(5.7)	(5.1)	(0.2)	3.4×10^5
sampling floor at $N = 5000$	/	5.3	8.8	0.5	–	/	8.5	11.9	0.6	–
masked-attention band, one sweep†	0.862 ± 0.057	6.5 ± 0.6	10.5 ± 0.1	0.7 ± 0.0	2.4×10^{11}	0.015 ± 0.019	236.2 ± 202.2	388.3 ± 271.0	24.8 ± 17.8	3.6×10^{14}
masked-attention band, two sweeps	0.981 ± 0.012	$\mathbf{4.6 \pm 0.6}$	8.9 ± 0.7	0.8 ± 0.1	2.4×10^{11}	0.759 ± 0.030	8.3 ± 1.6	27.0 ± 4.2	1.7 ± 0.1	3.2×10^{11}
+ exact field	0.980 ± 0.013	4.8 ± 0.3	9.3 ± 0.6	0.6 ± 0.1	2.4×10^{11}	0.812 ± 0.010	11.1 ± 2.0	$\mathbf{21.3 \pm 1.1}$	$\mathbf{1.5 \pm 0.1}$	3.0×10^{11}
prefix-sum band, one sweep	0.756 ± 0.264	7.0 ± 1.9	11.1 ± 2.1	0.8 ± 0.2	1.5×10^{11}	0.062 ± 0.041	48.9 ± 39.4	280.4 ± 72.2	26.9 ± 5.8	8.0×10^{12}
prefix-sum band, two sweeps†	0.640 ± 0.452	73.3 ± 96.6	77.6 ± 96.4	2.0 ± 1.9	5.3×10^{13}	0.732 ± 0.023	9.0 ± 2.2	24.5 ± 2.8	1.6 ± 0.1	1.9×10^{11}
+ exact field	0.988 ± 0.003	5.3 ± 0.8	9.1 ± 0.6	0.7 ± 0.1	1.4×10^{11}	0.823 ± 0.005	$\mathbf{7.4 \pm 2.1}$	24.4 ± 2.4	1.7 ± 0.1	1.6×10^{11}
two-hole patch head	0.998 ± 0.001	5.4 ± 0.5	8.9 ± 0.1	0.6 ± 0.0	$\mathbf{3.8 \times 10^{10}}$	0.638 ± 0.029	9.8 ± 2.9	23.8 ± 5.1	1.8 ± 0.1	6.0×10^{10}
$R = 2$	$\mathbf{1.000 \pm 0.000}$	5.3 ± 0.3	$\mathbf{8.8 \pm 0.4}$	$\mathbf{0.5 \pm 0.1}$	4.3×10^{10}	$\mathbf{0.826 \pm 0.015}$	9.6 ± 1.9	22.9 ± 2.2	1.8 ± 0.2	$\mathbf{5.3 \times 10^{10}}$
GFlowNet, trajectory balance	0.877 ± 0.057	5.7 ± 0.5	9.1 ± 0.4	0.6 ± 0.0	6.5×10^7	0.883 ± 0.018	9.0 ± 2.2	15.5 ± 2.7	0.7 ± 0.1	6.4×10^7
GFlowNet, forward-looking DB	0.002 ± 0.002	189.7 ± 130.3	209.9 ± 51.1	8.2 ± 3.6	8.2×10^{10}	0.001 ± 0.001	394.5 ± 157.6	461.1 ± 125.9	9.1 ± 3.3	6.2×10^{10}

Table K.4. Sampler evaluation on 16×16 Ising at fixed composition $c_{\text{target}} = 0.5$: EMA checkpoints, 5000 draws per seed, mean ± SD over three seeds unless marked. Error columns are in units of 10^{-2}; parentheses give the Kawasaki reference's approximate uncertainty. The floor row gives the expected error at $N = 5000$. Columns and cost conventions follow Section 5.4; "/" means not applicable and "–" unavailable. †Two-seed means: masked-attention, one sweep at $\sigma = 0.1$, and prefix-sum, two sweeps at σ_c, each excluding one collapsed seed.

	$\sigma = 0.1$					$\sigma = \sigma_c$				
Method	ESS↑	ΔMag↓	ΔCorr↓	EW_2 ↓	FLOP/es↓	ESS↑	ΔMag↓	ΔCorr↓	EW_2 ↓	FLOP/es↓
Kawasaki (thesis engine), certified reference	/	(1.1)	(1.9)	(0.1)	1.1×10^5	/	(8.7)	(7.0)	(0.2)	8.2×10^5
sampling floor at $N = 5000$	/	6.0	10.0	0.4	–	/	10.2	14.0	0.4	–
two-hole patch head, $R = 2$	$\mathbf{0.999 \pm 0.000}$	6.1 ± 0.1	10.6 ± 0.7	0.5 ± 0.1	$\mathbf{9.9 \times 10^{10}}$	0.702 ± 0.025	14.3 ± 3.6	35.2 ± 1.1	2.4 ± 0.1	$\mathbf{1.4 \times 10^{11}}$
two-hole patch head, $R = 3$	0.999 ± 0.001	$\mathbf{5.9 \pm 0.6}$	$\mathbf{10.2 \pm 0.5}$	$\mathbf{0.4 \pm 0.1}$	1.2×10^{11}	$\mathbf{0.798 \pm 0.004}$	$\mathbf{10.6 \pm 4.3}$	$\mathbf{32.3 \pm 1.8}$	$\mathbf{2.2 \pm 0.2}$	1.4×10^{11}
GFlowNet, trajectory balance‡	0.357 ± 0.156	12.2 ± 5.5	19.0 ± 5.8	0.5 ± 0.1	3.8×10^8	0.735 ± 0.068	10.5 ± 1.2	18.3 ± 1.6	0.5 ± 0.1	1.4×10^8
GFlowNet, forward-looking DB	–	–	–	–	–	–	–	–	–	–

Table K.5. Sampler evaluation on 20×20 Ising at fixed composition $c_{\text{target}} = 0.5$: EMA checkpoints, 5000 draws per seed, mean ± SD over three seeds. Error columns are in units of 10^{-2}; parentheses give the Kawasaki reference's approximate uncertainty. The floor row gives the expected error at $N = 5000$. Columns and cost conventions follow Section 5.4; "/" means not applicable and "–" unavailable. Forward-looking DB was not run at this size after failing at 16×16. ‡Budget-matched comparator: same 100k-update budget as the swap heads at σ_c (50k at $\sigma = 0.1$), training ESS still rising at both couplings.

K.2 The free-energy surface

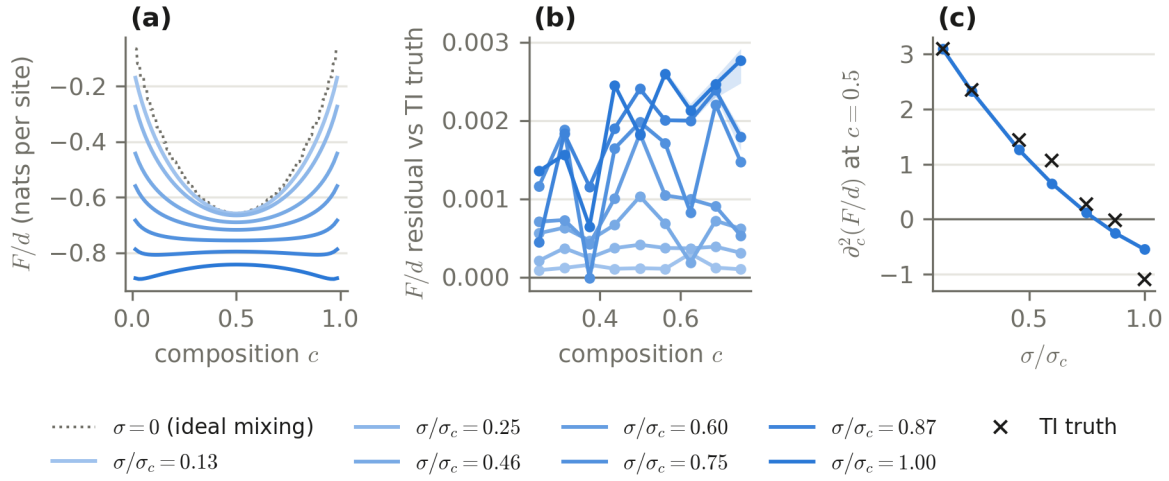


Figure K.2. Composition–coupling estimates from the amortised 8×8 hard sampler, three training seeds, frozen EMA checkpoints. (a) Free energy in nats/site at seven couplings, with min–max seed bands and the ideal-mixing limit $-\log\binom{d}{N_A}/d$ (dotted). The model is evaluated on all 63 interior composition slices. (b) Residuals against thermodynamic-integration references at seven compositions. These use the native $n_e = 128$ grid; finite-grid and sampling errors prevent a bound guarantee. (c) Central curvature at $c = 0.5$, using a composition step $1/16$; crosses are the corresponding thermodynamic-integration finite differences. Differencing amplifies estimation errors, so the sign change is descriptive.

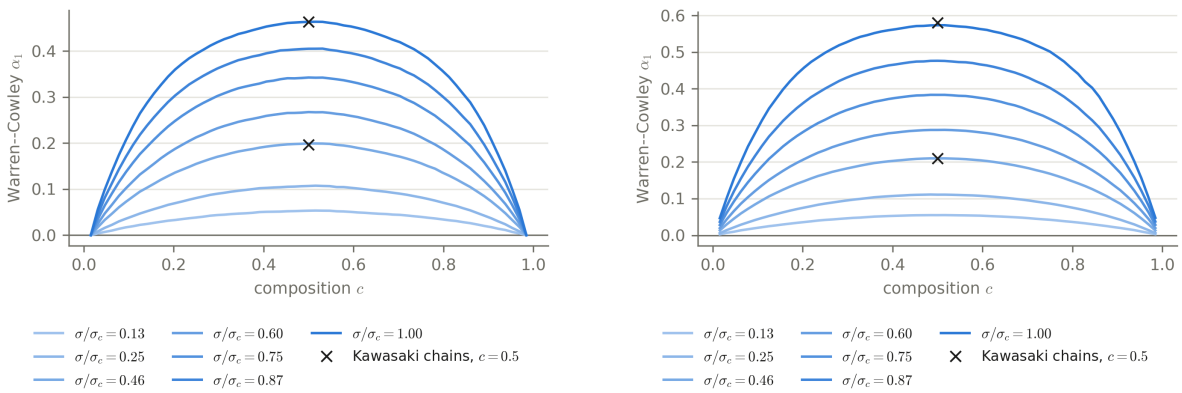


Figure K.3. Short-range order from the same archived importance-weighted draws: 8×8 (left) and 16×16 (right), three seeds each; every plotted composition is a measured slice. The plotted parameter is corrected for the finite fixed-composition random-mixing baseline, $\alpha_1 = 1 - (1 - \langle x_i x_j \rangle)(d-1)/[4c(1-c)d]$; positive values indicate like-neighbour clustering. Composition is exact on every draw; the spin-product averages use importance weights. Crosses are certified Kawasaki references at $c = 0.5$, σ_c and $\sigma = 0.1$ (the sampler grid's nearest coupling is 0.1006).

K.3 The separable band contraction

The band's rather scary looking $\mathcal{O}(d^3)$ is entirely removable; the pair query of (5.19) is linear on a concatenation, so it splits, and the score is an outer sum,

$$W_q(\rho_i, \rho_j) \;=\; W_1\rho_i + W_2\rho_j + b_q \qquad \Longrightarrow \qquad s_{ij,k} \;=\; a_{ik} + b_{jk}. \tag{K.1}$$

The visible set factorises the same way; $k \in \mathcal{V}_{ij}$ is a product $u_{ik}w_{jk}$ under the interval predicate and under the whole-lattice one alike, so writing $\alpha_{ik} = u_{ik}e^{a_{ik}}$ and $\beta_{jk} = w_{jk}e^{b_{jk}}$, both halves of

the masked softmax are matrix products over the band axis,

$$Z_{ij} \;=\; \sum_k \alpha_{ik}\beta_{jk}, \qquad M_{ij} \;=\; \frac{1}{Z_{ij}} \sum_k \alpha_{ik}\beta_{jk}\, v_k. \tag{K.2}$$

The query bias is absorbed into a_{ik}, and $M_{ij} = 0$ on an empty band. The (B, d^2, n) score tensor is never materialised: outputs and parameter gradients match the dense form to round-off, and the head-forward FLOPs fall 1.98× at $d = 256$. The usual stabilising shift is omitted, because a shift that depends on an excluded partner breaks exact equivalence; this is safe while $\max_k |a_{ik}| + \max_k |b_{jk}|$ stays inside the fp32 exponent budget (at most 17 on a trained checkpoint, against 87). It's not necessarily true however that equal functions give equal training runs – the separable-trained twin lost two of three seeds at the floor where the dense run lost one, and the one-sweep rows were trained densely and billed separable (Table L.5).

K.4 The swap log-ratio in closed form

Let $\mathbf{u}_i$ denote the ith coordinate vector. Swapping sites i, j with $a = x_j - x_i$ gives

$$v = a(\mathbf{u}_i - \mathbf{u}_j), \qquad y = x + v. \tag{K.3}$$

For the symmetric, zero-diagonal adjacency matrix A of Section 2.1.3, write $h = Ax$ for the neighbour fields. Expanding the quadratic term gives

$$\begin{aligned} y^T A y - x^T A x &= (x+v)^T A (x+v) - x^T A x \\ &= 2v^T A x + v^T A v \\ &= 2a(h_i - h_j) + a^2(A_{ii} - A_{ij} - A_{ji} + A_{jj}) \\ &= 2(x_j - x_i)(h_i - h_j) - 2(x_j - x_i)^2 A_{ij}. \end{aligned} \tag{K.4}$$

The slice-uniform base and magnetisation term $b\sum_k x_k$ are swap-invariant. Multiplying by $t\sigma$ gives the path log-ratio:

$$\log \frac{\tilde{p}_t(y_{ij})}{\tilde{p}_t(x)} = t\,\sigma \Big[\, 2(x_j - x_i)(h_i - h_j) - 2(x_j - x_i)^2 A_{ij} \,\Big]. \tag{K.5}$$

The shared bond $x_i x_j$ is unchanged. Excluding each partner gives $\tilde{h}_i = h_i - A_{ij}x_j$, $\tilde{h}_j = h_j - A_{ij}x_i$, hence $\tilde{h}_i - \tilde{h}_j = h_i - h_j - A_{ij}(x_j - x_i)$. Substitution recovers (5.9); exponentiation gives (5.10).

K.5 Learned escape rates

Table K.6. Means over the last ten in-training evaluations of each of three completed seeds. Λ is the whole-lattice escape rate per unit sampler time. Rate diagnostics use training minibatches across path times and raw parameters; realised swap counts use rollout batches. The last three columns describe the critical runs: the one-event clip fraction is the fraction of states with $\Lambda\,\mathrm{d}t > 1$, a hypothetical diagnostic for matching updates. The 4×4 and 8×8 runs use one-event updates with 100 and 128 Euler steps, respectively; larger lattices use 128 matching steps. "—" means realised swap counts were not logged; "/" marks the 24×24 rung, run at the critical coupling only.

lattice	head	Λ, critical	Λ, $\sigma = 0.1$	p99 of $\Lambda\,\mathrm{d}t$	one-event clip	swaps per site per step
4×4	two-hole patch, $R = 1$	2.4	1.0	0.06	0	—
8×8	masked-attention band, 2 sweeps + exact field	11.5	4.2	0.18	0	—
16×16	two-hole patch, $R = 2$	61.9	17.3	1.12	0.03	0.0019
20×20	two-hole patch, $R = 3$	99.5	26.7	1.94	0.25	0.0019
24×24	two-hole patch, $R = 4$	150.5	/	3.04	0.45	0.0020

L Hyperparameter Soup

L.1 Shared Ising DNFS settings

Table L.1. Defaults for the reported Ising DNFS runs, with chapter-specific overrides below. The GFlowNet and alloy recipes are described separately.

setting	value	note
optimisation		
optimiser	AdamW	weight decay 10^{-4}
learning rate	10^{-3}	per-stage override in the curriculum
warmup steps	500	linear, at step 0 only; not re-fired at curriculum stages
gradient-clip norm	500	
inner (loss) batch	128	
seeds	42 to 45	
EMA decay	off	overridden below where used
loop shape		
inner steps per outer	100	
outer batch	= inner batch	rollout trajectories per outer cycle; per chapter table where larger
replay flush on stage	on	
estimator		
estimator	Stein control variate	coefficient $\beta = 1$, Equation 3.5
control-variate batch	= outer batch	
c_t EMA half-life	off	
c_t from rollout	on	bit-identical grid from the rollout's own forwards; off in the archived $\sigma = 0.1$ cells of Chapter 3
log-ratio clamp	5	as the paper
integrator		
time grid	uniform	
matching step	off	Chapter 5 only
evaluation		
frozen-eval draws	5000	per frozen DNFS evaluation
in-training eval draws	= frozen-eval draws	
SMC resampling threshold	off	off unless a row says otherwise
execution		
precision	fp32	hard-run diagnostic autocast is listed below
model compilation	on	off in the archived $\sigma = 0.1$ cells of Chapter 3

L.2 The unconstrained baseline (Chapter 3)

Table L.2. What the unconstrained runs of Chapter 3 set against Table L.1. The parent paper's recipe for the same 10×10 lattice [29, App. E.1]: 100k steps, batch 128, width 128, 64 Euler steps, AdamW at 10^{-3}, log-ratio clamp 5, uniform base, no warmup and no curriculum at either coupling. The critical 10×10 cell was re-run late in the project on the house recipe of Chapter 5; its rows below replace the original 200k-step run.

setting	value	applies to
target		
lattice sites d	16 / 100	4×4 / 10×10
coupling σ	0.1 / σ_c	$\sigma_c = \ln(1+\sqrt{2})/4 = 0.220343$
external field	0	

continued on the next page

Table L.2, continued

setting	value	applies to
model		
architecture	leTF	
trunk width	64 / 128	4×4 / 10×10
depth	3	
attention heads	4	leTF only
patch size	1	
budget		
training steps	10k / 50k / 100k	4×4 / 10×10 at $\sigma = 0.1$ / 10×10 at σ_c
inner (loss) batch	512	10×10 at σ_c; the outer batch equals it
outer batch	256	10×10 at $\sigma = 0.1$
replay retention	8 / 4 / 8	4×4 / 10×10 at $\sigma = 0.1$ / 10×10 at σ_c
Euler steps	50 / 64 / 128	4×4 / 10×10 at $\sigma = 0.1$ / 10×10 at σ_c
eval interval	200 / 500	4×4 / 10×10
in-training eval draws	512	10×10 at σ_c; diagnostic only, the frozen eval draws 5000
EMA decay	0.9999	10×10 at σ_c; tracked, the printed row is the raw weights
exact-field channel	on	10×10 at σ_c, Equation 3.11; off elsewhere in this chapter
curriculum (10×10 at σ_c only)		
stage boundaries	0, 5k, 10k, 15k, 20k, 25k, 30k	$\sigma = 0.100, 0.140, 0.170, 0.190, 0.205, 0.215, \sigma_c$; held at σ_c from 30k
per-stage learning rate	10^{-3}, 3×10^{-4}	first four stages / last three
re-warmup on stage	off	

L.3 The soft-constrained sampler (Chapter 4)

Table L.3. What the soft-constrained runs of Chapter 4 set against Table L.1; the baseline architecture rows of Table L.2 carry over unless named here.

setting	value	applies to
penalty		
target composition c	0.25 / 0.375 / 0.5	the three trained windows
penalty strength λ	50	10 / 100 twins at the centre window
base composition	$= c$ / 0.5	matched-base rows and the conditioned sampler / every other cell
exact-field channel	on, gain init 0	off in the no-channel controls
λ curriculum	off	annealed arm only: $\lambda = 10$, 25, 50 at 0, 10k, 20k
budget and recipe deltas		
training steps	10k / 50k	4×4 / 8×8
outer batch	128 / 256	4×4 / 8×8
learning-rate warmup	500 / 2000	4×4 / 8×8
Euler steps	50 / 128	4×4 / 8×8
trunk width	64 / 128	4×4 / 8×8
replay retention	8 / 4	4×4 / 8×8
weight averaging	0.9999	off in the earlier 4×4, $\lambda = 100$ control
c_t from rollout	on	off in the earlier 4×4, $\lambda = 100$ control
compiled model	on	except the earlier 4×4, $\lambda = 100$ control
σ ladder	0.10, 0.14, 0.17, 0.19, 0.205, 0.215, σ_c at 0, 5k, ..., 30k	ladder arms only; learning rate $10^{-3} \to 3 \times 10^{-4}$ at 20k
amortisation over c		
conditioning on c	on	conditioned sampler only

continued on the next page

Table L.3, continued

setting	value	applies to
enumerated values	$\{0.25, 0.375, 0.5\}$ / $\{0.25, 0.25+\frac{1}{64}, \ldots, 0.5\}$	three values at 4×4 / seventeen at 8×8

L.4 The hard-constrained sampler (Chapter 5)

L.4.1 Averaging and evaluation details

Raw and EMA parameters are evaluated where averaging is enabled (Table L.4); 4×4 has no EMA. Polyak–Ruppert arithmetic averaging motivates parameter averaging [103, 104], without establishing the behaviour of our EMA. The implemented EMA uses the MDNS/MetaDNS decay [31, 49]. Training diagnostics use fewer draws in reduced precision; the frozen fp32 evaluations determine the reported tables.

The 8×8 reference uses 15 non-local Kawasaki chains of 10^6 trials in `mchammer`, discarding 20% as burn-in, with energy $\hat{R} \leq 1.001$; larger lattices use the thesis's swap engine. The separate 4×4 Kawasaki comparator uses 15 chains of 200k trials and 20% burn-in. Its roughly 16-fold larger record pool explains errors below the 512-draw floor. GFlowNets use the first 512 stored draws at 4×4 and all 5000 above it.

Separable attention and KV-cached GFlowNet rollouts preserve the evaluated functions while reducing sampling cost. This does not imply identical training trajectories (Appendix K.3). Reference uncertainty can be comparable to critical differences between heads; the main-table extrema are descriptive comparisons.

Tables L.4 and L.5 specify the Ising swap runs printed in the main evaluation tables. All use the uniform base on the requested composition slice, the control-variate estimator, eight replay cycles and 100 inner updates per outer cycle. The outer batch and the control-variate batch equal the trajectory batch; the control-variate grid is taken from the rollout, without additional temporal averaging.

Table L.4. Size-dependent settings of the reported Ising swap runs. Training steps at the weaker and critical couplings are separated where they differ; the 24×24 rung was run at σ_c only.

setting	4×4	8×8	16×16	20×20	24×24
training steps, $\sigma = 0.1$	10k	50k	50k	50k	–
training steps, $\sigma = \sigma_c$	10k	50k	100k	100k	100k
trajectory batch	128	128	512	512	512
loss microbatch	–	–	–	–	128
uniform Euler grid points	100	128	128	128	128
simulation step	one event	one event	matching	matching	matching
EMA decay	off	0.9999	0.9999	0.9999	0.9999
main-table parameters	raw	raw	EMA	EMA	EMA
frozen evaluation draws	512	5000	5000	5000	5000
diagnostic draws	512	512	256	256	256
diagnostic interval (updates)	200	200	500	500	500
diagnostic precision	fp32	bf16	bf16	bf16	bf16
training precision	fp32	fp32	fp32	fp32, bf16	bf16
patch radius R	1	1	1, 2	2, 3	3, 4

The specialist runs train at $c = 0.5$ with seeds 42–44; the main-table captions identify omitted seeds. At 4×4, both couplings are trained directly. Larger runs at σ_c use the ladder $0.10, 0.14, 0.17, 0.19, 0.205, 0.215, \sigma_c$, starting at updates $0, 5, 10, 15, 20, 25, 30$k. The learning rate

falls from 10^{-3} to 3×10^{-4} at 20k. Runs at $\sigma = 0.1$ have no curriculum. Warmup lasts 500 updates and is not restarted at stage changes; replay is flushed.

Table L.5. Architecture and execution settings for the retained swap heads. The prefix-sum head is the `interval` implementation; it is distinct from the retired factorised head.

component	settings
backbone	leTF, width 32, depth 2, four attention heads and single-site tokens. The patch head uses the embeddings and time map without the causal trunk.
mask-one / oracle	all anchors together (no anchor chunk); oracle at 4×4 only.
both band heads	feature width 16, pair offsets $(1, D)$ for lattice edge D, MLP exterior combiner, readout scale 1. One sweep uses row order; two sweeps add column order.
masked attention	attention width 32, interval window and absolute position code; no stencil. The named 4×4 window probe uses the whole lattice.
attention contraction	dense in the original one-sweep runs and 4×4 ladder; separable for the 8×8 weaker-coupling two-sweep runs and the 16×16 two-sweep runs. Frozen FLOP accounting uses the equivalent separable form (Section 5.4).
exact-field channel	off unless the main-table row names it; gain initialised at 0.
two-hole patch	patch hidden width 32, feature width 32; pooled box radii are powers of two with $2r + 1 \leq D$, plus a global pool. The local radius R is listed in Table L.4.
execution	Training precision follows Table L.4; frozen swap evaluation uses fp32. Compile the complete head, except the eager oracle; no separate backbone compilation. At 16×16, band losses use microbatches of 128. Frozen chunks: 256 at 8×8; 128 for 16×16 bands, 512 for the larger patch heads.

Composition amortisation. The patch models use the same size-dependent recipe at σ_c, replacing the fixed training count by a uniform per-trajectory mixture over $c \in \{0.3125, 0.375, 0.4375, 0.46875, 0.5\}$. There is no explicit composition input. The 8×8 experiment uses seeds 42–44 and $R = 1$; the 16×16 experiment uses seed 42 and $R = 2$. Transfer evaluations use EMA parameters and 5000 draws per requested slice. The free-energy comparison additionally evaluates the 8×8 models on a 256-point grid.

GFlowNet comparator. These runs use a separate autoregressive policy: width 64 at 4×4 and 8×8, width 68 at 16×16, 72 at 20×20 and 76 at 24×24, two layers and four attention heads. Both objectives use batch 128, policy learning rate 10^{-3} and behaviour exploration $\epsilon = 0.05$; trajectory balance uses learning rate 0.1 for its scalar $\log Z$. The forward-looking detailed-balance head shares the policy trunk. Budgets, seeds and the critical-coupling ladder match the swap runs, with the ladder advanced every 5000 updates; the policy learning rate stays at 10^{-3}. At 4×4, warmup, clipping and EMA are off; larger runs use warmup 500, clip 500 and EMA decay 0.9999. Frozen evaluations use 5000 draws, fp32 at 4×4 and bf16 at the larger sizes, with exploration switched off.

Cu–Au hard samplers. The 16-site specialists and amortised model use mask-one with leTF width 64, depth 3 and four attention heads. They train for 50k updates with batch 128, 128 Euler grid points and one-event simulation. EMA decay is 0.9999, but the reported alloy tables use raw parameters. The exact-field channel and compilation are off; the control-variate grid is estimated separately from the rollout. Evaluation uses 5000 fp32 draws every 500 updates. The amortised model mixes uniformly over gold counts 4–8; its final 5000 draws are split across those counts.

The 500 K ladder has seven inverse temperatures evenly spaced from 1200 to 500 K, starting at updates $0, 7.1, 14.3, 21.4, 28.6, 35.7, 42.9$k. The learning rate is 10^{-3} at the first stage and 10^{-4} thereafter. The archived 64-site hard runs at 1200 and 680 K use width 128, depth 3 and the fcc patch head with two neighbour shells. They train for 10k and 30k updates with one and four

inverse-temperature stages, respectively; the four-stage starts are $0, 7.5, 15, 22.5$k. Diagnostic draws and evaluation chunks are 256.

The hard rows at 500 K in Table 5.8 share this head and width, trained on the seven-stage ladder above with the same budget and grid as the 16-site specialists.

Training FLOPs. Training adds to the sampling costs the main tables report, seen in Table L.6 which uses the measured per-forward cost and the backward charged at twice the forward [105]. The GFlowNet comparator, matched on updates rather than compute, trained on far fewer FLOPs than the patch heads.

Table L.6. Training cost in FLOP of every critical-coupling cell in the body tables, from `training_flops_table.py`: head forwards per recipe times the per-forward cost, backward at twice the forward, training only (the periodic in-training diagnostic is excluded). Masked-attention cells are billed under the separable contraction; † marks cells that trained densely, whose as-run cost is higher by 1.44× at 8×8 and 1.98× at 16×16. The 8×8 patch head and the 16×16 $R = 2$ patch head were measured end to end (`measure_training_flops.py`): measured over accounted 0.997 and 0.999 once the in-training diagnostic is included. One frozen 5000-draw evaluation of the same cell is 1/43 of every 50k swap recipe and 1/342 of every 100k one; for the GFlowNet recipes the ratios are 5100 and 10200 of their much cheaper rollouts.

head	8×8	16×16	20×20	24×24
mask-one head	1.5×10^{16}	–	–	–
masked-attention band, one sweep	$3.5 \times 10^{15\dagger}$	$3.5 \times 10^{17\dagger}$	–	–
masked-attention band, two sweeps	$4.6 \times 10^{15\dagger}$	4.1×10^{17}	–	–
+ exact field	$4.6 \times 10^{15\dagger}$	4.1×10^{17}	–	–
prefix-sum band, one sweep	2.7×10^{15}	1.7×10^{17}	–	–
prefix-sum band, two sweeps	3.8×10^{15}	2.3×10^{17}	–	–
+ exact field	3.8×10^{15}	2.3×10^{17}	–	–
two-hole patch head, $R = 1$	6.4×10^{14}	6.5×10^{16}	–	–
two-hole patch head, $R = 2$	–	7.4×10^{16}	1.7×10^{17}	–
two-hole patch head, $R = 3$	–	–	2.0×10^{17}	3.7×10^{17}
two-hole patch head, $R = 4$	–	–	–	4.4×10^{17}
GFlowNet, trajectory balance	3.2×10^{14}	2.9×10^{15}	5.1×10^{15}	8.2×10^{15}
GFlowNet, forward-looking DB	3.2×10^{14}	2.9×10^{15}	–	–